

Foundations of Position-Based Rigid Body Dynamics

Tom Waterson *

September 15, 2026

*Guildford, UK email:tom.watson@epicgames.com

Contents

1	Introduction	4
1.1	Motivation	4
1.2	Overview	4
	Notation	6
2	Rigid Body Configuration	8
2.1	Angular Velocity	9
2.1.1	The Path Ordered Exponential	9
2.1.2	Axis/Angle Representation and the Exponential Map	11
2.1.3	The Left Jacobian of $SO(3)$	11
2.2	Quaternion Representation*	14
2.2.1	Quaternions from the Path Ordered Exponential	15
2.2.2	Relation to Axis/Angle Representation	16
3	Equations of Motion	17
3.1	The Euler-Poincaré Method	18
3.2	Holonomic Constraints	20
3.3	Non-Holonomic Constraints	21
4	Solving the Constrained Equations of Motion	23
4.1	Solving the Unconstrained Equations of Motion	23
4.2	Finding a First Order Approximate Solution	24
4.3	An Iterative Solution Using Gauss-Seidel	26
4.4	Comparison to the Jacobi Method	28
4.5	Convergence	28
4.5.1	Rate of convergence	29
4.6	A More Accurate Update Using Non-Linear Gauss-Seidel	32
4.7	Adding Non-Holonomic Constraints	32
4.8	Regularization	34
5	Interpretation and Connections to Other Known Methods	35
5.1	Moore-Penrose Inverse and Least Squares Minimization	35
5.2	Sequential Impulses	36
5.3	Position Based Dynamics	36
5.4	Discrete Variational Mechanics	37
6	Techniques to Improve Convergence and Stability	40
6.1	Relaxation	40
6.2	Warm Starting	41
6.3	Sub-stepping	42
6.4	Baumgarte Stabilization	44
6.5	Split Impulse	45

7	Stiff Constraint Examples	47
7.1	Point to Point Constraint	47
7.1.1	Comparison to an Impulse Based Approach	49
7.2	Symmetries and Conservation of Momentum	49
7.3	Energy Conservation	52
7.4	Inequality Constraints	53
7.5	Contact Constraint	54
7.6	Contact Velocity Constraint	55
7.7	Contact Friction Constraint	56
7.8	Angular Axis Constraint	58
8	Compliant Constraints	60
8.1	Generalized Implicit Non-Linear Forces	60
8.2	Connection to Vertex Block Descent	61
8.3	Connection to Lagrange Multiplier Method and PBD	61
8.4	Compliant Constraint Forces and XPBD	62
8.4.1	XPBD as a Regularization on PBD	64
8.5	Compliant Constraint Examples	65
8.5.1	Spring Damper	65
8.5.2	Volumetric Constraint	66
9	Augmented Lagrangian Method	68
9.1	Augmented Vertex Block Descent	69
9.1.1	AVBD, PBD and Geometric Stiffness	69
9.2	Convergence Comparison vs. PBD For Large Mass Ratios	70
9.3	Point to Point Constraint	71
9.3.1	Exact Solution	74
9.3.2	Iterative Solution	74
10	Convergence of Constraint Chains	77
10.1	Setup	77
10.2	Effect of Iteration Count	78
10.2.1	Relaxation	79
10.2.2	Jacobi Iterations	80
10.2.3	AVBD Iterations	81
10.3	Effect of Mass Ratios	82
10.3.1	Relaxation	83
10.3.2	AVBD Iterations	84
10.4	Results	85
11	Summary	86

1 Introduction

1.1 Motivation

Position-Based Dynamics (PBD) and its extensions are widely used in real-time physics simulation for games and interactive applications due to their robustness and computational efficiency. However, the relationship between these methods and the classical formulations of rigid body dynamics is often not presented in a unified or principled way.

This document develops a derivation of position-based rigid body dynamics from first principles using constrained Lagrangian mechanics. Starting from the configuration space of rigid bodies on $SE(3)$, we derive the equations of motion and linearize the system using a first order approximation and solve via Gauss-Seidel. We show how common position based methods, such as PBD and XPBD are manifestations of this approach. The goal is to provide a clear mathematical foundation for techniques that are widely used in practical physics engines while clarifying their connection to classical rigid body dynamics and constrained optimization. With this motivation, these notes differ from the majority of the literature on this subject, where the goal is to provide clear practical implementation details, and to show the efficacy of these methods. For some excellent notes on the practical approach to position based dynamics methods, see, for example [2, 3, 25, 20, 19, 7, 10, 13, 23].

Recently, Macklin et al. [18] derive Projective Dynamics and XPBD from a common variational objective and name the primal (body-space) / dual (constraint-space) distinction. In some sense, these notes extend that programme to rigid bodies on $SO(3)$, add the augmented-Lagrangian (VBD/AVBD) branch, and replace their numerical conditioning study with closed-form Gauss-Seidel rates on chains.

These notes are an accumulation of notes that I have been making and refining over the years of implementing rigid body simulation in the games industry. The motivation was to provide a solid theoretical grounding for the techniques that are commonly used in industry. While the algorithms of PBD and its variants are well documented, their connection to classical mechanics is rarely presented in a unified or mathematically consistent way, particularly for the angular degrees of freedom, where the relationship between configuration variables, constraint Jacobians, and incremental updates is frequently left implicit. The goal here is to make that connection explicit and to illuminate the connections and shared foundations of position based methods.

1.2 Overview

The main substance of these notes is a derivation of position based dynamics for rigid bodies from first principles using constrained Lagrangian mechanics. In the first section, there is some background on rigid bodies and their configuration. In particular, I try to give a careful treatment of the representation of angular degrees of freedom.

The next section derives the equations of motion for a constrained set of rigid bodies. Here, we use some results from geometric mechanics that exploit the rotation group symmetry. Next is where I start to make the connection to real time simulation and I derive a first order approximation to the constrained equations of motion and show how this can be iteratively solved using Gauss-Seidel and discuss convergence properties of this algorithm.

From there I show how this approach connects to the algorithms of Position Based Dynamics. I cover PBD, XPBD, Vertex Block Descent (VBD) and Augmented Vertex Block Descent (AVBD), and also show the connection to a Sequential Impulse approach, to the Moore-Penrose pseudoinverse method, and to Tikhonov regularization.

In the final section, I analyze the convergence of a chain of connected rigid bodies with some of the different algorithms to show how they differ.

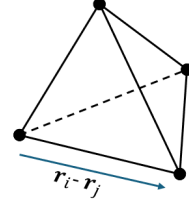
Notation

Symbol	Description
<i>Configuration and kinematics</i>	
$\mathbf{x} \in \mathbb{R}^3$	Position of the centre of mass
$\boldsymbol{\theta} \in \mathbb{R}^3$	Axis-angle rotation vector, $\boldsymbol{\theta} = \theta \hat{\boldsymbol{\theta}}$
$R(\boldsymbol{\theta}) \in SO(3)$	Rotation matrix, $R = e^{[\boldsymbol{\theta}]_{\times}}$
$\mathbf{X} = (x, \boldsymbol{\theta})$	Generalised configuration of a single body
$\mathcal{V} = (\dot{\mathbf{x}}, \boldsymbol{\omega})$	Generalised velocity of a single body
$\boldsymbol{\omega} \in \mathbb{R}^3$	World-space angular velocity
$\boldsymbol{\omega}_b \in \mathbb{R}^3$	Body-space angular velocity, $\boldsymbol{\omega}_b = R^T \boldsymbol{\omega}$
q	Unit quaternion representing orientation
$\boldsymbol{\Omega}$	Angular velocity bivector, $\boldsymbol{\Omega} = 2\dot{q}\bar{q} = -I\boldsymbol{\omega}$
$\mathbf{p}(\mathbf{X})$	World-space position of a point on the body, $\mathbf{p} = \mathbf{x} + R(\boldsymbol{\theta})\mathbf{r}_0$
$\mathbf{r}_0 \in \mathbb{R}^3$	Fixed body-space offset of a point from the centre of mass
$\mathbf{r}(\theta)$	World-space offset, $\mathbf{r} = R(\boldsymbol{\theta})\mathbf{r}_0$
<i>Lie group structure</i>	
$[\cdot]_{\times}$	Map from $\mathbb{R}^3$ to 3×3 skew-symmetric matrices (hat map)
$SO(3)$	Special orthogonal group of 3×3 rotation matrices
$J_L(\boldsymbol{\theta})$	Left Jacobian of $SO(3)$, maps $\dot{\boldsymbol{\theta}}$ to $\boldsymbol{\omega}$: $\boldsymbol{\omega} = J_L \dot{\boldsymbol{\theta}}$
<i>Mass and inertia</i>	
m	Scalar mass of a body
$I(\theta)$	World-space inertia tensor, $I = R(\boldsymbol{\theta})I_0R(\boldsymbol{\theta})^T$
$I_L(\boldsymbol{\theta})$	World space inertia rotated by the left Jacobian $J_L(\boldsymbol{\theta})^T I(\boldsymbol{\theta}) J_L(\boldsymbol{\theta})$
I_b	Body-space inertia tensor
$M(\mathbf{X})$	Generalised mass matrix for a system of N bodies (block diagonal)
$M_L(\mathbf{X})$	Generalized mass matrix with left Jacobian rotated inertias I_L
μ	Scalar effective mass for a single constraint
$\mathcal{M}$	Effective mass of a constraint
<i>Constraints and forces</i>	
$\mathbf{c}(\mathbf{X})$	Vector of holonomic constraint functions
$\boldsymbol{\lambda}$	Vector of Lagrange multipliers (constraint forces)
$\mathcal{D}\mathbf{c}$	Left-trivialized constraint Jacobian, satisfies $\dot{\mathbf{c}} = \mathcal{D}\mathbf{c}\mathcal{V}$
$\dot{\mathcal{D}}\mathbf{c}$	Velocity-space constraint Jacobian $\partial\mathbf{c}/\partial\mathcal{V}$ for non-holonomic constraints
$\mathcal{D}^2\mathbf{c}$	Constraint Hessian using the left-trivialized derivative

Symbol	Description
$\Delta \mathbf{X}$	Incremental displacement applied to resolve constraints
$\mathbf{F} = (\mathbf{f}, \boldsymbol{\tau})$	Generalised force vector (force and torque)
$\boldsymbol{\tau}_w$	World-space torque
<i>Iterative solver</i>	
k	Iteration index
Δt	Timestep
ξ	SOR relaxation parameter ($\xi < 1$ under-relaxation, $\xi > 1$ over-relaxation)
G	Gauss-Seidel iteration matrix, $G = -(D + L)^{-1}L^T$
G_J	Jacobi iteration matrix, $G_J = -D^{-1}(L + L^T)$
G_ξ	SOR iteration matrix
$\rho(\cdot)$	Spectral radius of a matrix
D, L, L^T	Diagonal, strictly lower triangular, and strictly upper triangular parts of A
<i>Compliant constraints and regularization</i>	
A	Diagonal matrix of stiffness parameters in compliant constraints
B	Diagonal matrix of damping parameters
$\tilde{\alpha}_i$	XPBD compliance parameter, $\tilde{\alpha}_i = 1/(A_{ii}\Delta t^2)$
$\tilde{\beta}_i$	XPBD damping parameter, $\tilde{\beta}_i = B_{ii}\Delta t^2$
ρ	Diagonal matrix of penalty coefficients in the augmented Lagrangian method
ε	Tikhonov regularization parameter
<i>Convergence analysis</i>	
μ_k	Eigenvalues of the Jacobi iteration matrix, $\mu_k = \cos(k\pi/N)$
r_i	Diagonal dominance ratio for row i , $r_i = \sum_{j \neq i} A_{ij} / A_{ii} $
γ	Mass ratio m/M in the chain convergence analysis
α	Fraction of error to be removed (convergence target)

2 Rigid Body Configuration

Rigid bodies are an idealised type of object that does not deform. I.e., for any two points $\mathbf{p}_i, \mathbf{p}_j$ in the rigid body the distance $|\mathbf{p}_i - \mathbf{p}_j|$ remains constant.



They have no internal degrees of freedom, and only have six total degrees of freedom:

- Three to define the position $\mathbf{x}$
- Three to define the orientation $\boldsymbol{\theta}$

A rigid body rotates around its centre of mass and moves through space on the trajectory of its centre of mass, so these six coordinates are the coordinates of the rigid body centre of mass $\mathbf{x}$. The location of any point on the rigid body p at any time t can be deduced from the position of the centre of mass $\mathbf{x}(t)$, the orientation of the body $R(t)$ and the (fixed) offset of the point on the rigid body $\mathbf{r}_0$:

$$\mathbf{p}(t) = \mathbf{x}(t) + R(t)\mathbf{r}_0. \quad (2.1)$$

The condition that distances between points on the rigid body remain fixed places conditions on the matrix R

$$\|\mathbf{p}_i - \mathbf{p}_j\|^2 = \|R(\mathbf{r}_i - \mathbf{r}_j)\|^2 = (R(\mathbf{r}_i - \mathbf{r}_j)) \cdot (R(\mathbf{r}_i - \mathbf{r}_j)) = (\mathbf{r}_i - \mathbf{r}_j)^T R^T R (\mathbf{r}_i - \mathbf{r}_j). \quad (2.2)$$

Therefore, we require $R^T R = \mathbb{1}$ so that distances remain the same, i.e. R is an orthogonal matrix, with $R^{-1} = R^T$.

Orthogonal matrices form a group: if two matrices R_i and R_j are orthogonal then their product will also be orthogonal

$$(R_i R_j)^T (R_i R_j) = R_j^T R_i^T R_i R_j = R_j^T \mathbb{1} R_j = \mathbb{1}. \quad (2.3)$$

This group is called the Orthogonal Group in three dimensions $O(3)$.

Since $R^T R = \mathbb{1}$, the determinant

$$\det(R^T R) = \det(R)^2 = 1, \quad (2.4)$$

so the determinant is either $+1$ or -1 . Matrices with $\det(R) = +1$ form a subgroup, since the product of two of matrices with determinant $+1$ is also a matrix with determinant $+1$. This is called the Special Orthogonal Group $SO(3)$ and represents rotations. Matrices with determinant -1 do not form a subgroup, as the product of two such matrices is a matrix with determinant $+1$. These represent reflections, and the product of two reflections is a rotation.

To justify that these two classes represent rotations and reflections respectively, we can look at two examples. First, a rotation about the z axis

$$R = \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad \det(R) = \cos^2 \theta + \sin^2 \theta = 1. \quad (2.5)$$

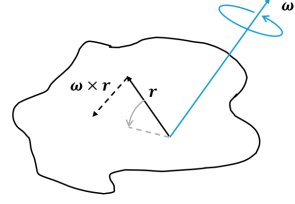
And a reflection in the xy plane

$$R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & -1 \end{pmatrix}, \quad \det(R) = -1. \quad (2.6)$$

2.1 Angular Velocity

The time derivative of the offset vector $\mathbf{r}(t) = R(t)\mathbf{r}_0$ is

$$\begin{aligned}\dot{\mathbf{r}}(t) &= \dot{R}(t)\mathbf{r}_0, \\ &= \dot{R}(t)R^T(t)\mathbf{r}.\end{aligned}\tag{2.7}$$



We know that the matrix $\dot{R}R^T$ must be anti-symmetric, since $RR^T = \mathbb{1}$ so

$$\frac{d}{dt}(RR^T) = \dot{R}R^T + R\dot{R}^T = \dot{R}R^T + (\dot{R}R^T)^T = 0.\tag{2.8}$$

Since $(\dot{R}R^T)$ is anti-symmetric, we can use it to define a vector $\boldsymbol{\omega}$ using the anti-symmetric matrix $[\boldsymbol{\omega}]_{\times}$

$$\dot{R}R^T = [\boldsymbol{\omega}]_{\times} = \begin{pmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{pmatrix}.\tag{2.9}$$

Comparing $\dot{\mathbf{r}} = \dot{R}(t)R(t)\mathbf{r}$ to the standard result from mechanics

$$\dot{\mathbf{r}} = \boldsymbol{\omega} \times \mathbf{r}.\tag{2.10}$$

We identify $\boldsymbol{\omega}$ as the world space angular velocity.

Body space angular velocity $\boldsymbol{\omega}_b$ can be derived from the world space angular velocity using

$$R\boldsymbol{\omega}_b = \boldsymbol{\omega},\tag{2.11}$$

so

$$\begin{aligned}[R\boldsymbol{\omega}_b]_{\times} &= [\boldsymbol{\omega}]_{\times}, \\ R[\boldsymbol{\omega}_b]_{\times}R^T &= \dot{R}R^T, \\ [\boldsymbol{\omega}_b]_{\times} &= R^T\dot{R},\end{aligned}\tag{2.12}$$

where we have used the fact that matrices A transform under a rotation R as $A' = RAR^T$.

2.1.1 The Path Ordered Exponential

Consider a body rotating with a constant angular velocity $\boldsymbol{\omega}$. For an infinitesimal time Δt a vector undergoes an infinitesimal rotation

$$\mathbf{r}' = \mathbf{r} + \boldsymbol{\omega} \times \mathbf{r}\Delta t = (\mathbb{1} + [\boldsymbol{\omega}]_{\times}\Delta t)\mathbf{r}.\tag{2.13}$$

We can build up a finite rotation as a product of infinitesimal rotations

$$\begin{aligned}R\mathbf{r} &= (\mathbb{1} + [\boldsymbol{\omega}\Delta t]_{\times})(\mathbb{1} + [\boldsymbol{\omega}\Delta t]_{\times})\cdots(\mathbb{1} + [\boldsymbol{\omega}\Delta t]_{\times})\mathbf{r}, \\ &= \lim_{n \rightarrow \infty} \left(\mathbb{1} + \frac{1}{n}[\boldsymbol{\omega}\Delta T]_{\times} \right)^n \mathbf{r},\end{aligned}\tag{2.14}$$

where $\Delta t = \Delta T/n$. This limit gives an exponential

$$R(\Delta T) = e^{[\boldsymbol{\omega}\Delta T]_{\times}} \quad (2.15)$$

This can also be seen in the case of constant angular velocity $\boldsymbol{\omega}(t) = \boldsymbol{\omega}_0$ by integrating the equation (2.9).

$$\ln(R) = \int \dot{R}R^T dt = \int [\boldsymbol{\omega}_0]_{\times} dt \quad (2.16)$$

It is tempting to want to generalize this for a general non-constant $\boldsymbol{\omega}(t)$ to

$$R(t) = e^{\int_0^t [\boldsymbol{\omega}(t')]_{\times} dt'} \quad (2.17)$$

but this is not exactly right, as different values of $[\boldsymbol{\omega}]_{\times}$ at different times do not commute with each other (since rotations do not commute). The answer is to use the *path ordered exponential*

$$\begin{aligned} R(t) &= P \exp \left(\int_0^t [\boldsymbol{\omega}(t')]_{\times} dt' \right), \\ &= 1 + \int_0^t [\boldsymbol{\omega}(t')]_{\times} dt' + \int_0^{t''} \int_{t'}^t [\boldsymbol{\omega}(t'')]_{\times} [\boldsymbol{\omega}(t')]_{\times} dt' dt'' + \dots \end{aligned} \quad (2.18)$$

for $0 < t' < t'' < t$. I.e. the omegas are ordered so that later times appear on the left.

This simplifies if we just want to take an incremental update going from some initial state $R(t)$ at time t to a new state at time $t + \Delta t$ for some small Δt . We can use

$$\begin{aligned} R(t + \Delta t) &= \exp \left(\int_t^{t+\Delta t} [\boldsymbol{\omega}(t')]_{\times} dt' \right) R(t), \\ &\approx \exp ([\boldsymbol{\omega}(t)]_{\times} \Delta t) R(t), \\ &= R(t) + [\boldsymbol{\omega}(t)\Delta t]_{\times} R(t) + \mathcal{O}(\Delta t^2), \end{aligned} \quad (2.19)$$

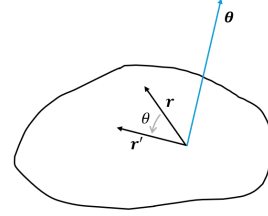
$$R(t + \Delta t) \approx (\mathbb{1} + [\boldsymbol{\omega}\Delta t]_{\times}) R(t) \quad (2.20)$$

where we assume that for a small enough time Δt the angular velocity can be considered to be constant. See David Tong's notes [33] for more detail on angular velocity, rotations and path ordered exponentials.

Note that, in practice, when updating a rotation matrix, it is important to retain its orthogonality, so in a rigid body simulation if we were updating rotation matrices in this way, we would have to orthonormalize the matrix after the update, which can be expensive, hence the general use of quaternions, which will be covered in section §2.2.

2.1.2 Axis/Angle Representation and the Exponential Map

The rotation matrix is a 3×3 matrix, so has 9 elements. But there are only three independent angular degrees of freedom. Later, when we come to look at rigid body constraints, we will need to be able to take the derivative of a function with respect to the angular degrees of freedom, so it is useful to find a representation that is a function of three independent variables. One such representation is the axis/angle representation.



Any orientation of a rigid body can be seen as a single rotation about a fixed axis $\hat{n}$ by an angle θ . This statement is known as *Euler's rotation theorem*. Therefore, we can define a vector $\boldsymbol{\theta} = \theta \hat{\boldsymbol{\theta}}$, where the magnitude θ is the angle of rotation, and the normalized direction $\hat{\boldsymbol{\theta}} = \hat{n}$ is the axis of rotation, to encode a rotation.

This means that, using (2.15), we can write any rotation matrix as

$$R(\boldsymbol{\theta}) = e^{[\boldsymbol{\theta}]_{\times}}. \quad (2.21)$$

Technically, this exponential map is a map between elements R of the Lie group $SO(3)$ consisting of 3×3 orthogonal matrices with unit determinant and its corresponding Lie algebra elements $[\boldsymbol{\theta}]_{\times}$ in $\mathfrak{so}(3)$ consisting of 3×3 skew-symmetric matrices. See, for example [38] for more information on the theory of Lie groups.

This exponential can be expanded to give the very useful *Rodrigues Rotation Formula*

$$\begin{aligned} R(\boldsymbol{\theta}) &= e^{[\boldsymbol{\theta}]_{\times}}, \\ &= \mathbb{1} + [\boldsymbol{\theta}]_{\times} + \frac{1}{2!}[\boldsymbol{\theta}]_{\times}^2 + \frac{1}{3!}[\boldsymbol{\theta}]_{\times}^3 + \dots, \\ &= \mathbb{1} + [\boldsymbol{\theta}]_{\times} \left(1 - \frac{1}{3!}|\boldsymbol{\theta}|^2 + \dots\right) + [\boldsymbol{\theta}]_{\times}^2 \left(\frac{1}{2!} - \frac{1}{4!}|\boldsymbol{\theta}|^2 + \dots\right), \\ &= \mathbb{1} + \frac{\sin \theta}{\theta}[\boldsymbol{\theta}]_{\times} + \frac{1 - \cos \theta}{\theta^2}[\boldsymbol{\theta}]_{\times}^2 \end{aligned} \quad (2.22)$$

where we have used $[\boldsymbol{\theta}]_{\times}^3 = -|\boldsymbol{\theta}|^2[\boldsymbol{\theta}]_{\times}$.

2.1.3 The Left Jacobian of $SO(3)$

It's important to remember that $R(\boldsymbol{\theta})$ represents a single rotation, and we can't compose rotations by adding axis/angles as

$$e^{[\boldsymbol{\theta}_1 + \boldsymbol{\theta}_2]_{\times}} \neq e^{[\boldsymbol{\theta}_1]_{\times}} e^{[\boldsymbol{\theta}_2]_{\times}} \quad (2.23)$$

since $[\boldsymbol{\theta}_1]_{\times}$ and $[\boldsymbol{\theta}_2]_{\times}$ do not necessarily commute. They only commute if the rotations are about the same axis. What we can do is to ask what is $R(\boldsymbol{\theta} + \Delta\boldsymbol{\theta})$. The *Baker-Campbell-Hausdorff Formulas* can be used to relate small perturbations in $[\boldsymbol{\theta}]_{\times}$ to small perturbations in $R(\boldsymbol{\theta})$.

The BCH formula finds Z for $e^Z = e^X e^Y$

$$Z = X + Y + \frac{1}{2}[X, Y] + \frac{1}{12}[X, [X, Y]] - \frac{1}{12}[Y, [X, Y]] + \dots \quad (2.24)$$

where $[A, B] = AB - BA$ is the commutator of A and B .

We want to use this to find a decomposition of the rotation for small angles, i.e. find a $J_L \Delta \theta$ such that $e^{[\theta + \Delta \theta]_\times} = e^{[J_L \Delta \theta]_\times} e^{[\theta]_\times}$, or $e^{[J_L \Delta \theta]_\times} = e^{[\theta + \Delta \theta]_\times} e^{-[\theta]_\times}$. So, letting $Z = [J_L \Delta \theta]_\times$, $X = [\theta + \Delta \theta]_\times$, and $Y = -[\theta]_\times$ we have

$$\begin{aligned}
[J_L \Delta \theta]_\times &= [\theta + \Delta \theta]_\times - [\theta]_\times - \frac{1}{2}[[\theta + \Delta \theta]_\times, [\theta]_\times] - \frac{1}{12}[[\theta + \Delta \theta]_\times, [[\theta + \Delta \theta]_\times, [\theta]_\times]] \\
&\quad - \frac{1}{12}[[\theta]_\times, [[\theta + \Delta \theta]_\times, [\theta]_\times]] + \dots, \\
&= [\Delta \theta]_\times + \frac{1}{2}[[\theta]_\times, [\Delta \theta]_\times] + \frac{1}{6}[[\theta]_\times, [[\theta]_\times, [\Delta \theta]_\times]] + \dots, \\
&= \left(\mathbb{1} + \frac{1}{2} \text{ad}_{[\theta]_\times} + \frac{1}{6} \text{ad}_{[\theta]_\times}^2 \right) [\Delta \theta]_\times + \dots, \\
&= \sum_{k=0}^{\infty} \frac{1}{(k+1)!} \text{ad}_{[\theta]_\times}^k [\Delta \theta]_\times.
\end{aligned} \tag{2.25}$$

where we have used the adjoint notation $\text{ad}_{[\theta]_\times} [\Delta \theta]_\times = [[\theta]_\times, [\Delta \theta]_\times]$. This obeys the identity

$$\begin{aligned}
\text{ad}_{[\theta]_\times} [\Delta \theta]_\times &= [[\theta]_\times, [\Delta \theta]_\times], \\
&= [\theta \times \Delta \theta]_\times, \\
&= [[\theta]_\times \Delta \theta]_\times.
\end{aligned} \tag{2.26}$$

Therefore, we have

$$\text{ad}_{[\theta]_\times}^k [\Delta \theta]_\times = [[\theta]_\times^k \Delta \theta]_\times. \tag{2.27}$$

Substituting this into the equation for $[J_L \Delta \theta]_\times$ we get

$$[J_L \Delta \theta]_\times = \sum_{k=0}^{\infty} \frac{1}{(k+1)!} [[\theta]_\times^k \Delta \theta]_\times, \tag{2.28}$$

so

$$J_L = \sum_{k=0}^{\infty} \frac{1}{(k+1)!} [\theta]_\times^k \tag{2.29}$$

Now, using the fact that $[\theta]_\times^3 = -\theta^2 [\theta]_\times$, where θ is the magnitude of θ , we see that odd powers are all functions of $[\theta]_\times$ and even powers a function of $[\theta]_\times^2$ and we get the closed form solution

$$J_L(\theta) = \mathbb{1} + \frac{1 - \cos \theta}{\theta^2} [\theta]_\times + \frac{\theta - \sin \theta}{\theta^3} [\theta]_\times^2. \tag{2.30}$$

$J_L(\theta)$ is known as the *Left Jacobian* of $SO(3)$ [5, 30] and it allows us to approximate a new rotation given a small change in the axis/angle $\Delta \theta$

$$\begin{aligned}
R(\theta + \Delta \theta) &= e^{[J_L(\theta) \Delta \theta]_\times} R(\theta), \\
&\approx (\mathbb{1} + [J_L(\theta) \Delta \theta]_\times) R(\theta).
\end{aligned} \tag{2.31}$$

Comparing to (2.20) we see that for small perturbations

$$\omega(t) \Delta t = \Delta \omega \approx J_L(\theta) \Delta \theta, \tag{2.32}$$

so J_L maps small changes in axis angle to small changes in angular velocity.

To see this more precisely, we can take another route. First, take the time derivative of the rotation matrix.

$$\begin{aligned}\dot{R}(t) &= \frac{d}{dt} e^{[\boldsymbol{\theta}(t)]_{\times}}, \\ &= \frac{d}{dt} \lim_{N \rightarrow \infty} \left(\mathbb{1} + \frac{1}{N} [\boldsymbol{\theta}(t)]_{\times} \right)^N, \\ &= \lim_{N \rightarrow \infty} \sum_{k=1}^N \left(\mathbb{1} + \frac{1}{N} [\boldsymbol{\theta}(t)]_{\times} \right)^{N-k} \frac{1}{N} [\dot{\boldsymbol{\theta}}(t)]_{\times} \left(\mathbb{1} + \frac{1}{N} [\boldsymbol{\theta}(t)]_{\times} \right)^{k-1}.\end{aligned}\quad (2.33)$$

Note that $[\boldsymbol{\theta}(t)]_{\times}$ and $[\dot{\boldsymbol{\theta}}(t)]_{\times}$ do not necessarily commute. Now re-scale the interval into $[0, \dots, 1]$ using $\Delta s = \Delta k/N = 1/N$, since k are integers so Δk is 1. Taking the limit as $N \rightarrow \infty$ gives

$$\dot{R}(t) = \int_0^1 e^{(1-s)[\boldsymbol{\theta}(t)]_{\times}} [\dot{\boldsymbol{\theta}}(t)]_{\times} e^{s[\boldsymbol{\theta}(t)]_{\times}} ds. \quad (2.34)$$

Now let $s' = 1 - s$

$$\begin{aligned}\dot{R}(t) &= - \int_1^0 e^{s'[\boldsymbol{\theta}(t)]_{\times}} [\dot{\boldsymbol{\theta}}(t)]_{\times} e^{(1-s')[\boldsymbol{\theta}(t)]_{\times}} ds', \\ &= \int_0^1 e^{s[\boldsymbol{\theta}(t)]_{\times}} [\dot{\boldsymbol{\theta}}(t)]_{\times} e^{(1-s)[\boldsymbol{\theta}(t)]_{\times}} ds, \\ &= \left(\int_0^1 e^{s[\boldsymbol{\theta}(t)]_{\times}} [\dot{\boldsymbol{\theta}}(t)]_{\times} e^{-s[\boldsymbol{\theta}(t)]_{\times}} ds \right) R(t).\end{aligned}\quad (2.35)$$

Looking at the expression in the integral this is $R(s\boldsymbol{\theta})[\dot{\boldsymbol{\theta}}]_{\times} R(s\boldsymbol{\theta})^T$. We know that this is how a matrix R transforms under a rotation. A vector would transform as $R(s\boldsymbol{\theta})\dot{\boldsymbol{\theta}}$, so

$$\dot{R}(t)R^T(t) = [\omega(t)]_{\times} = \left[\left(\int_0^1 e^{s[\boldsymbol{\theta}(t)]_{\times}} ds \right) \dot{\boldsymbol{\theta}}(t) \right]_{\times}. \quad (2.36)$$

(Technically, the adjoint action of the group $SO(3)$ acting on its algebra $\mathfrak{so}(3)$ is $Ad_R[X]_{\times} = R[X]_{\times} R^T$. Elements of the Lie algebra can be mapped to vectors $\boldsymbol{x} \in \mathbb{R}^3$ with $Ad_R \boldsymbol{x} = R\boldsymbol{x}$).

If we define

$$J_L(\boldsymbol{\theta}) = \int_0^1 e^{s[\boldsymbol{\theta}]_{\times}} ds \quad (2.37)$$

then

$$\boldsymbol{\omega} = J_L(\boldsymbol{\theta})\dot{\boldsymbol{\theta}}. \quad (2.38)$$

It can be verified that this is the same J_L as we got from the BCH expansion by using the Rodrigues rotation formula and performing the integral

$$\begin{aligned}J_L(\boldsymbol{\theta}) &= \int_0^1 e^{s[\boldsymbol{\theta}]_{\times}} ds, \\ &= \int_0^1 \left(\mathbb{1} + \frac{\sin s\theta}{\theta} [\boldsymbol{\theta}]_{\times} + \frac{1 - \cos s\theta}{\theta^2} [\boldsymbol{\theta}]_{\times}^2 \right) ds, \\ &= \mathbb{1} + \frac{1 - \cos \theta}{\theta^2} [\boldsymbol{\theta}]_{\times} + \frac{\theta - \sin \theta}{\theta^3} [\boldsymbol{\theta}]_{\times}^2.\end{aligned}\quad (2.39)$$

In summary:

J_L is known as the left Jacobian of $SO(3)$ and maps the derivative of the axis angle vector to the world space angular velocity.

$$\boldsymbol{\omega} = J_L(\boldsymbol{\theta})\dot{\boldsymbol{\theta}}$$

It can be used to find how infinitesimal changes in the axis/angle $\boldsymbol{\theta}$ update the rotation $R = e^{[\boldsymbol{\theta}]^\times}$

$$e^{[\boldsymbol{\theta}+\Delta\boldsymbol{\theta}]^\times} \approx e^{[J_L(\boldsymbol{\theta})\Delta\boldsymbol{\theta}]^\times} e^{[\boldsymbol{\theta}]^\times} = e^{[\Delta\boldsymbol{\omega}]^\times} R(\boldsymbol{\theta}). \quad (2.40)$$

This is very useful for rigid body simulations where we are taking small perturbations in the configuration space, and taking derivatives of constraints with respect to the constraint rotation.

Note that the left Jacobian here is what Macklin et al [18] describe as the *kinematic map* that maps spatial velocities to system coordinate time derivatives $\mathbf{G}$. J_L is $\mathbf{G}$ for $SO(3)$.

Derivatives

It will be important to be able to take derivatives with respect to $\boldsymbol{\theta}$, e.g. in 3.2 where we want to be able to take derivatives of a constraint that is a function of $\boldsymbol{\theta}$. We can see how to do this using the left Jacobian:

Define the *left-trivialized derivative* of a function f as

$$\mathcal{D}f = \frac{\partial f}{\partial \boldsymbol{\theta}} J_L(\boldsymbol{\theta})^{-1}. \quad (2.41)$$

in this way

$$\dot{f} = \frac{\partial f}{\partial \boldsymbol{\theta}} \dot{\boldsymbol{\theta}} = \frac{\partial f}{\partial \boldsymbol{\theta}} J_L(\boldsymbol{\theta})^{-1} \boldsymbol{\omega} = (\mathcal{D}f) \boldsymbol{\omega}. \quad (2.42)$$

The left Jacobian inverse is used to convert from a rate of change with respect to the axis/angle coordinates $\boldsymbol{\theta}$ to a rate of change with respect to the angular velocity $\boldsymbol{\omega}$.

2.2 Quaternion Representation*

This section is mostly tangential to the rest of the notes which, on the whole, use the axis/angle and exponential map representation of rotations. The exception is in integrating rigid body rotations, where maintaining orthogonality is important, so we use the quaternion representation as it is more efficient to normalize.

In this section, I will be using and assuming some familiarity with geometric algebra, the natural and most intuitive setting for quaternions. Good references for this are [16, 12]. For readers familiar with Hamilton's formulation of quaternions, the end result will be familiar and you can safely replace I with the unit complex number i .

When using quaternions, we define a point on a rigid body by its centre of mass $\mathbf{x}(t)$ and a quaternion $q(t)$ representing the orientation, so that a point on the body is

$$\mathbf{p}(t) = \mathbf{x}(t) + q(t)\mathbf{r}_0\bar{q}(t), \quad (2.43)$$

where $\bar{q}$ is the quaternion conjugate. The condition that the distance between two points on the body remains fixed means that, using the geometric product

$$\|\mathbf{p}_i - \mathbf{p}_j\|^2 = q(\mathbf{r}_i - \mathbf{r}_j)\bar{q}q(\mathbf{r}_i - \mathbf{r}_j)\bar{q} \quad (2.44)$$

So we require that $q\bar{q} = \bar{q}q = 1$ for this to be invariant.

The time derivative of a vector $\mathbf{r}(t) = q(t)\mathbf{r}_0\bar{q}(t)$ is

$$\begin{aligned} \dot{\mathbf{r}}(t) &= \dot{q}(t)\mathbf{r}_0\bar{q}(t) + q(t)\mathbf{r}_0\dot{\bar{q}}(t), \\ &= \dot{q}(t)\bar{q}(t)\mathbf{r}(t) + \mathbf{r}(t)q(t)\dot{\bar{q}}(t), \\ &= (\dot{q}(t)\bar{q}(t))\mathbf{r}(t) - \mathbf{r}(t)(\dot{q}(t)\bar{q}(t)). \end{aligned} \quad (2.45)$$

Now define the bivector $\mathbf{\Omega} = 2\dot{q}(t)\bar{q}(t)$. This is then

$$\begin{aligned} \dot{\mathbf{r}}(t) &= \frac{1}{2}(\mathbf{\Omega}\mathbf{r} - \mathbf{r}\mathbf{\Omega}), \\ &= \mathbf{\Omega} \cdot \mathbf{r}, \end{aligned} \quad (2.46)$$

where $\mathbf{\Omega} \cdot \mathbf{r} = \frac{1}{2}(\mathbf{\Omega}\mathbf{r} - \mathbf{r}\mathbf{\Omega})$ is the inner product of a bivector with a vector. $\mathbf{\Omega}$ is (minus) the dual of the angular velocity vector $\boldsymbol{\omega}$

$$\boldsymbol{\omega} = I\mathbf{\Omega}, \quad \mathbf{\Omega} = -I\boldsymbol{\omega}, \quad (2.47)$$

where $I = \mathbf{e}_1\mathbf{e}_2\mathbf{e}_3$ is the unit pseudoscalar, with the property $I^2 = -1$. The sign is fixed by requiring $\dot{\mathbf{r}} = \boldsymbol{\omega} \times \mathbf{r}$: with the standard duality $\mathbf{a} \times \mathbf{b} = -I(\mathbf{a} \wedge \mathbf{b})$ one finds $(I\boldsymbol{\omega}) \cdot \mathbf{r} = -\boldsymbol{\omega} \times \mathbf{r}$, so $\mathbf{\Omega} \cdot \mathbf{r} = -(I\boldsymbol{\omega}) \cdot \mathbf{r} = \boldsymbol{\omega} \times \mathbf{r}$ as required. (A quick check: $I\mathbf{e}_3 = \mathbf{e}_1\mathbf{e}_2$ and $(\mathbf{e}_1\mathbf{e}_2) \cdot \mathbf{e}_1 = -\mathbf{e}_2$, whereas $\mathbf{e}_3 \times \mathbf{e}_1 = +\mathbf{e}_2$.)

2.2.1 Quaternions from the Path Ordered Exponential

Just as we found a differential equation for the rotation matrix $\dot{R} = [\boldsymbol{\omega}]_{\times}R$, we can find the equivalent differential equation for the quaternion q . Starting from our definition of the angular velocity bivector $\mathbf{\Omega} = 2\dot{q}\bar{q}$ we have

$$\dot{q}(t) = \frac{1}{2}\mathbf{\Omega}(t)q(t). \quad (2.48)$$

This equation is identical in form to the one we solved for rotation matrices. If the body is rotating with a constant angular velocity bivector $\mathbf{\Omega}$, we can integrate this directly to find the quaternion at time t :

$$q(t) = \exp\left(\frac{1}{2}\mathbf{\Omega}t\right). \quad (2.49)$$

However, for a general non-constant angular velocity $\mathbf{\Omega}(t)$, the bivectors at different times do not commute. Just as in the rotation matrix case, we must use the *path ordered exponential* to build up the finite rotation from infinitesimal steps:

$$q(t) = P \exp\left(\frac{1}{2} \int_0^t \mathbf{\Omega}(t')dt'\right) \quad (2.50)$$

In practice, for physics simulations and games, we usually want to perform an incremental update over a small timestep Δt . We can assume the angular velocity is roughly constant over this small

window, giving the discrete update step:

$$\begin{aligned}
q(t + \Delta t) &= \exp \left(\frac{1}{2} \int_t^{t+\Delta t} \boldsymbol{\Omega}(t') dt' \right) q(t), \\
&\approx \exp \left(\frac{1}{2} \boldsymbol{\Omega}(t) \Delta t \right) q(t), \\
&\approx \exp \left(-\frac{1}{2} I \boldsymbol{\omega}(t) \Delta t \right) q(t).
\end{aligned} \tag{2.51}$$

This quaternion update is generally favoured in numerical integration as normalizing the quaternion to avoid numerical drift is computationally easier than orthonormalizing a 3×3 matrix.

2.2.2 Relation to Axis/Angle Representation

To connect this back to our axis/angle representation $\boldsymbol{\theta} = \theta \hat{\mathbf{n}}$, we need to evaluate the exponential map of the bivector. Let $\hat{B}$ be the unit bivector representing the plane of rotation, which is (minus) the dual of the rotation axis $\hat{\mathbf{n}}$, $\hat{B} = -I\hat{\mathbf{n}}$, consistent with $\boldsymbol{\Omega} = -I\boldsymbol{\omega}$ above. If a body rotates by a total angle θ in this plane, the integral of the angular velocity over that time is $\int \boldsymbol{\Omega} dt = \theta \hat{B}$. Substituting this into our exponential solution, the quaternion representing a rotation by $\boldsymbol{\theta}$ is:

$$q(\boldsymbol{\theta}) = \exp \left(\frac{1}{2} \theta \hat{B} \right). \tag{2.52}$$

Because the square of a unit bivector in 3D Euclidean space is negative ($\hat{B}^2 = -1$), the Taylor series expansion of this exponential gives Euler's formula for complex numbers:

$$\begin{aligned}
\exp \left(\frac{1}{2} \theta \hat{B} \right) &= 1 + \frac{1}{2} \theta \hat{B} + \frac{1}{2!} \left(\frac{1}{2} \theta \hat{B} \right)^2 + \frac{1}{3!} \left(\frac{1}{2} \theta \hat{B} \right)^3 + \dots \\
&= \left(1 - \frac{1}{2!} \left(\frac{\theta}{2} \right)^2 + \dots \right) + \hat{B} \left(\frac{\theta}{2} - \frac{1}{3!} \left(\frac{\theta}{2} \right)^3 + \dots \right) \\
&= \cos \left(\frac{\theta}{2} \right) + \hat{B} \sin \left(\frac{\theta}{2} \right).
\end{aligned} \tag{2.53}$$

This is the link between the axis/angle representation and the quaternion. A unit quaternion q is an even-grade multivector composed of a scalar part and a bivector part:

$$q = \cos \left(\frac{\theta}{2} \right) - I \hat{\mathbf{n}} \sin \left(\frac{\theta}{2} \right). \tag{2.54}$$

In standard (Hamilton) quaternion notation the unit bivectors $-I\mathbf{e}_1$, $-I\mathbf{e}_2$, $-I\mathbf{e}_3$ are the imaginary units i , j , k (this is the identification for which $ij = k$), so the pure quaternion $(0, \boldsymbol{\omega})$ is exactly $\boldsymbol{\Omega}$ and $\dot{q} = \frac{1}{2} \boldsymbol{\Omega} q$ is the familiar quaternion kinematic equation $\dot{q} = \frac{1}{2} (0, \boldsymbol{\omega}) q$. Notice the appearance of the half-angle $\theta/2$. This occurs because the vector transformation law $\mathbf{r} = q \mathbf{r}_0 \bar{q}$ applies the quaternion twice (once on the left, once on the right). To rotate a vector by a full angle θ , each side of the sandwich product must contribute half of the rotation.

3 Equations of Motion

The goal is to find the equations of motion for a rigid body. To do this we follow a Lagrangian derivation. The Lagrangian for a system is the difference between the kinetic and potential energy. For a single rigid body this is

$$L = \frac{1}{2}m|\dot{\mathbf{x}}|^2 + \frac{1}{2}\boldsymbol{\omega} \cdot I(\boldsymbol{\theta})\boldsymbol{\omega} - V(\mathbf{x}, \boldsymbol{\theta}), \quad (3.1)$$

for mass m , inertia $I(\boldsymbol{\theta})$ and potential energy $V(\mathbf{x}, \boldsymbol{\theta})$. For standard variational mechanics, we want to make this a function of $\mathbf{X}$ and its time derivative $\dot{\mathbf{X}}$. In this notation

The Lagrangian for a single rigid body is

$$L(\mathbf{X}, \dot{\mathbf{X}}) = \frac{1}{2}\dot{\mathbf{X}} \cdot M_L(\mathbf{X})\dot{\mathbf{X}} - V(\mathbf{X}) \quad (3.2)$$

where

$$M_L(\mathbf{X}) = \begin{pmatrix} m\mathbb{1} & 0 \\ 0 & J_L(\boldsymbol{\theta})^T I(\boldsymbol{\theta}) J_L(\boldsymbol{\theta}) \end{pmatrix}, \quad \mathbf{X} = \begin{pmatrix} \mathbf{x} \\ \boldsymbol{\theta} \end{pmatrix}, \quad \dot{\mathbf{X}} = \begin{pmatrix} \mathbf{v} \\ J_L(\boldsymbol{\theta})^{-1}\boldsymbol{\omega} \end{pmatrix}. \quad (3.3)$$

Notice that we have factored in the left Jacobian $J_L(\boldsymbol{\theta})$ into the mass matrix $M_L(\mathbf{X})$, so that we get an equation in terms of $\dot{\mathbf{X}}$, and we have used it to map $\dot{\boldsymbol{\theta}}$ to the angular velocity $\boldsymbol{\omega}$.

In standard Lagrangian mechanics we vary the action according to

$$\begin{aligned} \delta S &= \int dt \delta L(\dot{\mathbf{X}}, \mathbf{X}), \\ &= \int dt \left(\frac{\partial L}{\partial \dot{\mathbf{X}}} \delta \dot{\mathbf{X}} + \frac{\partial L}{\partial \mathbf{X}} \delta \mathbf{X} \right), \\ &= \int dt \left(-\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{\mathbf{X}}} \right) + \frac{\partial L}{\partial \mathbf{X}} \right) \delta \mathbf{X} + \left[\frac{\partial L}{\partial \dot{\mathbf{X}}} \delta \mathbf{X} \right]_{\text{boundary}}. \end{aligned} \quad (3.4)$$

Where the last step uses integration by parts. The term $\delta \mathbf{X}$ is taken to be zero on the boundary, so we are left with the Euler-Lagrange equations of motion

$$-\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{\mathbf{X}}} \right) + \frac{\partial L}{\partial \mathbf{X}} = 0, \quad (3.5)$$

which, for our rigid body Lagrangian gives

$$\frac{d}{dt} \left(M_L(\mathbf{X}) \dot{\mathbf{X}} \right) = -\frac{\partial V}{\partial \mathbf{X}} + \frac{1}{2} \dot{\mathbf{X}} \cdot \frac{\partial M_L}{\partial \mathbf{X}} \dot{\mathbf{X}}. \quad (3.6)$$

For the linear term this gives Newton's second law for the linear motion of the centre of mass

$$m\ddot{\mathbf{x}} = -\frac{\partial V}{\partial \mathbf{x}} = \mathbf{f}. \quad (3.7)$$

for force $\mathbf{f}$.

The angular degrees of freedom give

$$\frac{d}{dt} \left(I_L(\boldsymbol{\theta}) \dot{\boldsymbol{\theta}} \right) = -\frac{\partial V}{\partial \boldsymbol{\theta}} + \frac{1}{2} \dot{\boldsymbol{\theta}} \cdot \frac{\partial I_L}{\partial \boldsymbol{\theta}} \dot{\boldsymbol{\theta}} \quad (3.8)$$

with $I_L(\boldsymbol{\theta}) = J_L(\boldsymbol{\theta})^T I(\boldsymbol{\theta}) J_L(\boldsymbol{\theta})$. This approach can be used (see, for example [8]) to recover Euler's equations

$$I(\boldsymbol{\theta})\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (I(\boldsymbol{\theta})\boldsymbol{\omega}) = \boldsymbol{\tau} \quad (3.9)$$

for a torque $\boldsymbol{\tau}$, but the algebra is complicated due to the third order tensor term $\frac{\partial I}{\partial \boldsymbol{\theta}}$.

3.1 The Euler-Poincaré Method

Instead, we can improve on the standard Lagrangian approach when dealing with actions that are symmetric under some group action. The angular configuration of a rigid body lies in $SO(3)$, rather than a linear vector space, so it makes sense when computing variations to look for ones that preserve that group symmetry. In general, this approach leads to the *Euler-Poincaré equations of motion*.

Start by considering the variation of a rotation matrix R . Since $RR^T = 1$ we have

$$\delta RR^T + R\delta R^T = 0, \quad (3.10)$$

so

$$(\delta RR^T) + (\delta RR^T)^T = 0 \quad (3.11)$$

meaning that δRR^T must be antisymmetric. Say

$$\delta RR^T = [\eta]_{\times}. \quad (3.12)$$

Now, from (2.9) we have $[\boldsymbol{\omega}]_{\times} = \dot{R}R^T$. From this we can compute the variation of $\boldsymbol{\omega}$.

$$\begin{aligned} [\delta\boldsymbol{\omega}]_{\times} &= \delta\dot{R}R^T + \dot{R}\delta R^T, \\ &= [\dot{\eta}]_{\times} + [\eta]_{\times}(\dot{R}R^T) - (\dot{R}R^T)[\eta]_{\times}, \\ &= [\dot{\eta}]_{\times} + [\eta]_{\times}[\boldsymbol{\omega}]_{\times} - [\boldsymbol{\omega}]_{\times}[\eta]_{\times}, \\ &= [\dot{\eta}]_{\times} - [\boldsymbol{\omega} \times \eta]_{\times}. \end{aligned} \quad (3.13)$$

Therefore,

$$\delta\boldsymbol{\omega} = \dot{\eta} - \boldsymbol{\omega} \times \eta. \quad (3.14)$$

The kinetic energy is $T = \frac{1}{2}\boldsymbol{\omega} \cdot I\boldsymbol{\omega}$, so taking the variation gives

$$\delta T = (I\boldsymbol{\omega}) \cdot \delta\boldsymbol{\omega} + \frac{1}{2}\boldsymbol{\omega} \cdot (\delta I\boldsymbol{\omega}). \quad (3.15)$$

Using $I = RI_bR^T$ we have

$$\begin{aligned} \delta I &= \delta RR^T I + IR\delta R^T, \\ &= [\eta]_{\times}I - I[\eta]_{\times}. \end{aligned} \quad (3.16)$$

So

$$\begin{aligned} \delta T &= (I\boldsymbol{\omega}) \cdot (\dot{\eta} - \boldsymbol{\omega} \times \eta) + \frac{1}{2}\boldsymbol{\omega} \cdot ([\eta]_{\times}I - I[\eta]_{\times})\boldsymbol{\omega}, \\ &= (I\boldsymbol{\omega}) \cdot (\dot{\eta} - \boldsymbol{\omega} \times \eta) + \boldsymbol{\omega} \cdot (\eta \times (I\boldsymbol{\omega})), \\ &= (I\boldsymbol{\omega}) \cdot \dot{\eta}. \end{aligned} \quad (3.17)$$

The terms constant in η cancel, as they must. If η is constant then the variation $\delta R = [\eta]_{\times} R$ is a rigid rotation of the whole configuration $R \rightarrow e^{[\eta]_{\times}} R$. The kinetic energy is unchanged under a fixed rotation of the system, so terms with constant η must cancel.

We can now compute the variation of the action

$$\begin{aligned}\delta S &= \delta \int dt \left(\frac{1}{2} \boldsymbol{\omega} \cdot I \boldsymbol{\omega} - V(\boldsymbol{\theta}) \right), \\ &= \int dt \left((I \boldsymbol{\omega}) \cdot \dot{\eta} - \frac{\partial V}{\partial \boldsymbol{\theta}} \cdot \delta \boldsymbol{\theta} \right), \\ &= [(I \boldsymbol{\omega}) \cdot \eta]_{\text{boundary}} + \int dt \left(-\frac{d}{dt} (I \boldsymbol{\omega}) \cdot \eta - \frac{\partial V}{\partial \boldsymbol{\theta}} \cdot \delta \boldsymbol{\theta} \right),\end{aligned}\tag{3.18}$$

where in the third step we used integration by parts and by construction variation η is zero on the boundary.

For the variation $\delta \boldsymbol{\theta}$ consider the variation δR this time using (2.40).

$$\begin{aligned}\delta R &= \frac{d}{d\epsilon} e^{[\boldsymbol{\theta} + \epsilon \delta \boldsymbol{\theta}]_{\times}} \Big|_{\epsilon=0}, \\ &= \frac{d}{d\epsilon} e^{[\epsilon J_L \delta \boldsymbol{\theta}]_{\times}} R \Big|_{\epsilon=0}, \\ &= \frac{d}{d\epsilon} (\mathbb{1} + \epsilon [J_L \delta \boldsymbol{\theta}]_{\times} + \dots) R \Big|_{\epsilon=0}, \\ &= [J_L \delta \boldsymbol{\theta}]_{\times} R.\end{aligned}\tag{3.19}$$

So

$$\delta R R^T = [J_L \delta \boldsymbol{\theta}]_{\times} = [\eta]_{\times}\tag{3.20}$$

and

$$\delta \boldsymbol{\theta} = J_L^{-1} \eta.\tag{3.21}$$

Therefore, the variation in the action is

$$\delta S = \int dt \left(-\frac{d}{dt} (I \boldsymbol{\omega}) - \frac{\partial V}{\partial \boldsymbol{\theta}} J_L^{-1} \right) \cdot \eta\tag{3.22}$$

Giving the equations of motion

$$\frac{d}{dt} (I \boldsymbol{\omega}) = -(\mathcal{D}V)^T = \boldsymbol{\tau}_w,\tag{3.23}$$

where we have used the left Jacobian derivative $\mathcal{D}$ notation from (2.41), and identified $-(\mathcal{D}V)^T$ as the world space torque $\boldsymbol{\tau}_w$.

Note that, using the time derivative of the inertia

$$\begin{aligned}\dot{I}(\boldsymbol{\theta}) &= \frac{d}{dt} (R(\boldsymbol{\theta}) I_b R(\boldsymbol{\theta})^T), \\ &= \dot{R}(\boldsymbol{\theta}) I_b R(\boldsymbol{\theta})^T + R(\boldsymbol{\theta}) I_b \dot{R}(\boldsymbol{\theta})^T, \\ &= \dot{R}(\boldsymbol{\theta}) R(\boldsymbol{\theta})^T I(\boldsymbol{\theta}) + I(\boldsymbol{\theta}) R(\boldsymbol{\theta}) \dot{R}(\boldsymbol{\theta})^T, \\ &= \dot{R}(\boldsymbol{\theta}) R(\boldsymbol{\theta})^T I(\boldsymbol{\theta}) - I(\boldsymbol{\theta}) \left(\dot{R}(\boldsymbol{\theta}) R(\boldsymbol{\theta})^T \right), \\ &= [\boldsymbol{\omega}]_{\times} I(\boldsymbol{\theta}) - I(\boldsymbol{\theta}) [\boldsymbol{\omega}]_{\times},\end{aligned}\tag{3.24}$$

we recover Euler's equations in world space

$$I(\boldsymbol{\theta})\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (I(\boldsymbol{\theta})\boldsymbol{\omega}) = \boldsymbol{\tau}_w. \quad (3.25)$$

Using this result combined with the linear equations of motion, we can write down a compact form for the equations of motion of N rigid bodies:

The equations of motion for a system of N rigid bodies can be written as

$$\frac{d}{dt} (M(\mathbf{X})\mathcal{V}) = \mathbf{F} \quad (3.26)$$

where

$$M(\mathbf{X}) = \begin{pmatrix} m_1 \mathbb{1} & & & \\ & I_1(\boldsymbol{\theta}_1) & & \\ & & \ddots & \\ & & & m_N \mathbb{1} \\ & & & & I_N(\boldsymbol{\theta}_N) \end{pmatrix} \quad (3.27)$$

and

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1 \\ \boldsymbol{\theta}_1 \\ \vdots \\ \mathbf{x}_N \\ \boldsymbol{\theta}_N \end{pmatrix}, \quad \mathcal{V} = \begin{pmatrix} \dot{\mathbf{x}}_1 \\ \boldsymbol{\omega}_1 \\ \vdots \\ \dot{\mathbf{x}}_N \\ \boldsymbol{\omega}_N \end{pmatrix}, \quad \mathbf{F} = \begin{pmatrix} \mathbf{f}_1 \\ \boldsymbol{\tau}_1 \\ \vdots \\ \mathbf{f}_N \\ \boldsymbol{\tau}_N \end{pmatrix} \quad (3.28)$$

and the world space force $\mathbf{f}_i$ and torque $\boldsymbol{\tau}_i$ body i can be derived from a potential V

$$\mathbf{f}_i = -\frac{\partial V(\mathbf{X})}{\partial \mathbf{x}_i}, \quad \boldsymbol{\tau}_i = -\frac{\partial V(\mathbf{X})}{\partial \boldsymbol{\theta}_i} J_L^{-1}(\boldsymbol{\theta}_i) = -(\mathcal{D}_i V)^T. \quad (3.29)$$

3.2 Holonomic Constraints

In rigid body simulation, we want to add constraints to the system. These constraints can, for instance, model joints or contacts between rigid bodies. We first consider holonomic constraints. These are constraints that are a function of the configuration $\mathbf{X}$ and not the velocity $\dot{\mathbf{X}}$. These constraints can be added to the system using the method of Lagrange multipliers. For each (scalar) constraint function $c_i(\mathbf{X})$ we add a term $\lambda_i c_i(\mathbf{X})$, where λ_i is a Lagrange multiplier. The Lagrange multiplier is an additional free parameter of the system to be solved, i.e., $L(\mathbf{X}, \dot{\mathbf{X}}) \rightarrow L(\mathbf{X}, \dot{\mathbf{X}}, \lambda)$ and we also vary the action with respect to λ . For a vector of constraint functions $\mathbf{c}$ and Lagrange multipliers $\boldsymbol{\lambda}$ we have

$$L(\mathbf{X}, \dot{\mathbf{X}}, \boldsymbol{\lambda}) = \frac{1}{2} \dot{\mathbf{X}} \cdot M_L(\mathbf{X}) \dot{\mathbf{X}} - V(\mathbf{X}) + \boldsymbol{\lambda} \cdot \mathbf{c}(\mathbf{X}). \quad (3.30)$$

Varying with respect to the Lagrange multipliers will just give back the constraint equations

$$\mathbf{c}(\mathbf{X}) = 0, \quad (3.31)$$

and the Euler-Lagrange equations pick up an additional term $\left(\frac{\partial \mathbf{c}}{\partial \mathbf{X}}\right)^T \boldsymbol{\lambda}$

$$\frac{d}{dt} \left(M_L(\mathbf{X}) \dot{\mathbf{X}} \right) = -\frac{\partial V}{\partial \mathbf{X}} + \frac{1}{2} \dot{\mathbf{X}} \cdot \frac{\partial M_L}{\partial \mathbf{X}} \dot{\mathbf{X}} + \left(\frac{\partial \mathbf{c}}{\partial \mathbf{X}} \right)^T \boldsymbol{\lambda}. \quad (3.32)$$

This additional term is identified as the generalized constraint force

$$\mathbf{F}_c = \left(\frac{\partial \mathbf{c}}{\partial \mathbf{X}} \right)^T \boldsymbol{\lambda}. \quad (3.33)$$

For linear degrees of freedom we have for each constraint i a set of forces per body j of

$$\mathbf{f}_{ij} = \left(\frac{\partial c_i}{\partial \mathbf{x}_j} \right)^T \lambda_i. \quad (3.34)$$

For angular degrees of freedom we go back into velocity space (rather than $\dot{\mathbf{X}}$ space) using the Euler-Poincaré formulation.

Now, with the addition of $\boldsymbol{\lambda} \cdot \mathbf{c}(\mathbf{X})$ to the Lagrangian, the variation of the action (3.22) becomes

$$\delta S = \int dt \left(-\frac{d}{dt} (I\boldsymbol{\omega}) - \frac{\partial V}{\partial \boldsymbol{\theta}} J_L^{-1} + \left(\frac{\partial \mathbf{c}}{\partial \boldsymbol{\theta}} J_L^{-1} \right)^T \boldsymbol{\lambda} \right) \cdot \boldsymbol{\eta}, \quad (3.35)$$

and the equation of motion becomes

$$\frac{d}{dt} (I\boldsymbol{\omega}) = \boldsymbol{\tau}_w + (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda}. \quad (3.36)$$

The derivative $\mathcal{D}\mathbf{c}$ of a vector of constraint functions is

$$\mathcal{D}\mathbf{c} = \begin{pmatrix} \frac{\partial c_1}{\partial \mathbf{x}_1} & \frac{\partial c_1}{\partial \boldsymbol{\theta}_1} J_L(\boldsymbol{\theta}_1)^{-1} & \cdots & \frac{\partial c_1}{\partial \mathbf{x}_N} & \frac{\partial c_1}{\partial \boldsymbol{\theta}_N} J_L(\boldsymbol{\theta}_N)^{-1} \\ \vdots & & \ddots & & \\ \frac{\partial c_M}{\partial \mathbf{x}_1} & \frac{\partial c_M}{\partial \boldsymbol{\theta}_1} J_L(\boldsymbol{\theta}_1)^{-1} & \cdots & \frac{\partial c_M}{\partial \mathbf{x}_N} & \frac{\partial c_M}{\partial \boldsymbol{\theta}_N} J_L(\boldsymbol{\theta}_N)^{-1} \end{pmatrix}. \quad (3.37)$$

Using this notation, we can summarize the equations of motion with constraints as

$$\frac{d}{dt} (M(\mathbf{X})\mathcal{V}) = \mathbf{F} + (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda}. \quad (3.38)$$

The result of correctly using the left trivialized derivative $\mathcal{D}\mathbf{c}$ is that the angular equations are in the space of angular velocity, rather than the axis/angle space.

3.3 Non-Holonomic Constraints

Non-Holonomic Constraints are constraints that are a function of velocities $\dot{\mathbf{X}}$ as well as configuration $\mathbf{X}$. These are handled differently than holonomic constraints. As described in [9] and [15], if you try to add the non-holonomic constraints to the Lagrangian and take the variation, as we did with holonomic constraints, you get the incorrect answer. The reason is that, as the Lagrange multipliers represent physical constraint forces, substituting the constraints into the Lagrangian before differentiation effectively assumes that the constraint forces can be derived from a potential, which is not necessarily true for non-holonomic systems.

Instead, we can use the Lagrange-d'Alembert Principle. This states that the non-conservative constraint forces do no virtual work. This modifies our Lagrangian variation by adding an extra term to enforce this condition

$$\delta \int L(\dot{\mathbf{X}}, \mathbf{X}) dt + \int \mathbf{Q} \cdot \delta \mathbf{X} = 0 \quad (3.39)$$

where $\mathbf{Q}$ are the non-conservative constraint forces.

If we have a non-holonomic constraint of the form $c(\dot{\mathbf{X}}, \mathbf{X}) = \mathbf{a}(\mathbf{X}) \cdot \dot{\mathbf{X}} = 0$ this means that we are preventing displacement in the direction $\mathbf{a}(\mathbf{X})$, so the constraint force $\mathbf{Q}$ must be proportional to that direction $Q_i = \lambda_i \mathbf{a}_i(\mathbf{X})$.

Therefore, these non-holonomic constraints can be added to the system as

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{\mathbf{X}}} \right) - \frac{\partial L}{\partial \mathbf{X}} = \mathbf{a} \cdot \boldsymbol{\lambda} = \left(\frac{\partial c}{\partial \dot{\mathbf{X}}} \right)^T \boldsymbol{\lambda}. \quad (3.40)$$

I.e. exactly the same form as the holonomic constraints, but with the constraint function differentiated with respect to the velocity $\dot{\mathbf{X}}$.

Note that, as with the previous section, we have to be careful about distinguishing between differentiation with respect to the axis/angle variable $\dot{\boldsymbol{\theta}}$ and the angular velocity $\boldsymbol{\omega}$. The constraint torque needs to be multiplied by J_L^{-T} in order to bring it to world space

$$\boldsymbol{\tau}_w = J_L^{-T} \left(\frac{\partial c}{\partial \dot{\boldsymbol{\theta}}} \right)^T \boldsymbol{\lambda} \quad (3.41)$$

But also

$$\frac{\partial c}{\partial \dot{\boldsymbol{\theta}}} = \frac{\partial c}{\partial \boldsymbol{\omega}} \frac{\partial \boldsymbol{\omega}}{\partial \dot{\boldsymbol{\theta}}} = \frac{\partial c}{\partial \boldsymbol{\omega}} J_L \quad (3.42)$$

since $\boldsymbol{\omega} = J_L \dot{\boldsymbol{\theta}}$. So the constraint torque in world space is

$$\left(\frac{\partial c}{\partial \boldsymbol{\omega}} \right)^T \boldsymbol{\lambda}. \quad (3.43)$$

As before with the holonomic constraints, we extend the definition of $\dot{\mathcal{D}}$ to include linear derivatives.

The left trivialized velocity derivative is defined as

$$\dot{\mathcal{D}}\mathbf{c} = \begin{pmatrix} \frac{\partial c_1}{\partial \dot{\mathbf{x}}_1} & \frac{\partial c_1}{\partial \dot{\boldsymbol{\omega}}_1} & \cdots & \frac{\partial c_1}{\partial \dot{\mathbf{x}}_N} & \frac{\partial c_1}{\partial \dot{\boldsymbol{\omega}}_N} \\ \vdots & & \ddots & & \\ \frac{\partial c_M}{\partial \dot{\mathbf{x}}_1} & \frac{\partial c_M}{\partial \dot{\boldsymbol{\omega}}_1} & \cdots & \frac{\partial c_M}{\partial \dot{\mathbf{x}}_N} & \frac{\partial c_M}{\partial \dot{\boldsymbol{\omega}}_N} \end{pmatrix}. \quad (3.44)$$

With this, the constrained equations of motion for both holonomic and non-holonomic constraints are

$$\frac{d}{dt} (M(\mathbf{X})\mathcal{V}) = \mathbf{F} + (\mathcal{D}\mathbf{c}_h)^T \boldsymbol{\lambda}_h + (\dot{\mathcal{D}}\mathbf{c}_{nh})^T \boldsymbol{\lambda}_{nh}, \quad (3.45)$$

4 Solving the Constrained Equations of Motion

In this section we show how to solve the system of equations of motion as a first order approximation and show the relationship between this solution and

1. A Pseudo-Inverse approach to solving the constraint equations
2. A Sequential Impulse approach
3. Traditional Position Based Dynamics

4.1 Solving the Unconstrained Equations of Motion

We solve the constrained equations of motion incrementally as a first order approximation to the solution to the unconstrained (zero constraint forces) equations, so first we will briefly cover integrating the unconstrained system.

Finding the incremental time advancement of the unconstrained equations of motion to find the provisional state $\mathbf{X}^{t+1}$, $\dot{\mathbf{X}}^{t+1}$ can be done with a standard numerical integration scheme. A common choice in real-time rigid body simulation is a semi-implicit (symplectic) Euler integration scheme. This is a simple first order scheme with low cost and high stability and long-term energy behaviour compared with explicit Euler. In this scheme, first the velocity is updated, then the new velocity is used to update the position.

For linear degrees of freedom the equation of motion is

$$m\ddot{\mathbf{x}} = \mathbf{f} \quad (4.1)$$

and the discrete integration step for symplectic Euler is

$$\begin{aligned} \dot{\mathbf{x}}^{t+1} &= \dot{\mathbf{x}}^t + \frac{1}{m} \mathbf{f} \Delta t, \\ \mathbf{x}^{t+1} &= \mathbf{x}^t + \dot{\mathbf{x}}^{t+1} \Delta t. \end{aligned} \quad (4.2)$$

For angular degrees of freedom the equation of motion is

$$\frac{d}{dt}(I\boldsymbol{\omega}) = \boldsymbol{\tau} \quad (4.3)$$

and the discrete integration step for symplectic Euler is

$$\begin{aligned} \boldsymbol{\omega}^{t+1} &= \boldsymbol{\omega}^t + I^{-1} (\boldsymbol{\tau} - \boldsymbol{\omega}^t \times (I\boldsymbol{\omega}^t)) \Delta t, \\ q^{t+1} &= e^{\frac{1}{2}\boldsymbol{\Omega}^{t+1}\Delta t} q^t = e^{-\frac{1}{2}I\boldsymbol{\omega}^{t+1}\Delta t} q^t \end{aligned} \quad (4.4)$$

where the inertia $I(t)$ is evaluated at the current configuration at t and we are using the quaternion representation for orientation. We could equally choose to use the rotation matrix representation where

$$R^{t+1} = e^{[\boldsymbol{\omega}^{t+1}\Delta t] \times} R^t \approx (\mathbb{1} + [\boldsymbol{\omega}^{t+1}\Delta t] \times) R^t \quad (4.5)$$

but prefer the quaternion approach for reasons discussed in section §2.2.

Note that, for simplicity, we have shown the gyroscopic term $\boldsymbol{\omega} \times I\boldsymbol{\omega}$ evaluated at time t , i.e. it is treated explicitly. Unlike the linear equation of motion, this term is nonlinear in the angular velocity

and couples the rotational degrees of freedom when the inertia is non-spherical. If the angular velocity is high relative to the timestep, this explicit treatment can therefore cause instability. In such cases, the gyroscopic term can instead be treated implicitly, requiring a solve for $\boldsymbol{\omega}^{t+1}$, or the timestep can be reduced through sub-stepping. The choice between these approaches is primarily a practical numerical consideration.

For the remainder of this section we only require that a solution at $t + 1$ can be found that is first order accurate in Δt . The particular integration scheme used to obtain this provisional unconstrained state does not affect the subsequent constraint derivation.

4.2 Finding a First Order Approximate Solution

Now, given the solution to the unconstrained equations of motion, we assume that the solution to the constrained equations at $t + 1$ is given by a small perturbation $\Delta \mathbf{X}$ away from the solution to the unconstrained equations at $t + 1$. I.e. for $\boldsymbol{\lambda} = 0$ we solve the unconstrained equations to find $\mathbf{X}^{t+1}, \dot{\mathbf{X}}^{t+1}$ using a standard rigid body integration scheme, then we apply a perturbation $\Delta \mathbf{X}$ where

$$\Delta \mathbf{X} = \begin{pmatrix} \Delta \mathbf{x}_i \\ \Delta \boldsymbol{\omega}_i \\ \vdots \\ \Delta \mathbf{x}_N \\ \Delta \boldsymbol{\omega}_N \end{pmatrix} \quad (4.6)$$

and we define

$$\Delta \boldsymbol{\omega}_i = J_L(\boldsymbol{\theta}_i) \Delta \boldsymbol{\theta}_i. \quad (4.7)$$

This is interpreted as an angular displacement. I.e. a small change in angular velocity integrated over time Δt .

With this definition, the updated configuration is

$$\begin{aligned} \mathbf{x}_i^{t+1} &\rightarrow \mathbf{x}_i^{t+1} + \Delta \mathbf{x}_i, & R_i^{t+1} &\rightarrow e^{[\Delta \boldsymbol{\omega}_i] \times} R_i^{t+1}, \\ \mathcal{V}_i^{t+1} &\rightarrow \mathcal{V}_i^{t+1} + \Delta \mathbf{X}_i / \Delta t. \end{aligned} \quad (4.8)$$

We solve the constrained equations to first order in $\Delta \mathbf{X}$ and $\boldsymbol{\lambda}$ to find an approximate solution to the constrained equations at $t + 1$. As long as the time step is small, and so the displacement due to applying the constraint forces over Δt is small, then this remains a valid approximation.

Start with the linear degrees of freedom. We have

$$\begin{aligned} \frac{1}{\Delta t} (m(\dot{\mathbf{x}}^{t+1} \Delta t + \Delta \mathbf{x}) - m\dot{\mathbf{x}}^{t+1} \Delta t) &= \mathbf{f}(\mathbf{x}^{t+1} + \Delta \mathbf{x}) \Delta t + (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda} \Delta t \\ &\Rightarrow M \Delta \mathbf{x} = (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda} \Delta t^2, \end{aligned} \quad (4.9)$$

where

$$M = m\mathbb{1} - \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \Delta t^2 \quad (4.10)$$

as we have assumed that the unconstrained equation is satisfied at $(\mathbf{x}^{t+1}, \dot{\mathbf{x}}^{t+1})$. Now expand the constraint equation, as this must be satisfied at $\mathbf{x}^{t+1} + \Delta \mathbf{x}$

$$\mathbf{c}(\mathbf{x}^{t+1} + \Delta \mathbf{x}) = \mathbf{c}(\mathbf{x}) + (\mathcal{D}\mathbf{c}) \Delta \mathbf{x} + \mathcal{O}(\Delta \mathbf{x})^2 = 0. \quad (4.11)$$

Taking these together we find an equation for $\boldsymbol{\lambda}$

$$(\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T) \boldsymbol{\lambda} \Delta t^2 = -\mathbf{c}(\mathbf{x}^{t+1}). \quad (4.12)$$

We solve this for λ and then obtain Δx from

$$\Delta x = M^{-1} (\mathcal{D}c)^T \lambda \Delta t^2. \quad (4.13)$$

Now we do the same for the angular equations of motion. Start with

$$\frac{d}{dt} (I(\theta)\omega) = \tau(\theta) + (\mathcal{D}c)^T \lambda. \quad (4.14)$$

Expanding, using the assumption that $\theta^{t+1}, \dot{\theta}^{t+1}$ solve the unconstrained equation, we get

$$\begin{aligned} & \frac{1}{\Delta t} (I(\theta^{t+1} + \Delta\theta)(\omega^{t+1}\Delta t + \Delta\omega) - I(\theta^{t+1})\omega^{t+1}\Delta t) = \tau_w(\theta^{t+1} + \Delta\theta)\Delta t + (\mathcal{D}c)^T \lambda \Delta t \\ \Rightarrow & (\mathbb{1} + [\Delta\omega]_{\times})I(\theta^{t+1})(\mathbb{1} - [\Delta\omega]_{\times})(\omega^{t+1}\Delta t + \Delta\omega) - I(\theta^{t+1})\omega^{t+1}\Delta t \\ & = \tau_w(\theta^{t+1})\Delta t^2 + (\mathcal{D}\tau)^T \Delta\omega \Delta t^2 + (\mathcal{D}c)^T \lambda \Delta t^2. \end{aligned} \quad (4.15)$$

which simplifies to

$$\tilde{I}\Delta\omega = (\mathcal{D}c)^T \lambda \Delta t^2, \quad (4.16)$$

Where

$$\tilde{I} = I + I[\omega^{t+1}\Delta t]_{\times} - [I\omega^{t+1}\Delta t]_{\times} - (\mathcal{D}\tau)^T \Delta t^2 \quad (4.17)$$

is the modified inertia matrix. Note that for spherical inertia or for small $\omega^{t+1}\Delta t$ this reduces to the standard world space inertia. This modification is a result of the precession term in the equations of motion and means, for instance, a spinning body configured as a gyroscope will be more difficult to rotate when applying the $\Delta\theta$ due to the constraint correction.

In practice, this term can usually be ignored, as the time step Δt is taken to be small and so $|\omega\Delta t| \ll 1$ and usually inertias in rigid body simulation are reasonably uniform $I_{ii}/I_{jj} \sim 1$. These assumptions break down for fast spinning objects with highly non-spherical inertia, where gyroscopic effects are important. In this case, although the motion can be correctly modelled if we include the $\omega \times I\omega$ term in the integration, constraint interactions will be incorrect. The left hand side of the angular equation above is the change in angular momentum $I(\theta + \Delta\theta)(\omega + \Delta\omega/\Delta t) - I(\theta)\omega$ produced by the correction, and $\tilde{I}$ is the Jacobian of the map $\Delta\omega \mapsto I(\theta + \Delta\theta)\Delta\omega$: the extra terms are there because rotating the body also rotates its inertia. Dropping them means, for example, that a constraint impulse applied to a fast spinning body produces a simple tilt rather than the precession that the gyroscopic coupling should give, with an error of order $|\omega\Delta t|$ per step.

A perhaps more important argument for ignoring this term is that its addition makes the mass matrix non-symmetric, which means that the matrix $\mathcal{D}c I^{-1}(\mathcal{D}c)^T$ is not necessarily symmetric. This means that the system is no longer guaranteed to converge, as we explore in the next section. For these reasons, we will omit this correction term in our analysis from now on, and assume $\tilde{I} = I$. It is easy to add it back though by just replacing I with $\tilde{I}$, but the convergence analysis will not be valid in this case.

Back to the constraint function, we again expand to first order

$$c(\theta^{t+1} + \Delta\theta) = c(\theta^{t+1}) + \mathcal{D}c \Delta\omega + \mathcal{O}(\Delta t^2) = 0, \quad (4.18)$$

and solving gives

$$\begin{aligned} \left(\mathcal{D}\mathbf{c} \, I^{-1} (\mathcal{D}\mathbf{c})^T \right) \boldsymbol{\lambda} \Delta t^2 &= -c(\boldsymbol{\theta}^{t+1}), \\ \Delta \boldsymbol{\omega} &= I^{-1} (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda} \Delta t^2. \end{aligned} \quad (4.19)$$

This brings the angular and linear equations into the same form. We can then combine all degrees of freedom together into a single linear system.

First solve the following linear system to find $\boldsymbol{\lambda}$

$$\left(\mathcal{D}\mathbf{c} \, M^{-1} (\mathcal{D}\mathbf{c})^T \right) \boldsymbol{\lambda} \Delta t^2 = -c(\mathbf{X}^{t+1}) \quad (4.20)$$

and then use this to solve for $\Delta \mathbf{X}$

$$\Delta \mathbf{X} = M^{-1} (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda} \Delta t^2, \quad (4.21)$$

This equation for $\boldsymbol{\lambda}$ which is then used to solve for $\Delta \mathbf{X}$ is a linear system that can be solved using a number of numerical methods.

4.3 An Iterative Solution Using Gauss-Seidel

We now turn to the method for solving the system of equations iteratively. We need to solve $\boldsymbol{\lambda}$ from

$$\left(\mathcal{D}\mathbf{c} \, M^{-1} (\mathcal{D}\mathbf{c})^T \right) \boldsymbol{\lambda} \Delta t^2 = -c(\mathbf{X}^{t+1}), \quad (4.22)$$

which can then be used to compute $\Delta \mathbf{X}$ using

$$\Delta \mathbf{X} = M^{-1} (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda} \Delta t^2. \quad (4.23)$$

The equation is a linear system $A\boldsymbol{\lambda} = -\mathbf{c}$, where $A = \left(\mathcal{D}\mathbf{c} \, M^{-1} (\mathcal{D}\mathbf{c})^T \right) \Delta t^2$ is a square matrix. Writing out the rows explicitly, this is

$$\begin{aligned} A_{11}\lambda_1 + A_{12}\lambda_2 + \cdots + A_{1m}\lambda_m &= -c_1, \\ A_{21}\lambda_1 + A_{22}\lambda_2 + \cdots + A_{2m}\lambda_m &= -c_2, \\ &\vdots \\ A_{m1}\lambda_1 + A_{m2}\lambda_2 + \cdots + A_{mm}\lambda_m &= -c_m \end{aligned}$$

which can be solved using the Gauss-Seidel method. See, for example, [36].

We start with an initial guess of $\boldsymbol{\lambda}^{(0)} = 0$, then solve each equation i in turn for λ_i using the latest value from previous solutions, and then iterate to a solution:

$$\begin{aligned}
\lambda_1^{(1)} &= \frac{1}{A_{11}} \left(-c_1 - A_{12}\lambda_2^{(0)} - \dots - A_{1m}\lambda_m^{(0)} \right), \\
\lambda_2^{(1)} &= \frac{1}{A_{22}} \left(-c_2 - A_{21}\lambda_1^{(1)} - A_{23}\lambda_3^{(0)} \dots - A_{2m}\lambda_m^{(0)} \right), \\
&\vdots \\
\lambda_i^{k+1} &= \frac{1}{A_{ii}} \left(-c_i - \sum_{j<i} A_{ij}\lambda_j^{k+1} - \sum_{j>i} A_{ij}\lambda_j^k \right), \\
&= \lambda_i^k - \frac{1}{A_{ii}} (\mathbf{c} + A\boldsymbol{\lambda}^*)_i,
\end{aligned} \tag{4.24}$$

where λ^* is the latest value of all of the λ s.

For our system this is

$$\lambda_i^{k+1} \Delta t^2 = \lambda_i^k \Delta t^2 - \left(\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T \right)_{ii}^{-1} \left(\mathbf{c}(\mathbf{X}^{t+1}) + \left(\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T \right) \lambda^* \Delta t^2 \right)_i. \tag{4.25}$$

Using (4.23) we can update the displacements incrementally per iteration

$$\Delta \mathbf{X}'_j = \Delta \mathbf{X}_j + M_j^{-1} (\mathcal{D}\mathbf{c})_{ij}^T (\boldsymbol{\lambda}^{k+1} - \boldsymbol{\lambda}^k)_i \Delta t^2 \tag{4.26}$$

and use this to rewrite the update for λ_i as

$$\lambda_i^{k+1} \Delta t^2 = \lambda_i^k \Delta t^2 - \left(\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T \right)_{ii}^{-1} (\mathbf{c}(\mathbf{X}^{t+1}) + \mathcal{D}\mathbf{c} \Delta \mathbf{X})_i. \tag{4.27}$$

Summary:

The equations of motion can be solved using a Gauss-Seidel update. We start with an initial guess for a solution for $\boldsymbol{\lambda}$, typically $\boldsymbol{\lambda} = 0$, and set all of the displacements $\Delta \mathbf{X} = 0$. Then for $k = 0$ to $N - 1$ iterations we solve one row at a time for each constraint's Lagrange multiplier using the current value of the displacement

$$\lambda_i^{k+1} \Delta t^2 = \lambda_i^k \Delta t^2 - \left(\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T \right)_{ii}^{-1} (\mathbf{c}(\mathbf{X}^{t+1}) + \mathcal{D}\mathbf{c} \Delta \mathbf{X})_i. \tag{4.28}$$

and then update the current displacement

$$\Delta \mathbf{X}'_j = \Delta \mathbf{X}_j + M_j^{-1} (\mathcal{D}\mathbf{c})_{ij}^T (\boldsymbol{\lambda}^{k+1} - \boldsymbol{\lambda}^k)_i \Delta t^2. \tag{4.29}$$

In the end we are left with a set of displacements that resolve the constraints and can be use to update the configuration of the system to give a first order solution to the constrained equations of motion at time $t + 1$

$$\begin{aligned}
\dot{\mathbf{x}}_i^{t+1} &\rightarrow \dot{\mathbf{x}}_i^{t+1} + \Delta \mathbf{x}_i / \Delta t, \\
\boldsymbol{\omega}_i^{t+1} &\rightarrow \boldsymbol{\omega}_i^{t+1} + \Delta \boldsymbol{\omega}_i / \Delta t, \\
\mathbf{x}_i^{t+1} &\rightarrow \mathbf{x}_i^{t+1} + \Delta \mathbf{x}_i, \\
q(\boldsymbol{\theta}^{t+1}) &\rightarrow e^{-\frac{1}{2} I \Delta \boldsymbol{\omega}_i q(\boldsymbol{\theta}^{t+1})}
\end{aligned} \tag{4.30}$$

4.4 Comparison to the Jacobi Method

The Jacobi method is similar to the Gauss-Seidel method except that instead of feeding the result from the previous row into the solution for the next row, each row is solved independently using the solution from the previous iteration. In our system this means instead of the Gauss-Seidel update

$$\lambda_i^{k+1} = \frac{1}{A_{ii}} \left(-c_i - \sum_{j < i} A_{ij} \lambda_j^{k+1} - \sum_{j > i} A_{ij} \lambda_j^k \right), \quad (4.31)$$

we would have a Jacobi update of

$$\lambda_i^{k+1} = \frac{1}{A_{ii}} \left(-c_i - \sum_{j \neq i} A_{ij} \lambda_j^k \right). \quad (4.32)$$

The Jacobi update is known to converge slower than Gauss-Seidel, which we will demonstrate with a simple example in section §10, but as solving each λ is independent, the update can be parallelized much more easily than Gauss-Seidel, making it popular for GPU based simulations.

One known convergence problem for Jacobi is that, for instance, when solving a system with multiple contact constraints per body pair, as each constraint is solved independently, the system tends to over correct the position each iteration. A useful trick to alleviate this is *mass splitting* where the effective mass is divided by the number of constraint points [34] making the total per iteration correction correct.

4.5 Convergence

It's useful to look at the convergence criteria for the Gauss-Seidel update of the linear system (4.20). If we take the unmodified inertia, then the matrix $A = \mathcal{D}c M^{-1}(\mathcal{D}c)^T$ is symmetric positive definite (SPD), since M is SPD. I.e. for any $\mathbf{x}$, $\mathbf{x} \cdot A\mathbf{x} \geq 0$. A is strictly positive definite if $(\mathcal{D}c)^T \mathbf{x} = 0$ implies that $\mathbf{x} = 0$. I.e. $\mathcal{D}c$ has full row rank, meaning that all of the constraints are linearly independent. The Gauss-Seidel update will converge for any SPD matrix A , so our system will always converge if all of the constraints are linearly independent.

To show this, we construct an energy function

$$E(\boldsymbol{\lambda}) = \frac{1}{2} \boldsymbol{\lambda}^T A \boldsymbol{\lambda} + \boldsymbol{\lambda}^T \mathbf{c}. \quad (4.33)$$

This is a convex function for SPD matrix A with a unique minimum at $A\boldsymbol{\lambda}^* = -\mathbf{c}$. Taking a single λ_i and fixing the energy for all of the other λ_j s the energy as a function of λ_i is

$$E(\lambda_i) = \frac{1}{2} A_{ii} |\lambda_i|^2 + \lambda_i \left(\sum_{j \neq i} A_{ij} \lambda_j + c_i \right), \quad (4.34)$$

and minimizing with respect to λ_i gives

$$\frac{\partial E(\lambda_i)}{\partial \lambda_i} = A_{ii} \lambda_i + \sum_{i \neq j} A_{ij} \lambda_j + c_i = \mathbf{r}_i = 0 \quad (4.35)$$

for a residual $\mathbf{r}_i$. The value of λ_i^{k+1} that sets this residual to zero is

$$\begin{aligned}\lambda_i^{k+1} &= -A_{ii}^{-1} \left(\sum_{j \neq i} A_{ij} \lambda_j^k + c_i \right), \\ &= \lambda_i^k - A_{ii}^{-1} (A\boldsymbol{\lambda} + \mathbf{c})_i,\end{aligned}\tag{4.36}$$

i.e. exactly the iterative update we defined in (4.28).

Using $e_i = (A\boldsymbol{\lambda} + \mathbf{c})_i$ we compute the change in energy

$$\begin{aligned}E(\lambda_i^{k+1}) - E(\lambda_i^k) &= \frac{1}{2} A_{ii} |\lambda_i^k - A_{ii}^{-1} e_i|^2 + (\lambda_i^k - A_{ii}^{-1} e_i) \left(\sum_{j \neq i} A_{ij} \lambda_j^k + c_i \right) \\ &\quad - \frac{1}{2} A_{ii} |\lambda_i^k|^2 + \lambda_i^k \left(\sum_{j \neq i} A_{ij} \lambda_j^k + c_i \right), \\ &= -\lambda_i^k e_i + \frac{1}{2} A_{ii}^{-1} |e_i|^2 - A_{ii}^{-1} e_i \left(\sum_{j \neq i} A_{ij} \lambda_j^k + c_i \right), \\ &= -\frac{1}{2} A_{ii}^{-1} |e_i|^2.\end{aligned}\tag{4.37}$$

Since A is SPD then $A_{ii} > 0$ and so the energy decreases each iteration. Combining this with the fact that the energy is bounded below by the minimum $A\boldsymbol{\lambda}^* = -\mathbf{c}$ means that the solution converges to $\boldsymbol{\lambda}^*$.

4.5.1 Rate of convergence

To look at the rate of convergence we write the update as

$$\lambda_i^{k+1} = A_{ii}^{-1} \left(-c_i - \sum_{j < i} A_{ij} \lambda_j^{k+1} - \sum_{j > i} A_{ij} \lambda_j^k \right)\tag{4.38}$$

as, with Gauss-Seidel, the result from solving a previous row of λ_j^{k+1} with $j < i$ is fed into the update for λ_i^{k+1} . So this is

$$A_{ii} \lambda_i^{k+1} + \sum_{j < i} A_{ij} \lambda_j^{k+1} = - \sum_{j > i} A_{ij} \lambda_j^k - c_i.\tag{4.39}$$

Writing the matrix A in terms of its diagonal D , its lower triangular matrix L and upper U , where $U = L^T$ due to A being symmetric, we have

$$(L + D)\boldsymbol{\lambda}^{k+1} = -L^T \boldsymbol{\lambda}^k - \mathbf{c}\tag{4.40}$$

so

$$\boldsymbol{\lambda}^{k+1} = -(L + D)^{-1} L^T \boldsymbol{\lambda}^k - (L + D)^{-1} \mathbf{c}.\tag{4.41}$$

Letting

$$\boldsymbol{\epsilon}^k = \boldsymbol{\lambda}^k - \boldsymbol{\lambda}^*\tag{4.42}$$

be the error at iteration k from the solution $\boldsymbol{\lambda}^*$ defined by $A\boldsymbol{\lambda}^* = -\mathbf{c}$, then

$$\epsilon^{k+1} = -(D + L)^{-1}L^T\epsilon^k. \quad (4.43)$$

So, for an initial error ϵ_0 the error goes like

$$\epsilon^k = G^k\epsilon_0 \quad (4.44)$$

for

$$G = -(D + L)^{-1}L^T. \quad (4.45)$$

This means that the convergence of the system depends on the spectral radius $\rho(G)$. Express ϵ in terms of the basis of eigenvalues of G

$$\epsilon = \sum_i c_i v_i \quad (4.46)$$

for constants c_i and eigenvalues v_i . From this, if α_i are the eigenvalues of G then

$$\epsilon^k = \sum_i \alpha_i^k c_i v_i. \quad (4.47)$$

So the error becomes dominated by the largest eigenvalue α_i . This needs to be less than one for the error to shrink, hence the convergence criterion is

$$\rho(G) < 1 \quad (4.48)$$

and how fast the system will converge depends on how small the spectral radius of G is.

In practice, it is often more practical to use the diagonal dominance of the matrix A in order to give an intuition on the rate of convergence of the system. Strict diagonal dominance is defined as

$$|A_{ii}| > \sum_{j \neq i} |A_{ij}|. \quad (4.49)$$

The claim is that for a strictly diagonally dominant matrix A the Gauss-Seidel update will converge, and the more diagonally dominant, the faster the rate of convergence. To see this consider the update for the first row

$$\epsilon_1^{k+1} = -A_{11}^{-1} \sum_{j>1} A_{1j} \epsilon_j^k. \quad (4.50)$$

Taking absolute values

$$\begin{aligned} |\epsilon_1^{k+1}| &\leq |A_{11}|^{-1} \sum_{j>1} |A_{1j}| |\epsilon_j^k|, \\ &\leq r_1 \|\epsilon^k\|_{\max}, \end{aligned} \quad (4.51)$$

where

$$r_1 = \frac{\sum_{j>1} |A_{1j}|}{|A_{11}|}, \quad (4.52)$$

which for a diagonally dominant matrix is less than 1, and $\|\epsilon^k\|_{\max}$ is the maximum error from the previous iteration. So

$$|\epsilon_1^{k+1}| < \|\epsilon^k\|_{\max}. \quad (4.53)$$

The error for the second row is

$$\epsilon_2^{k+1} = -A_{22}^{-1} \left(A_{21}\epsilon_1^{k+1} + \sum_{j>2} A_{2j}\epsilon_j^k \right). \quad (4.54)$$

Taking absolutes

$$\begin{aligned} |\epsilon_2^{k+1}| &\leq |A_{22}|^{-1} \left(|A_{21}||\epsilon_1^{k+1}| + \sum_{j>2} |A_{2j}||\epsilon_j^k| \right), \\ &\leq \left(\frac{|A_{21}|}{|A_{22}|} + \frac{\sum_{j>2} |A_{2j}|}{|A_{22}|} \right) \|\epsilon^k\|_{\max}, \end{aligned} \quad (4.55)$$

where we have used the previous result for ϵ_1^{k+1} .

Continuing, row by row we get

$$|\epsilon_i^{k+1}| \leq \frac{\sum_{j \neq i} |A_{ij}|}{|A_{ii}|} \|\epsilon^k\|_{\max} = r_i \|\epsilon^k\|_{\max} \quad (4.56)$$

and overall

$$\|\epsilon^{k+1}\|_{\max} \leq r \|\epsilon^k\|_{\max} \quad (4.57)$$

where

$$r = \max_i \frac{\sum_{j \neq i} |A_{ij}|}{|A_{ii}|}. \quad (4.58)$$

So, if A is strictly diagonally dominant then $r < 1$ and each iteration the error is reduced, hence the system converges.

Summary:

The Gauss-Seidel algorithm for finding the solution to $A\mathbf{x} = \mathbf{b}$ is guaranteed to converge for any symmetric positive definite matrix A . For the constrained equations of motion $A = \mathcal{D}\mathbf{c} M^{-1}(\mathcal{D}\mathbf{c})^T$ is symmetric, since the mass matrix is symmetric, and positive definite if the constraints are linearly independent, and so $\mathcal{D}\mathbf{c}$ has full row rank.

The rate of convergence depends on the spectral radius $\rho(G)$ of the Gauss-Seidel iteration matrix $G = -(D+L)^{-1}L^T$, where we split $A = (D+L+L^T)$ into its diagonal, lower triangular and upper triangular parts. The per iteration error goes like

$$\|\epsilon^k\| \lesssim \rho(G)^k \|\epsilon_0\|. \quad (4.59)$$

Diagonal dominance is determined by the ratio

$$r = \max_i \frac{\sum_{j \neq i} |A_{ij}|}{|A_{ii}|}. \quad (4.60)$$

The more diagonally dominant the matrix is (the smaller the value of r) the faster the rate of convergence.

4.6 A More Accurate Update Using Non-Linear Gauss-Seidel

We can improve on the result (4.28) by noticing that the term $\mathbf{c}(\mathbf{X}^{t+1}) + \mathcal{D}\mathbf{c} \Delta\mathbf{X}$ is the first order expansion of $\mathbf{c}(\mathbf{X}^{t+1} + \Delta\mathbf{X})$, i.e. the constraint function evaluated at the most recent value of the displacement $\Delta\mathbf{X}$.

In order to obtain a more accurate update, we can replace the linearized term $\mathbf{c}(\mathbf{X}^{t+1}) + \mathcal{D}\mathbf{c} \Delta\mathbf{X}$ with the full constraint evaluation at the latest version of $\Delta\mathbf{X}$.

$$\lambda_i^{k+1} \Delta t^2 = \lambda_i^k \Delta t^2 - \left(\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T \right)_{ii}^{-1} \mathbf{c}(\mathbf{X}^{t+1} + \Delta\mathbf{X})_i, \quad (4.61)$$

This turns the update from a linear solution to (4.22) into a non-linear iterative algorithm.

This is now straying from the standard Gauss-Seidel method, but it is demonstrated to converge better when applied in [24]. This non-linear update is particularly important when solving rigid body systems with rotation when the time step is large, as errors due to using a linear approximation to rotations can be large, leading to inaccuracies in the solution and instability.

Another thing that is often done to improve the convergence of the result is to recompute the constraint derivatives in the “resistance” term $\left(\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T \right)_{ii}^{-1}$ for each iteration. However, this is often expensive, and having an accurate measure of the error is the more important factor in improving stability and the robustness of the result in the case of large angular velocities and/or time steps.

Given that this update strays from the linearized Gauss-Seidel update, we can’t use the same convergence arguments that we did in the previous section. However, if the constraints are smooth then the constraint function is well approximated by its first order approximation near the solution and the nonlinear Gauss-Seidel update converges asymptotically to the linear Gauss-Seidel update and the local convergence analysis holds, and if the constraint row is diagonally dominant then we are still reducing the per iteration error in the non-linear regime.

What we gain by this method though is that we are no longer accumulating linearization error, and so the frame to frame stability in cases of large angular velocity is greatly improved. This is the practical reason for the better convergence observed in [24].

4.7 Adding Non-Holonomic Constraints

We saw in 3.3 that the equations of motion including non-holonomic constraints follow the holonomic constraint formulation but now with the constraint force

$$(\dot{\mathcal{D}}\mathbf{c})^T \boldsymbol{\lambda}. \quad (4.62)$$

The derivation of the linearized update directly follows the holonomic constraint derivation, leading to a per iteration update of

$$\begin{aligned} \lambda_i^{k+1} \Delta t &= \lambda_i^k \Delta t - \left(\dot{\mathcal{D}}\mathbf{c} M^{-1} (\dot{\mathcal{D}}\mathbf{c})^T \right)_{ii}^{-1} \mathbf{c}(\dot{\mathbf{X}} + \Delta\mathbf{X}^k / \Delta t)_i, \\ \Delta\mathbf{X}^{k+1} &= \Delta\mathbf{X}^k + M^{-1} (\dot{\mathcal{D}}\mathbf{c})^T (\lambda^{k+1} - \lambda^k) \Delta t^2. \end{aligned} \quad (4.63)$$

So with this update we can see that velocity constraints and position constraints can be part of the same solver loop, since they are both incrementally updating the displacement $\Delta \mathbf{X}$. Alternatively, we can treat the velocity solve as being separate from the position solve and solve the position constraints first, then solve the velocity constraints, with the $\Delta \mathbf{X}$'s from the velocity solve only updating the velocities and not the positions.

Algorithm 1 Solver Update

- 1: **Input:** Predicted state $\mathbf{X}^{t+1}$, mass matrix M , timestep Δt
- 2: **Warm Starting:** $\lambda^0 = \eta \lambda^{\text{prev}}$, for $0 \leq \eta \leq 1$, $\Delta \mathbf{X}^0 = M^{-1}(\mathcal{D}\mathbf{c})^T \lambda^0 \Delta t^2$
- 3: **for** each constraint i **do**
- 4: Compute the constraint derivative $\mathcal{D}\mathbf{c}$ and effective mass

$$\mu_i = \left(\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T \right)_{ii}^{-1}$$

- 5: **end for**
- 6: **for** $k = 0$ to $N - 1$ **do**
- 7: **for** each constraint i **do**
- 8: Compute the Lagrange multiplier update using the latest value of the constraint error

$$\lambda_i^{k+1} \Delta t^2 = \lambda_i^k \Delta t^2 - \xi \mu_i c(\mathbf{X}^{t+1} + \Delta \mathbf{X}^k)$$

- 9: Update the incremental displacement

$$\Delta \mathbf{X}_j^{k+1} = \Delta \mathbf{X}_j^k + M_j^{-1} (\mathcal{D}\mathbf{c})_{ij} (\lambda_i^{k+1} - \lambda_i^k) \Delta t^2$$

- 10: **end for**
- 11: **end for**
- 12: **State Update:**

$$\mathbf{x}^{t+1} \rightarrow \mathbf{x}^{t+1} + \Delta \mathbf{x}$$

$$\mathbf{q}^{t+1} \rightarrow e^{-\frac{1}{2} I \Delta \omega} \mathbf{q}^{t+1}$$

$$\dot{\mathbf{x}}^{t+1} \rightarrow \dot{\mathbf{x}}^{t+1} + \Delta \mathbf{x} / \Delta t$$

$$\boldsymbol{\omega}^{t+1} \rightarrow \boldsymbol{\omega}^{t+1} + \Delta \boldsymbol{\omega} / \Delta t$$

4.8 Regularization

A useful extension to the method of Lagrange multipliers is a method of regularization by applying a quadratic term in the Lagrange multiplier to the Lagrangian. The idea is to add a term $\frac{1}{2}\boldsymbol{\lambda} \cdot \mathbf{E}\boldsymbol{\lambda}$ to the Lagrangian with $\mathbf{E} = \text{diag}\{\epsilon_1, \dots, \epsilon_M\}$ being a diagonal matrix of small constant values that ensures the Lagrange multipliers (and hence the constraint forces) remain small

$$L(\mathbf{X}, \dot{\mathbf{X}}, \boldsymbol{\lambda}) \rightarrow \frac{1}{2}\dot{\mathbf{X}} \cdot M(\mathbf{X})\dot{\mathbf{X}} - V(\mathbf{X}) + \boldsymbol{\lambda} \cdot \mathbf{c}(\mathbf{X}) + \frac{1}{2}\boldsymbol{\lambda} \cdot \mathbf{E}\boldsymbol{\lambda}. \quad (4.64)$$

This new term alters the constraint equation

$$\mathbf{c}(\mathbf{X}^{t+1} + \Delta\mathbf{X}) = \mathbf{c}(\mathbf{X}^{t+1}) + \mathcal{D}\mathbf{c} \Delta\mathbf{X} + \mathbf{E}\boldsymbol{\lambda} = 0. \quad (4.65)$$

The result of regularization is that equation to solve for $\boldsymbol{\lambda}$ now becomes

$$\left(\epsilon_i \mathbb{1} + \mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T \right)_{ii} \lambda_i \Delta t^2 = -c_i(\mathbf{X}^{t+1}). \quad (4.66)$$

for some small ϵ_i

This addition term ensures that if $\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T$ becomes singular, for instance if there are degenerate constraints, then the inversion of the matrix for $\boldsymbol{\lambda}$ does not also become singular. This is particularly important if the matrix solver uses a direct solve for all or part of the matrix.

This type of regularization is generally called *Tikhonov Regularization* [32], and is equivalent to compliance in XPBD, as we will see later.

5 Interpretation and Connections to Other Known Methods

5.1 Moore-Penrose Inverse and Least Squares Minimization

The Moore-Penrose inverse [26] (also called the pseudoinverse) of a matrix A^+ is a generalization of the matrix inverse for non-square matrices. For a full row rank matrix A (so that (AA^T) is invertible) this is defined as

$$A^+ = A^T(AA^T)^{-1}. \quad (5.1)$$

The Lagrange multiplier method gave us a solution for $\Delta \mathbf{X}$ of

$$\Delta \mathbf{X} = -M^{-1}(\mathcal{D}\mathbf{c})^T \left(\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T \right)^{-1} \mathbf{c}(\mathbf{X}^{t+1}). \quad (5.2)$$

If M were equal to the identity then this would reduce to

$$\Delta \mathbf{X} = -(\mathcal{D}\mathbf{c})^T \left(\mathcal{D}\mathbf{c} (\mathcal{D}\mathbf{c})^T \right)^{-1} \mathbf{c}(\mathbf{X}^{t+1}). \quad (5.3)$$

This is exactly the Moore-Penrose inverse solution to the constraint equation at first order in $\Delta \mathbf{X}$

$$\mathcal{D}\mathbf{c} \Delta \mathbf{X} = -\mathbf{c}(\mathbf{X}^{t+1}). \quad (5.4)$$

So the method of Lagrange multipliers applied to the first order approximation of the constrained equations of motion reduces to a mass weighted version of the pseudoinverse of the constraint Jacobian.

The linearization of the constrained equations of motion is equivalent to solving the constraint equations subject to a least squares minimization of $\Delta \mathbf{X}$ with a distance measure using “metric” M .

$$\min_{\Delta \mathbf{X}} \frac{1}{2} \Delta \mathbf{X}^T \cdot M \Delta \mathbf{X} \quad (5.5)$$

subject to the constraint

$$\mathcal{D}\mathbf{c} \Delta \mathbf{X} = -\mathbf{c}(\mathbf{X}^{t+1}). \quad (5.6)$$

We are finding a solution that satisfies the constraint equations while minimizing the energy term $\frac{1}{2} \Delta \mathbf{X} \cdot M \Delta \mathbf{X}$.

This is an extension of the Moore-Penrose Pseudoinverse method $A^+ = A^T(AA^T)^{-1}$ to a space with an inner product $\langle, \rangle_M$.

Tikhonov regularization can be used together with the pseudoinverse [4] for ill posed problems where $(AA^T)^{-1}$ does not exist. In this case for small ϵ the inverse is corrected by

$$A^+ = A^T(\epsilon \mathbb{1} + AA^T)^{-1} \quad (5.7)$$

and, applying this to the solution of the constraint equations with a metric M we recover our previous result

$$\Delta \mathbf{X} = -M^{-1}(\mathcal{D}\mathbf{c})^T \left(\epsilon \mathbb{1} + \mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T \right)^{-1} \mathbf{c}(\mathbf{X}^{t+1}) \quad (5.8)$$

5.2 Sequential Impulses

We rewrite (4.61) by introducing the quantity

$$\mathbf{j} = (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda} \Delta t, \quad (5.9)$$

which has dimensions $[MLT^{-1}]$, i.e. of an impulse. Using the position update $\Delta \mathbf{X} = M^{-1} (\mathcal{D}\mathbf{c})^T \Delta t^2 \boldsymbol{\lambda}$, we obtain

$$\mathbf{j} = M \frac{\Delta \mathbf{X}}{\Delta t}, \quad (5.10)$$

so that $\mathbf{j}$ represents the generalized impulse satisfying $\mathbf{j} = M \Delta \dot{\mathbf{X}}$.

Define

$$\mu_i = \left(\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T \right)_{ii}^{-1}, \quad (5.11)$$

The scalar μ_i is the effective mass, measuring the resistance of the system to motion in the direction that resolves the i -th constraint.

With these substitutions the Gauss–Seidel update becomes

$$\mathbf{j}_i^{k+1} = \mathbf{j}_i^k - \frac{\mu_i}{\Delta t} \left((\mathcal{D}\mathbf{c})^T \mathbf{c}(\mathbf{X}^{t+1} + M^{-1} \mathbf{j}^k \Delta t) \right)_i. \quad (5.12)$$

To relate this to a force-based formulation, consider the quadratic penalty potential

$$V(\mathbf{X}) = \frac{1}{2} k \|\mathbf{c}(\mathbf{X})\|^2. \quad (5.13)$$

Its associated force is

$$\mathbf{F} = -k (\mathcal{D}\mathbf{c})^T \mathbf{c}. \quad (5.14)$$

Substituting into the previous expression yields

$$\mathbf{j}_i^{k+1} = \mathbf{j}_i^k + \left(\frac{\mu_i}{k \Delta t^2} \right) \mathbf{F}^k \Delta t. \quad (5.15)$$

The factor Δt^2 arises from converting a position-level penalty into an impulse-level update. As a dimensional check, if $\mathbf{c}$ represents a distance constraint so that $[\mathbf{c}] = L$, then $[k] = MT^{-2}$ and $[\mu_i] = M$, making $\mu_i/(k \Delta t^2)$ dimensionless as required.

If we identify $\mathbf{j} = (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda} \Delta t$ as the impulse produced by the constraint, then the Gauss–Seidel update can be interpreted as incrementally applying impulses based on the per iteration configuration, and is equivalent to the sequential impulse method [10].

5.3 Position Based Dynamics

The equation (6.1) for the Lagrange multipliers λ_i together with the subsequent $\Delta \mathbf{X}$ update (4.23) can be directly related to the original position based dynamics update [25]. In this paper, they dealt with only point particles, so no angular degrees of freedom, and they defined an iterative update of the positions as

$$\delta \mathbf{p}_i = -s w_i \nabla_{\mathbf{p}_i} C(\mathbf{p}_i, \dots, \mathbf{p}_n), \quad (5.16)$$

with

$$s = \frac{C(\mathbf{p}_i, \dots, \mathbf{p}_n)}{\sum_i w_i |\nabla_{\mathbf{p}_i} C(\mathbf{p}_i, \dots, \mathbf{p}_n)|^2} \quad (5.17)$$

and $w_i = 1/m_i$.

In our notation, this is just

$$s = \left(\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T \right)^{-1} \mathbf{c}(\mathbf{x}^{t+1} + \Delta\mathbf{x}^k) \quad (5.18)$$

and

$$\begin{aligned} \lambda_i^{k+1} \Delta t^2 &= \lambda_i^k \Delta t^2 - s, \\ \Delta\mathbf{x}^{k+1} &= \Delta\mathbf{x}^k + M^{-1} (\mathcal{D}\mathbf{c})^T (\lambda_i^{k+1} - \lambda_i^k) \Delta t^2. \end{aligned} \quad (5.19)$$

In the PBD paper, they introduce a constraint compliance by multiplying the corrections at each iteration by a constant k . In our formulation, we can see that this is equivalent to adding a relaxation $k = \xi$ to the update. So, it is obvious that the compliance is just slowing the convergence of the hard constraint, making it effectively a soft constraint, but with the softness dependent on the number of iterations. The authors address this in their paper using a modified stiffness $k' = 1 - (1 - k)^n$, where n is the number of iterations.

The Gauss-Seidel method applied to solve the linearized constrained equations of motion reproduces the PBD algorithm, exactly so when the constraint derivative $\mathcal{D}\mathbf{c}$ and effective mass are re-evaluated at the current iterate as PBD does (Algorithm 1 computes them once at $\mathbf{X}^{t+1}$).

5.4 Discrete Variational Mechanics

The previous section derived PBD by starting from the continuous Euler-Lagrange equations of motion then perturbing the solution of the unconstrained equations and finding a first order correction that satisfies the constrained equations of motion. This provides a direct route to the algorithm, but it leaves open the relationship between this procedure and the variational formulation from which the continuous equations of motion were derived.

A discrete variational formulation provides this connection. Rather than first discretising the equations of motion, the action itself is discretised. The resulting discrete Euler-Lagrange equations define a discrete dynamical system, and the constrained version provides a good reference against which the PBD approximation can be compared.

Consider a mechanical system with configuration $\mathbf{x}$ and Lagrangian

$$L(\mathbf{x}, \dot{\mathbf{x}}) = \frac{1}{2} \dot{\mathbf{x}}^T M \dot{\mathbf{x}} - V(\mathbf{x}), \quad (5.20)$$

where M is the mass matrix. The continuous action is

$$S = \int_{t_0}^{t_1} L(\mathbf{x}, \dot{\mathbf{x}}), dt. \quad (5.21)$$

Discrete variational mechanics replaces the continuous path by a sequence of configurations

$$\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_N \quad (5.22)$$

at times $t_k = k\Delta t$, and introduces a *discrete Lagrangian* L_d satisfying

$$L_d(\mathbf{x}_k, \mathbf{x}_{k+1}) \approx \int_{t_k}^{t_{k+1}} L(\mathbf{x}, \dot{\mathbf{x}}), dt. \quad (5.23)$$

The discrete action is therefore

$$S_d = \sum_{k=0}^{N-1} L_d(\mathbf{x}_k, \mathbf{x}_{k+1}). \quad (5.24)$$

Requiring the discrete action to be stationary with respect to variations of the interior configurations gives the discrete Euler–Lagrange equations,

$$D_2 L_d(\mathbf{x}_{k-1}, \mathbf{x}_k) + D_1 L_d(\mathbf{x}_k, \mathbf{x}_{k+1}) = 0, \quad (5.25)$$

where D_i denotes differentiation with respect to the i -th argument.

For the Lagrangian above, consider the discrete Lagrangian

$$L_d(\mathbf{x}_k, \mathbf{x}_{k+1}) = \Delta t \left(\frac{1}{2} \left(\frac{\mathbf{x}_{k+1} - \mathbf{x}_k}{\Delta t} \right)^T M \left(\frac{\mathbf{x}_{k+1} - \mathbf{x}_k}{\Delta t} \right) - V(\mathbf{x}_k) \right). \quad (5.26)$$

The derivatives of this discrete Lagrangian are

$$D_1 L_d = -M \frac{\mathbf{x}_{k+1} - \mathbf{x}_k}{\Delta t} - \Delta t \nabla V(\mathbf{x}_k), \quad (5.27)$$

and

$$D_2 L_d = M \frac{\mathbf{x}_{k+1} - \mathbf{x}_k}{\Delta t}. \quad (5.28)$$

Substitution into (5.25) gives

$$M \frac{\mathbf{x}_{k+1} - 2\mathbf{x}_k + \mathbf{x}_{k-1}}{\Delta t^2} = -\nabla V(\mathbf{x}_k). \quad (5.29)$$

So the unconstrained prediction used in the preceding sections can be viewed as the discrete Euler–Lagrange equation of a discrete action.

Constraints can be incorporated into this scheme as before by using Lagrange multipliers.

$$L_d(\mathbf{x}_k, \mathbf{x}_{k+1}, \boldsymbol{\lambda}_k) = L_d(\mathbf{x}_k, \mathbf{x}_{k+1}) + \Delta t \boldsymbol{\lambda}_k^T \mathbf{c}(\mathbf{x}_k). \quad (5.30)$$

Variation with respect to $\boldsymbol{\lambda}_k$ gives

$$\mathbf{c}(\mathbf{x}_k) = 0, \quad (5.31)$$

while variation with respect to $\mathbf{x}_k$ gives

$$M \frac{\mathbf{x}_{k+1} - 2\mathbf{x}_k + \mathbf{x}_{k-1}}{\Delta t^2} = -\nabla V(\mathbf{x}_k) + J_k^T \boldsymbol{\lambda}_k, \quad (5.32)$$

where

$$J_k = \left. \frac{\partial \mathbf{c}}{\partial \mathbf{x}} \right|_{\mathbf{x}_k} \quad (5.33)$$

Equation (5.32) is a discrete variational constrained dynamics problem and is the basis of constraint methods such as SHAKE [28]. PBD can be understood as a first-order approximation to this problem.

Let $\tilde{\mathbf{x}}_{k+1}$ denote the unconstrained prediction obtained from the discrete Euler–Lagrange equations. We want to find a new configuration

$$\mathbf{x}_{k+1} = \tilde{\mathbf{x}}_{k+1} + \Delta \mathbf{x}. \quad (5.34)$$

That solves the constraint equations

$$\mathbf{c}(\mathbf{x}_{k+1}) = 0. \quad (5.35)$$

Rather than solving the full nonlinear constrained discrete equations, approximate the constraint to first-order about the configuration $\mathbf{x}_k$,

$$\mathbf{c}(\mathbf{x}_{k+1}) = \mathbf{c}(\mathbf{x}_k) + J_k(\mathbf{x}_{k+1} - \mathbf{x}_k) + \cdots. \quad (5.36)$$

Consequently,

$$0 = \mathbf{c}(\mathbf{x}_k) + J_k(\tilde{\mathbf{x}}_{k+1} - \mathbf{x}_k) + J_k \Delta \mathbf{x} + \cdots. \quad (5.37)$$

From (5.32) we have

$$\Delta \mathbf{x} = \Delta t^2 M^{-1} J_k^T \boldsymbol{\lambda}_k. \quad (5.38)$$

Substituting into the linearized constraint gives

$$J_k M^{-1} J_k^T \boldsymbol{\lambda}_k = -\frac{1}{\Delta t^2} (\mathbf{c}(\mathbf{x}_k) + J_k(\tilde{\mathbf{x}}_{k+1} - \mathbf{x}_k)). \quad (5.39)$$

Hence

$$\Delta \mathbf{x} = -M^{-1} J_k^T (J_k M^{-1} J_k^T)^{-1} [\mathbf{c}(\mathbf{x}_k) + J_k(\tilde{\mathbf{x}}_{k+1} - \mathbf{x}_k)]. \quad (5.40)$$

The discrete variational formulation clarifies a point that we brushed over the PBD formulation. Since the constrained discrete Lagrangian (5.32) evaluates the constraint at $\mathbf{x}_k$, its variation produces the Jacobian

$$J_k = \left. \frac{\partial \mathbf{c}}{\partial \mathbf{x}} \right|_{\mathbf{x}_k}. \quad (5.41)$$

The first-order approximation therefore uses the constraint gradient at the beginning of the time step. The constraint itself is still evaluated at the predicted configuration, but its linearization is taken about $\mathbf{x}_k$. This is in contrast to PBD implementations that tend to evaluate the Jacobian at the predicted state.

The nice thing about this approach is that the discrete equations are obtained by varying a discrete action, so the resulting evolution preserves the symplectic structure of the continuous system, rather than just approximating the continuous equations of motion.

Note, this section has been on the flat vector space of positions. For more on Lie group discrete variational methods see, e.g. [22].

6 Techniques to Improve Convergence and Stability

6.1 Relaxation

A well known modification to the Gauss-Seidel algorithm is to introduce a relaxation parameter ξ . This can be used to control how much of the error per constraint is fixed up per iteration, and thus can be used to control the convergence of the system.

To implement relaxation we modify the λ update (4.61) with the addition of a relaxation factor ξ

$$\lambda_i^{k+1} \Delta t^2 = \lambda_i^k \Delta t^2 - \xi \left(\mathcal{D} \mathbf{c} M^{-1} (\mathcal{D} \mathbf{c})^T \right)_{ii}^{-1} \mathbf{c}(\mathbf{X}^{t+1} + \Delta \mathbf{X}^k)_i, \quad (6.1)$$

The relaxation factor scales the per frame correction to λ .

This relaxation factor can help with the convergence of the system. Under-relaxation with $\xi < 1$ can allow for systems that are otherwise divergent to converge, and over-relaxation $\xi > 1$ can help speed up the convergence of an already convergent system.

Previously, in (4.37) we saw the change in energy per iteration $\Delta E = -\frac{1}{2} A_{ii}^{-1} |e_i|^2$. With the addition of the relaxation parameter, this becomes

$$\Delta E = -\xi \left(1 - \frac{\xi}{2} \right) A_{ii}^{-1} |e_i|^2. \quad (6.2)$$

So the energy decreases, and the system converges, if

$$0 < \xi < 2 \quad (6.3)$$

With the additional relaxation factor, the iteration matrix goes to

$$G_\xi = (D + \xi L)^{-1} ((1 - \xi)D - \xi L^T), \quad (6.4)$$

which reduces to the regular Gauss-Seidel iteration matrix for $\xi = 1$.

Under-Relaxation

If $\xi < 1$ this is called under-relaxation. With relaxation the update of the per iteration error (4.57) goes to

$$|\epsilon^{k+1}| \leq (1 - \xi(1 - r_i)) \|\epsilon^k\|_{\max} \quad (6.5)$$

so converges for

$$1 - \xi(1 - r_i) < 1. \quad (6.6)$$

This means that having $\xi < 1$ worsens the convergence rate but at the trade off of greater stability. We are reducing the per constraint per iteration step size, fixing less of the error each frame. This slows convergence but can also reduce potential oscillation between competing constraint targets or errors due to highly non-linear constraints.

Over-Relaxation

If $1 < \xi < 2$ this is called successive over-relaxation, or SOR. We are effectively extrapolating the error, which can speed up convergence, but may cause divergence depending on how large the spectral radius of the iteration matrix is.

The optimal relaxation factor ξ^* for a given system is related to the spectral radius of the Jacobi iteration matrix $\rho(G_J)$ by [37].

$$\xi^* \sim \frac{2}{1 + \sqrt{1 - \rho(G_J)^2}}. \quad (6.7)$$

where, from (4.32) it can be seen that the Jacobi iteration matrix is

$$G_J = -D^{-1}(L + L^T). \quad (6.8)$$

Under special conditions where the matrix $A = \mathcal{D}\mathbf{c} M^{-1}(\mathcal{D}\mathbf{c})^T$ is consistently ordered, e.g. has a block tri-diagonal structure (as it would have for a simple chain of constrained bodies) the spectral radius of the Jacobi matrix is related to the spectral radius of the Gauss-Seidel matrix by $\rho(G_{GS}) = \rho(G_J)^2$.

If the spectral radius of the iteration matrix is low (which will be true if A is diagonally dominant) then an optimal value of ξ can be found in the range $1 < \xi < 2$ that can help to speed up convergence.

Adaptive Relaxation

A good balance is to modify the relaxation per iteration based on the measured convergence of the system. If $|\epsilon^{k+1}| < |\epsilon^k|$ then the system is converging and we can increase ξ . If $|\epsilon^{k+1}| \geq |\epsilon^k|$ the system is diverging and we can try to recover by decreasing ξ .

6.2 Warm Starting

The iteration loop presented in Algorithm 1 sets the Lagrange multipliers to zero at the start of each update. This is a safe choice as the Lagrange multipliers correspond to the constraint forces and the goal is to find a minimum set of constraint forces to simultaneously solve all of the constraints. However, for stable systems where the constraints are not changing from frame to frame, such as solving a chain of connected bodies, the result from the previous frame is often a good guess for what the result will be for the subsequent frame, so we can speed up convergence by starting the next frame's solve using the resulting Lagrange multipliers from the previous frame. For stable systems we are effectively allowing for convergence of the system over multiple frames, which can mean greater stability and reduce the number of iterations required per frame. However, for highly dynamic systems where constraints are changing significantly between frames this can produce instability as we can end up injecting forces into the system that do not solve the new set of constraints well.

Concretely, if the set of Lagrange multipliers from the previous frame was $\boldsymbol{\lambda}^{prev}$, then set the initial Lagrange multipliers for the next frame (the initial guess for the solution to the system of equations) as

$$\boldsymbol{\lambda}^0 = \eta \boldsymbol{\lambda}^{prev}, \quad 0 \leq \eta \leq 1. \quad (6.9)$$

In practice, for systems where the set of constraints is not changing much from frame to frame - e.g. a stack of bodies, or a chain of constraints - setting η to be large, say $0.8 - 1.0$ is reasonable.

After initializing λ^0 it is also important to initialize $\Delta \mathbf{X}$ with the initial guess before the iteration loop begins:

$$\Delta \mathbf{X}^0 = M^{-1}(\mathcal{D}\mathbf{c})^T \lambda^0 \Delta t^2. \quad (6.10)$$

Warm starting does not improve the rate of convergence or change the convergence analysis of section §4.5 - the Gauss-Seidel update converges for any SPD matrix - what it does is to potentially reduce the initial error. For temporally coherent systems the previous result λ^{prev} is often a much better guess to the next frame's solution than starting from zero. Warm starting effectively allows the system to converge over multiple frames. Over N frames with k iterations the original error decays like $\rho(G)^{Nk}$ so, it is possible to solve at a lower number of iterations and amortize the cost of converging the system over multiple frames.

Contact Caching

Warm starting assumes that we have a consistent set of constraints and that the solution is only slowly changing from frame to frame. Contact constraints are generally regenerated each frame, so warm starting requires identifying contacts from one frame to the next. This is typically done via proximity to a previous frame contact, or, more robustly, via some form of feature id matching.

Warm starting sets

$$\begin{aligned} \lambda^0 &= \eta \lambda^{prev}, 0 \leq \eta \leq 1, \\ \Delta \mathbf{X}^0 &= M^{-1}(\mathcal{D}\mathbf{c})^T \lambda^0 \Delta t^2 \end{aligned} \quad (6.11)$$

before starting the iteration loop.

It does not affect the rate of convergence but does affect the initial error, and can amortize the cost of convergence over multiple frames, allowing each frame to run at a lower number of iterations.

It requires identifying constraints between frames and can inject energy when the constraints are changing rapidly.

6.3 Sub-stepping

So far we have considered the solution of a system going from a state at t to a state at $t + \Delta t$ in a single step. A common method of increasing accuracy and stability of the solution, particularly for large steps is to use sub-stepping, where a step of Δt is divided up into n smaller steps of $\Delta t/n$. Typically, collision detection is performed once per frame and then the system is solved for the same set of constraints over n steps of length $\Delta t/n$:

1. Perform collision detection and generate contact constraints.
2. Set the substep size $\Delta t = \Delta T/n$.
3. For each substep $s = 1, \dots, n$:
 - (a) Integrate the unconstrained equations of motion over Δt to obtain the predicted state $\mathbf{X}^{t+1}, \dot{\mathbf{X}}^{t+1}$.

(b) Solve the constraint system at the predicted state and apply the state update.

See, for example, [21]. In this paper, they explore reducing the number of iterations and increasing the number of sub-steps.

The benefits of sub-stepping are:

- **Smaller initial error:** The constraint violation produced by one unconstrained integration scales with Δt . Error $\sim \mathbf{v}_{\text{rel}} \Delta t$ for some relative velocity. Halving the time step halves the initial error. Where the relative velocity is zero, e.g. for stacking, then the error scales with the acceleration - Error $\sim g \Delta t^2$, where g is the gravity acceleration. In these cases the error shrinks quadratically, while the number of steps increases linearly.
- **Smaller linearization error:** For the linear version of the solver the rotational error is of order $\mathcal{O}(\omega \Delta t)$. Sub-stepping reduces this error. For the non-linear solver the error is computed exactly, so this benefit does not apply, although sub-stepping does mean that other solver terms, like the constraint derivative $\mathcal{D}\mathbf{c}$ are computed each sub-step making them more accurate.
- **Weaker gyroscopic coupling:** The correction $\tilde{I}$ we applied to the inertia in (4.17) differs from I by factors of order $\mathcal{O}(\omega \Delta t)$. The additional terms came from the gyroscopic terms in the Euler equation $\boldsymbol{\omega} \times I \boldsymbol{\omega}$. We dropped the modified inertia due to it making the matrix to solve non-SPD. Sub-stepping makes this approximation more valid.

Per frame there are two errors: solver error $\rho(G)^k \epsilon_0$ and discretization error — the $\mathcal{O}(\Delta t^2)$ from linearizing the constraint c about $\mathbf{X}^{t+1}$, from the $\mathcal{O}(\omega \Delta t)$ rotation linearization, and from $\mathcal{D}\mathbf{c}$ evaluated once per step. Iterations only touch the first. As k increases and the system converges the frame error is bounded by an $\mathcal{O}(\Delta t)$ floor that no iteration count can fix. Sub-stepping lowers this floor.

For resting or near-equilibrium constraints the initial error is $\epsilon_0 \sim g \Delta t^2$. Consider a fixed budget of n Gauss-Seidel iterations per frame of length ΔT , and suppose each iteration removes a fraction $1 - \rho$ of the accumulated error while the unconstrained integration adds $\sim g \Delta t^2$ per step. The residual then obeys $e_{s+1} = \rho(e_s + g \Delta t^2)$, with steady state $e_\infty = \rho g \Delta t^2 / (1 - \rho)$; with k iterations per step the same holds with $\rho \rightarrow \rho^k$. Hence

- n substeps $\times$ 1 iteration: $e_\infty = \frac{\rho}{1 - \rho} g \left(\frac{\Delta T}{n} \right)^2$
- 1 step $\times$ n iterations ($\Delta t = \Delta T$): $e_\infty = \frac{\rho^n}{1 - \rho^n} g \Delta T^2$

The ratio of the iterated to the sub-stepped residual is

$$\frac{n^2(1 - \rho) \rho^{n-1}}{1 - \rho^n} \rightarrow n \quad \text{as } \rho \rightarrow 1, \quad (6.12)$$

since $1 - \rho^n \approx n(1 - \rho)$. Sub-stepping is therefore roughly n times better when the system is poorly conditioned. E.g for $\rho = 0.9, n = 5$ the ratio is ~ 4.0 . For $\rho = 0.5, n = 5$ the ratio is ~ 0.8 . So sub-stepping wins in cases where there is work to do to converge the system. When the system is close to convergence iterating wins, but in that case we are already close to the solution so it doesn't matter. We cover convergence in more detail in §10.

6.4 Baumgarte Stabilization

The position based methods in these notes correct constraint error at the position level: the displacement $\Delta \mathbf{X}$ modifies the configuration directly. Historically, however, most rigid body engines solved constraints purely at the velocity level, i.e. using only the non-holonomic constraints of section §3.3. At the velocity level, position error is invisible: a velocity solve maintains $\dot{\mathbf{c}} = 0$ but does nothing about $\mathbf{c} \neq 0$, so drift accumulated from integration and linearization error grows without bound. Joints slowly pull apart and contacts sink.

The classical fix is Baumgarte stabilization [6], which replaces the velocity level constraint $\dot{\mathbf{c}} = 0$ with

$$\dot{\mathbf{c}}(\mathbf{X}, \dot{\mathbf{X}}) + \frac{\beta}{\Delta t} \mathbf{c}(\mathbf{X}) = 0, \quad 0 < \beta \leq 1, \quad (6.13)$$

so that the constraint velocity is driven towards a value that removes a fraction β of the position error over one timestep. (In the continuous setting this is $\dot{\mathbf{c}} + \gamma \mathbf{c} = 0$, giving exponential decay of the error $\mathbf{c}(t) \sim e^{-\gamma t}$; the discrete form identifies $\gamma = \beta/\Delta t$.)

In the PBD framework presented here this is a non-holonomic constraint with a configuration dependent bias. The velocity derivative is unchanged, so the effective mass is unchanged, and the per iteration update becomes

$$\lambda_i^{k+1} \Delta t = \lambda_i^k \Delta t - \xi \left(\dot{\mathbf{D}}_c M^{-1} \left(\dot{\mathbf{D}}_c \right)^T \right)^{-1}_{ii} \left[\dot{\mathbf{c}}(\dot{\mathbf{X}}^{t+1} + \Delta \mathbf{X}^k / \Delta t) + \frac{\beta}{\Delta t} \mathbf{c}(\mathbf{X}^{t+1}) \right]_i. \quad (6.14)$$

Comparing this to the position level update (6.1), the position error term is the same. The crucial difference is where the correction is applied. In the Baumgarte scheme the entire correction, including the part sourced by the position error $\beta \mathbf{c}$, is applied to the velocity, with the position only recovering in subsequent frames as the biased velocity is integrated. The correcting velocity is *real*: it persists after the solve.

This has two well known consequences. The first is energy injection: a body penetrating a surface receives an outward velocity proportional to the penetration depth, and if the penetration was created in a single frame (a fast impact with a large Δt) the body can be launched out of the surface. The second is interference with restitution: the bias velocity adds to the restitution target of the contact velocity constraint, making the effective coefficient of restitution depth dependent. Tuning β is then a compromise between drift (small β) and popping (large β).

PBD solves the position directly, essentially the $\beta = 1$ limit, and applies the correction to the position directly as well as the velocity. This avoids the one frame delay in correcting the error, but still retains the problem of velocity injection, which we will cover in the next section on split impulse.

Baumgarte stabilization feeds a fraction β of the position error into the velocity solve as a bias

$$\dot{\mathbf{c}} + \frac{\beta}{\Delta t} \mathbf{c} = 0.$$

The correcting velocity is real and persists after the solve, injecting energy and perturbing restitution. Position projection corrects the same error directly at the position level.

6.5 Split Impulse

Even within a position based scheme, the state update (4.30) couples position and velocity: the velocity is incremented by $\Delta \mathbf{X}/\Delta t$, so any displacement used to resolve position error also changes the momentum of the system. For constraint error that reflects a genuine dynamic interaction this is correct, and give position based methods their correct physical behaviour. However, some position error is an artifact of an imperfect method solving the system: penetration created by collision detection running at discrete times over a finite number of iterations, or joint drift accumulated from linearization error. Converting the correction of this error into velocity injects momentum that was never physically applied, producing the same popping artifacts as Baumgarte stabilization.

The split impulse method, see e.g. [10], decomposes the solve into an *"impulse"* that corrects both position and velocity, and one that only affects position.

$$\boldsymbol{\lambda} = \boldsymbol{\lambda}_{\text{vel}} + \boldsymbol{\lambda}_{\text{pos}}, \quad (6.15)$$

where the constraint error at the end of the step is split into the part generated during this step and the part that was already present at its start,

$$\mathbf{c}(\mathbf{X}^{t+1}) = [\mathbf{c}(\mathbf{X}^{t+1}) - \mathbf{c}(\mathbf{X}^t)] + \mathbf{c}(\mathbf{X}^t). \quad (6.16)$$

$\boldsymbol{\lambda}_{\text{vel}}$ removes the first bracket, the error created by the unconstrained motion over the step (to first order $\dot{\mathbf{c}} \Delta t$), which is a genuine dynamic interaction; $\boldsymbol{\lambda}_{\text{pos}}$ removes the second, the initial configuration error $\mathbf{c}(\mathbf{X}^t)$. Together they give $\mathbf{c}(\mathbf{X}^{t+1} + \Delta \mathbf{X}) = 0$ at convergence without correcting the same error twice. In a perfect system $\mathbf{c}(\mathbf{X}^t)$ would be zero as a result of the solve from the previous frame, but in reality the solve may not converge fully or the system could be modified between solves to introduce some initial configuration error.

Both are iterated with the same Gauss-Seidel machinery and the same effective masses. The difference is confined to the state update: $\boldsymbol{\lambda}_{\text{vel}}$ updates positions and velocities as in (4.30), while $\boldsymbol{\lambda}_{\text{pos}}$ produces a *pseudo-displacement* that updates positions only,

$$\begin{aligned} \mathbf{x}^{t+1} &\rightarrow \mathbf{x}^{t+1} + \Delta \mathbf{x}_{\text{vel}} + \Delta \mathbf{x}_{\text{pos}}, \\ \dot{\mathbf{x}}^{t+1} &\rightarrow \dot{\mathbf{x}}^{t+1} + \Delta \mathbf{x}_{\text{vel}}/\Delta t, \end{aligned} \quad (6.17)$$

and likewise for the angular terms. In the language of the sequential impulse interpretation, $\dot{\mathbf{j}}_{\text{pos}} = M \Delta \mathbf{X}_{\text{pos}}/\Delta t$ is a *pseudo-impulse*: it moves the configuration but carries no momentum into the next frame. This is the scheme anticipated at the end of the non-holonomic constraint discussion, where the position solve and velocity solve are separated, made precise: the position solve's $\Delta \mathbf{X}$ is withheld from the velocity update.

Note the contrast with pure position based dynamics, where the velocity is *defined* by the position change, $\dot{\mathbf{x}} = \Delta \mathbf{x}/\Delta t$, so that position correction always changes velocity. Split impulse is the explicit statement that these two roles of $\Delta \mathbf{X}$ can, and for artifact error should, be decoupled. The cost is that the pseudo-displacement does not participate in restitution or friction (usually desirable), and that the momentum conservation arguments of section §7.1 apply only to the $\boldsymbol{\lambda}_{\text{vel}}$ part: the pseudo-impulses are still equal and opposite per constraint, but they deliberately do not represent physical forces.

Interaction With Warm Starting

The split also makes it clear which part of the solution should be warm started, since the two components have different temporal characters. λ_{vel} represents sustained constraint forces – the weight of a stack transmitted through its contacts, the tension in a chain of joints etc. It is this quantity for which λ^{prev} is a good predictor, and it should be cached. λ_{pos} , however, represents the correction of *this frame's* initial error - if the previous frame's solve converged then the previous frame's position error should have already been removed so the position impulse would be zero. In practice however, the solver may not converge completely to solve the error exactly, and new initial error may be introduced (e.g. by artificially reconfiguring the system). Warm starting this initial error would double count the correction and manifest as jitter.

In an engine that does not split the impulse, the cached λ conflates the two components, and warm starting silently re-applies position corrections. This is one of the practical reasons that engines that warm start aggressively also tend to adopt a split impulse or a separate position correction pass: the split is what makes the cached quantity a pure, persistent force.

Split impulse decomposes the solution into a velocity part and a position part

$$\lambda = \lambda_{\text{vel}} + \lambda_{\text{pos}},$$

where the position part is applied as a pseudo-displacement that updates the configuration but not the velocity. This removes the energy injection from correcting artifact position error such as penetration, and identifies the warm-startable component of the solution: cache λ_{vel} , recompute λ_{pos} each frame.

7 Stiff Constraint Examples

7.1 Point to Point Constraint

Define a constraint between two bodies i and j such that the distance between a point $\mathbf{p}_i$ on body i and a point $\mathbf{p}_j$ on body j is zero.

Let $\mathbf{r}_0$ be the offset of the point $\mathbf{p}$ from the centre of mass $\mathbf{x}$ of body so that

$$\mathbf{p}(\mathbf{X}) = \mathbf{x} + R(\boldsymbol{\theta})\mathbf{r}_0 = \mathbf{x} + \mathbf{r}(\boldsymbol{\theta}), \quad (7.1)$$

The constraint function sets the distance between the two points to be zero

$$c(\mathbf{X}) = \|\mathbf{p}_i - \mathbf{p}_j\| = \|\delta\mathbf{p}\|. \quad (7.2)$$

We can find the constraint derivative by using $\dot{c} = \mathcal{D}c \mathcal{V}$ with

$$\mathcal{V} = \begin{pmatrix} \mathbf{x}_i \\ \boldsymbol{\omega}_i \\ \mathbf{x}_j \\ \boldsymbol{\omega}_j \end{pmatrix} \quad (7.3)$$

So computing

$$\begin{aligned} \dot{c}(\mathbf{X}) &= (\dot{\mathbf{p}}_i - \dot{\mathbf{p}}_j) \cdot \hat{\delta\mathbf{p}}, \\ &= (\dot{\mathbf{x}}_i + \boldsymbol{\omega}_i \times \mathbf{r}_i - \dot{\mathbf{x}}_j - \boldsymbol{\omega}_j \times \mathbf{r}_j) \cdot \hat{\delta\mathbf{p}} \end{aligned} \quad (7.4)$$

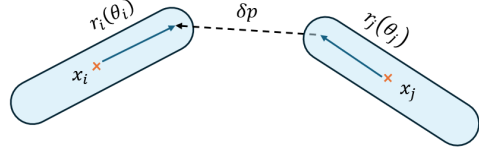
where $\hat{\delta\mathbf{p}}$ is the unit vector in the direction $\delta\mathbf{p}$

$$\hat{\delta\mathbf{p}} = \frac{\delta\mathbf{p}}{\|\delta\mathbf{p}\|} \quad (7.5)$$

gives

$$\mathcal{D}c = \begin{pmatrix} \hat{\delta\mathbf{p}}^T & (\mathbf{r}_i \times \hat{\delta\mathbf{p}})^T & -\hat{\delta\mathbf{p}}^T & -(\mathbf{r}_j \times \hat{\delta\mathbf{p}})^T \end{pmatrix}. \quad (7.6)$$

Note that, when the constraint is satisfied, the distance $\|\delta\mathbf{p}\| = 0$ and we cannot construct the unit vector $\hat{\delta\mathbf{p}}$. In this case, at $\delta\mathbf{p}^{t+1}$ can be used. If both are zero - the constraint is satisfied and there is no movement, then an arbitrary axis will have to be selected. If this is the case, then the constraint force will be zero unless there is some movement due to the resolution of another constraint. This only really affects the constructions of the correct effective mass. In practice, The vector $\delta\mathbf{p}(\mathbf{X}^{t+1} + \Delta\mathbf{X})$ will be constructed when solving the constraint.



Define the effective mass μ for this constraint by

$$\begin{aligned}\mu^{-1} &= \mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T, \\ &= \begin{pmatrix} \hat{\delta\mathbf{p}}^T & (\mathbf{r}_i \times \hat{\delta\mathbf{p}})^T & -\hat{\delta\mathbf{p}}^T & -(\mathbf{r}_j \times \hat{\delta\mathbf{p}})^T \end{pmatrix}\end{aligned}\quad (7.7)$$

$$\begin{aligned}&\begin{pmatrix} m_i \mathbb{1} & & & \\ & I_i & & \\ & & m_j \mathbb{1} & \\ & & & I_j \end{pmatrix}^{-1} \begin{pmatrix} \hat{\delta\mathbf{p}} \\ \mathbf{r}_i \times \hat{\delta\mathbf{p}} \\ -\hat{\delta\mathbf{p}} \\ -\mathbf{r}_j \times \hat{\delta\mathbf{p}} \end{pmatrix}, \\ &= \left(\frac{1}{m_i} + \frac{1}{m_j} \right) \mathbb{1} + (\mathbf{r}_i \times \hat{\delta\mathbf{p}}) \cdot I_i^{-1} (\mathbf{r}_i \times \hat{\delta\mathbf{p}}) + (\mathbf{r}_j \times \hat{\delta\mathbf{p}}) \cdot I_j^{-1} (\mathbf{r}_j \times \hat{\delta\mathbf{p}}), \\ &= \hat{\delta\mathbf{p}} \cdot \left(\left(\frac{1}{m_i} + \frac{1}{m_j} \right) \mathbb{1} - [\mathbf{r}_i]_{\times} I_i^{-1} [\mathbf{r}_i]_{\times} - [\mathbf{r}_j]_{\times} I_j^{-1} [\mathbf{r}_j]_{\times} \right) \hat{\delta\mathbf{p}}.\end{aligned}\quad (7.8)$$

This is the projection of the inverse of the effective mass matrix $\mathcal{M}^{-1}$ for the two body system along the direction $\hat{\delta\mathbf{p}}$.

Define the inverse of the effective mass as $\mathcal{M}^{-1}$ and its projection in the constraint direction as μ^{-1}

$$\mathcal{M}^{-1} = \left(\frac{1}{m_i} + \frac{1}{m_j} \right) \mathbb{1} - [\mathbf{r}_i]_{\times} I_i^{-1} [\mathbf{r}_i]_{\times} - [\mathbf{r}_j]_{\times} I_j^{-1} [\mathbf{r}_j]_{\times}, \quad (7.9)$$

$$\mu^{-1} = \hat{\delta\mathbf{p}} \cdot \mathcal{M}^{-1} \hat{\delta\mathbf{p}}. \quad (7.10)$$

The effective mass measures the resistance to movement of the two body system

Using this, the update is

$$\lambda^{k+1} \Delta t^2 = \lambda^k \Delta t^2 - \xi \mu \|\delta\mathbf{p}(\mathbf{X}^{t+1} + \Delta\mathbf{X})\|, \quad (7.11)$$

with

$$\begin{aligned}\Delta\mathbf{X}' &= \Delta\mathbf{X} + M^{-1} (\mathcal{D}\mathbf{c})^T (\lambda^{k+1} - \lambda^k) \Delta t^2, \\ &= \Delta\mathbf{X} + \begin{pmatrix} \frac{1}{m_i} \mathbb{1} & & & \\ & I_i^{-1} & & \\ & & \frac{1}{m_j} \mathbb{1} & \\ & & & I_j^{-1} \end{pmatrix} \begin{pmatrix} \hat{\delta\mathbf{p}} \\ \mathbf{r}_i \times \hat{\delta\mathbf{p}} \\ -\hat{\delta\mathbf{p}} \\ -\mathbf{r}_j \times \hat{\delta\mathbf{p}} \end{pmatrix} (\lambda^{k+1} - \lambda^k) \Delta t^2,\end{aligned}\quad (7.12)$$

Overall:

Constraint update for a point to point constraint:

Lagrange multiplier update:

$$\lambda^{k+1} \Delta t^2 = \lambda^k \Delta t^2 - \xi \mu \|\delta\mathbf{p}(\mathbf{X}^{t+1} + \Delta\mathbf{X})\|$$

Delta:

$$\Delta = -\xi\mu\delta\mathbf{p}(\mathbf{X}^{t+1} + \Delta\mathbf{X})$$

Configuration update:

$$\begin{aligned}\Delta\mathbf{x}'_i &= \Delta\mathbf{x}_i + \frac{1}{m_i}\Delta, \\ \Delta\boldsymbol{\omega}'_i &= \Delta\boldsymbol{\omega}_i + I_i^{-1}\mathbf{r}_i \times \Delta, \\ \Delta\mathbf{x}'_j &= \Delta\mathbf{x}_j - \frac{1}{m_j}\Delta, \\ \Delta\boldsymbol{\omega}'_j &= \Delta\boldsymbol{\omega}_j - I_j^{-1}\mathbf{r}_j \times \Delta.\end{aligned}\tag{7.13}$$

7.1.1 Comparison to an Impulse Based Approach

We can see that the constraint update is essentially an incremental impulse update

$$\begin{aligned}(m_i\Delta\mathbf{x}_i/\Delta t)' &= (m_i\Delta\mathbf{x}_i/\Delta t) + (\Delta/\Delta t), \\ (I_i\Delta\boldsymbol{\omega}_i/\Delta t)' &= (I_i\Delta\boldsymbol{\omega}_i/\Delta t) + \mathbf{r}_i \times (\Delta/\Delta t), \\ (m_j\Delta\mathbf{x}_j/\Delta t)' &= (m_j\Delta\mathbf{x}_j/\Delta t) - (\Delta/\Delta t), \\ (I_j\Delta\boldsymbol{\omega}_j/\Delta t)' &= (I_j\Delta\boldsymbol{\omega}_j/\Delta t) - \mathbf{r}_j \times (\Delta/\Delta t).\end{aligned}\tag{7.14}$$

The only difference being that in an incremental impulse update you would typically be computing the relative velocity between the two points on i and j , $\delta\dot{\mathbf{p}}$, rather than computing the change in position coordinates $\mathbf{x}$ and $\boldsymbol{\theta}$ then using that to find the exact error $\delta\mathbf{p}(\mathbf{X}^{t+1} + \Delta\mathbf{X})$ and dividing by the time step. A velocity/impulse based approach results in a linearization of this error, which can cause problems when the angular velocities are high.

7.2 Symmetries and Conservation of Momentum

The solution to the point to point constraint (7.13) preserves both linear and angular momentum. The change in linear momentum of the system is

$$\begin{aligned}\delta\mathbf{P} &= m_i\dot{\mathbf{x}}_i^{t+1} + m_j\dot{\mathbf{x}}_j^{t+1} - m_i\dot{\mathbf{x}}_i^t - m_j\dot{\mathbf{x}}_j^t, \\ &= m_i\Delta\mathbf{x}_i/\Delta t + m_j\Delta\mathbf{x}_j/\Delta t, \\ &= m_i\frac{1}{m_i}\mathbf{d} - m_j\frac{1}{m_j}\mathbf{d}, \\ &= 0.\end{aligned}\tag{7.15}$$

and the change in angular momentum about the origin is

$$\begin{aligned}\delta\mathbf{L} &= I_i\boldsymbol{\omega}_i^{t+1} + \mathbf{x}_i^{t+1} \times (m_i\dot{\mathbf{x}}_i^{t+1}) + I_j\boldsymbol{\omega}_j^{t+1} + \mathbf{x}_j^{t+1} \times (m_j\dot{\mathbf{x}}_j^{t+1}), \\ &= \mathbf{x}_i \times (m_i\Delta\mathbf{x}_i/\Delta t) + I_i\Delta\boldsymbol{\omega}_i/\Delta t + \mathbf{x}_j \times (m_j\Delta\mathbf{x}_j/\Delta t) + I_j\Delta\boldsymbol{\omega}_j/\Delta t, \\ &= (\mathbf{x}_i + \mathbf{r}_i - \mathbf{x}_j - \mathbf{r}_j) \times \mathbf{d}, \\ &= \delta\mathbf{p} \times \mathbf{d}, \\ &= 0.\end{aligned}\tag{7.16}$$

So, in order for the angular momentum to be preserved, $\mathbf{d}$ needs to be parallel to the vector between the two constrained points $\delta\mathbf{p}$, or the two constrained points need to be coincident, i.e. $\delta\mathbf{p} = 0$.

With our choice of constraint function $c(\mathbf{X}) = \|\delta\mathbf{p}(\mathbf{X})\|$, $\mathbf{d}$ is proportional to $\delta\mathbf{p}$ and so angular momentum is preserved. However, we could have chosen a different constraint function. For example, some engines [13] use a vector constraint $c(\mathbf{X}) = \delta\mathbf{p}(\mathbf{X})$. This also works, but there is a difference in the per iteration update. Instead of getting a scalar resistance μ_i the resistance is the full effective mass matrix $\mathcal{M}$. This means that $\mathbf{d}$ is now proportional to $\mathcal{M}\delta\mathbf{p}$, but, as $\mathcal{M}$ is a full 3×3 matrix with off diagonal elements, this is no longer necessarily parallel to $\delta\mathbf{p}$. Therefore, if the per iteration update does not leave $\delta\mathbf{p} = 0$ (say, there is some relaxation) then angular momentum will not be conserved, and is only conserved when the system converges.

The form of constraint required to ensure that the update conserves momentum can be understood using Noether's theorem, which relates symmetries of the action to conserved quantities [15].

Consider shifting all position coordinates by a small constant vector ϵ : $\mathbf{x}_i \rightarrow \mathbf{x}_i + \epsilon$. Then

$$\begin{aligned}
S &\rightarrow \int dt \left[L(\dot{\mathbf{x}}, \mathbf{x}) + \sum_i \frac{\partial L}{\partial \mathbf{x}_i} \cdot \epsilon + \lambda \cdot \left(c(\mathbf{x}) + \sum_i \frac{\partial c}{\partial \mathbf{x}_i} \epsilon \right) \right], \\
&= S + \int dt \sum_i \left[-\frac{\partial V}{\partial \mathbf{x}_i} + \left(\frac{\partial c}{\partial \mathbf{x}_i} \right)^T \lambda \right] \cdot \epsilon, \\
&= S + \int dt \sum_i \left[\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{\mathbf{x}}_i} \right) \right] \cdot \epsilon, \\
&= S + \int dt \frac{d}{dt} \left[\sum_i (m_i \dot{\mathbf{x}}_i) \right] \cdot \epsilon.
\end{aligned} \tag{7.17}$$

So, if the action is symmetric under $\mathbf{x} \rightarrow \mathbf{x} + \epsilon$, then that implies that $\sum_i (m_i \dot{\mathbf{x}}_i)$ is a constant, i.e. the total linear momentum of the system is conserved. This means we need constraints to be symmetric under translations in order to preserve linear momentum. E.g. a two body constraint should be a function of $(\mathbf{x}_i - \mathbf{x}_j)$.

The conservation of angular momentum follows in the same way from rotational symmetry of the action. Consider an infinitesimal rotation of the whole system by ϵ , $R(\epsilon) = e^{[\epsilon]_\times} \approx \mathbb{1} + [\epsilon]_\times$. Each body's orientation is left-multiplied, $R_i \rightarrow R(\epsilon)R_i$, so $\delta R_i R_i^T = [\epsilon]_\times$ and, by (2.40), the change in the axis-angle vector is $\delta\theta_i = J_L(\theta_i)^{-1}\epsilon$. The centres of mass rotate about the origin, so altogether,

$$\begin{aligned}
\delta\mathbf{x}_i &= \epsilon \times \mathbf{x}_i, \\
\delta\theta_i &= J_L(\theta_i)^{-1}\epsilon.
\end{aligned} \tag{7.18}$$

To find the quantity conserved by this symmetry, suppose $L + \lambda \cdot c$ is invariant under it, so that

$$\begin{aligned}
0 &= \delta \left(L(\dot{\mathbf{X}}, \mathbf{X}) + \lambda \cdot c(\mathbf{X}) \right) = \sum_i \left[\frac{\partial L}{\partial \dot{\mathbf{X}}_i} \cdot \delta \dot{\mathbf{X}}_i + \left(\frac{\partial L}{\partial \mathbf{X}_i} + \left(\frac{\partial c}{\partial \mathbf{X}_i} \right)^T \lambda \right) \cdot \delta \mathbf{X}_i \right], \\
&= \sum_i \left[\mathbf{P}_i \cdot \delta \dot{\mathbf{X}}_i + \dot{\mathbf{P}}_i \cdot \delta \mathbf{X}_i \right], \\
&= \frac{d}{dt} \sum_i \mathbf{P}_i \cdot \delta \mathbf{X}_i,
\end{aligned} \tag{7.19}$$

where $\mathbf{P}_i = \partial L / \partial \dot{\mathbf{X}}_i = M_{L,i} \dot{\mathbf{X}}_i$ is the momentum conjugate to $\mathbf{X}_i$, and in the second line we used the Euler–Lagrange equations, $\dot{\mathbf{P}}_i = \partial L / \partial \mathbf{X}_i + (\partial \mathbf{c} / \partial \mathbf{X}_i)^T \boldsymbol{\lambda}$. Hence

$$Q = \sum_i \mathbf{P}_i \cdot \delta \mathbf{X}_i \quad (7.20)$$

is conserved.

The conjugate momenta are

$$\mathbf{P}_i = \begin{pmatrix} m_i \dot{\mathbf{x}}_i \\ J_L(\boldsymbol{\theta}_i)^T I_i \boldsymbol{\omega}_i \end{pmatrix}, \quad (7.21)$$

so the momentum conjugate to $\boldsymbol{\theta}_i$ is J_L^T times the angular momentum $I_i \boldsymbol{\omega}_i$ — the momentum-side counterpart of $\boldsymbol{\omega} = J_L \dot{\boldsymbol{\theta}}$. For the infinitesimal rotation,

$$\begin{aligned} \mathbf{P}_i \cdot \delta \mathbf{X}_i &= m_i \dot{\mathbf{x}}_i \cdot (\boldsymbol{\epsilon} \times \mathbf{x}_i) + (J_L(\boldsymbol{\theta}_i)^T I_i \boldsymbol{\omega}_i) \cdot (J_L(\boldsymbol{\theta}_i)^{-1} \boldsymbol{\epsilon}), \\ &= \boldsymbol{\epsilon} \cdot (\mathbf{x}_i \times m_i \dot{\mathbf{x}}_i) + (I_i \boldsymbol{\omega}_i) \cdot (J_L(\boldsymbol{\theta}_i) J_L(\boldsymbol{\theta}_i)^{-1} \boldsymbol{\epsilon}), \\ &= [\mathbf{x}_i \times m_i \dot{\mathbf{x}}_i + I_i \boldsymbol{\omega}_i] \cdot \boldsymbol{\epsilon}. \end{aligned} \quad (7.22)$$

The two factors of J_L cancel, as they must (as $\mathbf{P}_i \cdot \delta \mathbf{X}_i$ is a scalar). Therefore

$$\frac{d}{dt} \sum_i [\mathbf{x}_i \times m_i \dot{\mathbf{x}}_i + I_i \boldsymbol{\omega}_i] \cdot \boldsymbol{\epsilon} = 0 \quad (7.23)$$

for every $\boldsymbol{\epsilon}$, i.e. the total angular momentum $\sum_i (\mathbf{x}_i \times m_i \dot{\mathbf{x}}_i + I_i \boldsymbol{\omega}_i)$ is conserved whenever the action is symmetric under an infinitesimal rotation. Since a finite rotation is a product of infinitesimal ones, angular momentum is conserved if the system is invariant under arbitrary rotations.

For example, a scalar like $\|\delta \mathbf{p}\|$ is invariant under rotations, since, if we apply a rotation R to the system

$$\begin{aligned} \|\delta \mathbf{p}\| &= [\delta \mathbf{p} \cdot \delta \mathbf{p}]^{1/2}, \\ &\rightarrow [(R \delta \mathbf{p}) \cdot (R \delta \mathbf{p})]^{1/2}, \\ &= [\delta \mathbf{p} \cdot (R^T R \delta \mathbf{p})]^{1/2}, \\ &= \|\delta \mathbf{p}\| \end{aligned} \quad (7.24)$$

since $R^T R = \mathbb{1}$. But also, a constraint $\mathbf{c} = \mathbf{p}_i - \mathbf{p}_j$ is fine

$$\boldsymbol{\lambda} \cdot (\mathbf{p}_i - \mathbf{p}_j) \rightarrow (R \boldsymbol{\lambda}) \cdot R(\mathbf{p}_i - \mathbf{p}_j) = \boldsymbol{\lambda} \cdot (\mathbf{p}_i - \mathbf{p}_j), \quad (7.25)$$

since the Lagrange multipliers are also transformed under rotations of the coordinate system.

A constraint like

$$\mathbf{c} = \mathbf{p}_i - \mathbf{p}_j - \mathbf{L} \quad (7.26)$$

that might try to enforce that the two points maintain a distance L apart, however, would not be rotationally invariant and so would not conserve angular momentum.

So, in order to preserve linear momentum, constraints must be invariant under translations and, in order to preserve angular momentum, the constraint term must be invariant under rotation.

Conserved quantities come from symmetries of the Lagrangian. These symmetries must be respected by the constraint term $\boldsymbol{\lambda} \cdot \mathbf{c}(\mathbf{X})$

1. Conservation of linear momentum comes from translational symmetry $\mathbf{x} \rightarrow \mathbf{x} + \mathbf{d}$ of the coordinates
2. Conservation of angular momentum comes from rotational symmetry $\mathbf{x} \rightarrow R\mathbf{x}$ of the coordinates

7.3 Energy Conservation

In the continuous system the constraint force does no work: its power is $\boldsymbol{\lambda} \cdot (\mathcal{D}\mathbf{c} \mathcal{V}) = \boldsymbol{\lambda} \cdot \dot{\mathbf{c}}$, and $\dot{\mathbf{c}} = 0$ on the constraint manifold. With conservative forces the constrained system therefore conserves energy. The discrete correction does not inherit this, it acts against the drift accumulated over the step, so does work.

The point to point constraint applies an impulse $\Delta = -\xi \mu \delta \mathbf{p}(\mathbf{X}^{t+1} + \Delta \mathbf{X}) / \Delta t$ on body i and $-\Delta$ on body j . Holding the inertia tensors at their initial values, the kinetic energy of the two bodies changes by

$$\Delta T = \Delta \cdot \delta \dot{\mathbf{p}} + \frac{1}{2} \Delta \cdot \mathcal{M}^{-1} \Delta, \quad (7.27)$$

with $\delta \dot{\mathbf{p}}$ the relative velocity of the attachment points from (7.4) and $\mathcal{M}^{-1}$ the inverse effective mass matrix (7.10). Since, also from (7.4), $\dot{\mathbf{c}} = \delta \dot{\mathbf{p}} \cdot \delta \mathbf{p}$

$$\Delta T = \|\Delta\| \dot{\mathbf{c}} + \frac{1}{2} \mu^{-1} \|\Delta\|^2. \quad (7.28)$$

where μ is the effective mass scalar (7.10).

The update for the point to point constraint sets $\|\Delta\| = -\xi \mu \|\delta \mathbf{p}(\mathbf{X}^{t+1})\| / \Delta t$. Using this, we can write the error at the end of the unconstrained step as the error present at its start plus the drift

$$\|\delta \mathbf{p}(\mathbf{X}^{t+1})\| = c_0 + \dot{\mathbf{c}} \Delta t \quad (7.29)$$

with $c_0 = c(\mathbf{X}^t)$. If the constraint was satisfied at the start of the step then $c_0 = 0$ and

$$\Delta T = -\xi \left(1 - \frac{\xi}{2}\right) \mu \dot{\mathbf{c}}^2, \quad (7.30)$$

which is never positive for $0 < \xi < 2$. For $\xi = 1$ and general c_0 ,

$$\Delta T = -\frac{1}{2} \mu \dot{\mathbf{c}}^2 + \frac{1}{2} \mu \left(\frac{c_0}{\Delta t}\right)^2. \quad (7.31)$$

For $c_0 = 0$, the update is equivalent to a perfectly inelastic impact along the joint: the update removes all of the kinetic energy in the relative motion of the attachment points along $\delta \hat{\mathbf{p}}$, measured with the effective mass. Subsequent iterations are not necessarily purely dissipative, however, because they generally operate with non-zero position error and therefore contain the positive c_0^2 term in (7.31).

The second term is position error already present at the start of the step converted into velocity, and it is never negative. It is the energy injection of Baumgarte stabilization in section §6.4, and the two terms of (7.31) are the two brackets of the split impulse decomposition (6.17): λ_{vel} produces the dissipative term and λ_{pos} the injection. Withholding λ_{pos} from the velocity update, as in (6.17), removes the second term and makes the correction purely dissipative. This is the energy argument for the split.

Equation (7.31) is one step. For a joint in motion the attachment points typically separate by $c(\mathbf{X}^{t+1}) \sim \frac{1}{2}a_c\Delta t^2$, where a_c is the relative acceleration along $\hat{\delta}\mathbf{p}$ that the joint force has to supply (e.g. for a link swinging about a pin, the centripetal term $\omega^2 L$ plus the component of gravity along the link), and the joint force that supplies it is $\lambda = \mu a_c$. The update removes this position error by introducing a velocity correction of $c/\Delta t$, so per step

$$\Delta T \sim -\frac{1}{2}\mu \left(\frac{c}{\Delta t}\right)^2 \sim -\frac{\lambda^2}{\mu} \Delta t^2, \quad \dot{E} \sim -\frac{\lambda^2}{\mu} \Delta t, \quad (7.32)$$

There are two things to note:

- The loss associated with correcting the velocity drift is first order in Δt , so it is reduced by sub-stepping but not by additional constraint iterations at fixed Δt , another benefit of sub-stepping to add to the list in §6.3,
- The loss scales with the square of the joint force, so, all else being equal, constraints carrying the largest loads damp energy the most.

7.4 Inequality Constraints

A standard extension to Gauss-Seidel is Projected Gauss-Seidel where, as well as solving equality constraints $\mathbf{c}(\mathbf{X}) = 0$, we can extend the system to also solve inequality constraints $\mathbf{c}(\mathbf{X}) \geq 0$. This extension comes from treating the system as a Linear Complementarity Problem (LCP). Starting from the linearized constraint

$$\mathbf{c}(\mathbf{X}^{t+1} + \Delta\mathbf{X}) \approx \mathbf{c}(\mathbf{X}^{t+1}) + \mathcal{D}\mathbf{c}\Delta\mathbf{X}, \quad (7.33)$$

the inequality $\mathbf{c}(\mathbf{X}^{t+1} + \Delta\mathbf{X}) \geq 0$ leads to the convex quadratic program

$$\min_{\Delta\mathbf{X}} \frac{1}{2} \Delta\mathbf{X} \cdot M \Delta\mathbf{X} \quad \text{s.t.} \quad \mathbf{c}(\mathbf{X}^{t+1}) + \mathcal{D}\mathbf{c}\Delta\mathbf{X} \geq 0. \quad (7.34)$$

Eliminating $\Delta\mathbf{X}$ gives the LCP:

$$0 \leq \boldsymbol{\lambda} \perp \mathbf{c}(\mathbf{X}^{t+1}) + \left(\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T\right) \boldsymbol{\lambda} \geq 0, \quad (7.35)$$

which is solved using a Projected Gauss-Seidel update of

$$\begin{aligned} \lambda_i^* &= \lambda_i^k - \left(\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T\right)_{ii}^{-1} \mathbf{c}(\mathbf{X}^{t+1} + \Delta\mathbf{X}^k)_i, \\ \lambda_i^{k+1} &= \begin{cases} \lambda_i^* & \text{if } \lambda_i^* \geq 0 \\ 0 & \text{otherwise} \end{cases} \end{aligned} \quad (7.36)$$

See, for example, [29] where they derive the projected Gauss-Seidel update as a solution to the LCP. Alternatively, [10] gives an impulse centric approach, where it is obvious that clamping the impulse to be one way in the constraint direction gives the correct solution.

7.5 Contact Constraint

A contact constraint is an example of an inequality constraint. The constraint is that for two points $\mathbf{p}_i$ and $\mathbf{p}_j$ on bodies i and j the distance between the points in the direction of a contact normal direction $\hat{\mathbf{n}}$ is positive.

$$c(\mathbf{X}) = (\mathbf{p}(\mathbf{X}_i) - \mathbf{p}(\mathbf{X}_j)) \cdot \hat{\mathbf{n}} \geq 0. \quad (7.37)$$

where, as in 7.1

$$\begin{aligned} \mathbf{p}(\mathbf{X}) &= \mathbf{x} + R(\boldsymbol{\theta})\mathbf{r}_0, \\ &= \mathbf{x} + \mathbf{r}(\boldsymbol{\theta}) \end{aligned} \quad (7.38)$$

for an offset $\mathbf{r}$ from a centre of mass $\mathbf{x}$.

The constraint derivative is

$$\mathcal{D}\mathbf{c} = (\hat{\mathbf{n}}^T \quad (\mathbf{r}_i \times \hat{\mathbf{n}})^T \quad -\hat{\mathbf{n}}^T \quad -(\mathbf{r}_j \times \hat{\mathbf{n}})^T) \quad (7.39)$$

leading to an inverse effective mass of

$$\begin{aligned} \mu^{-1} &= \mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T, \\ &= \hat{\mathbf{n}} \cdot \mathcal{M}^{-1} \hat{\mathbf{n}} \end{aligned} \quad (7.40)$$

where $\mathcal{M}^{-1}$ is the inverse of the effective mass matrix (7.10). I.e., we are projecting the inverse of the effective mass matrix in the direction $\hat{\mathbf{n}}$ to measure the resistance to movement in that direction.

With this, the update becomes

Contact normal constraint update

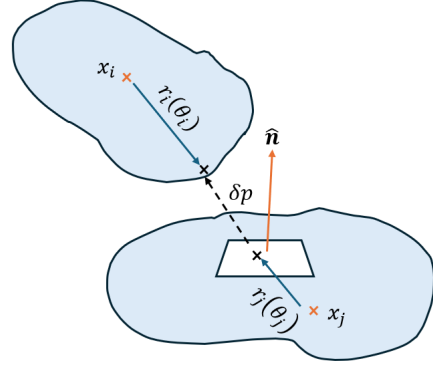
Lagrange multiplier update with clamping:

$$\begin{aligned} \lambda_i^* &= \lambda_i^k - \xi \mu \delta \mathbf{p}(\mathbf{X}^{t+1} + \Delta \mathbf{X}^k) \cdot \hat{\mathbf{n}}, \\ \lambda_i^{k+1} &= \begin{cases} \lambda_i^* & \text{if } \lambda_i^* \geq 0 \\ 0 & \text{otherwise} \end{cases} \end{aligned} \quad (7.41)$$

Delta:

$$\Delta = (\lambda_i^{k+1} - \lambda_i^k) \hat{\mathbf{n}}. \quad (7.42)$$

Configuration update:



$$\begin{aligned}
\Delta \mathbf{x}'_i &= \Delta \mathbf{x}_i + \frac{1}{m_i} \Delta, \\
\Delta \boldsymbol{\omega}'_i &= \Delta \boldsymbol{\omega}_i + I_i^{-1} \mathbf{r}_i \times \Delta, \\
\Delta \mathbf{x}'_j &= \Delta \mathbf{x}_j - \frac{1}{m_j} \Delta, \\
\Delta \boldsymbol{\omega}'_j &= \Delta \boldsymbol{\omega}_j - I_j^{-1} \mathbf{r}_j \times \Delta.
\end{aligned} \tag{7.43}$$

7.6 Contact Velocity Constraint

For collisions with restitution e the final relative velocity along the contact normal should be equal to $-e$ times the incoming relative velocity.

$$\dot{\delta \mathbf{p}}(\dot{\mathbf{X}}^{t+1}) \cdot \hat{\mathbf{n}} = -e \dot{\delta \mathbf{p}}(\dot{\mathbf{X}}^t) \cdot \hat{\mathbf{n}} \tag{7.44}$$

with

$$\dot{\mathbf{p}} = \dot{\mathbf{x}} + \boldsymbol{\omega} \times \mathbf{r}. \tag{7.45}$$

We can make this into a velocity constraint

$$c(\dot{\mathbf{X}}) = \dot{\delta \mathbf{p}}(\dot{\mathbf{X}}^{t+1}) \cdot \hat{\mathbf{n}} + e \dot{\delta \mathbf{p}}(\dot{\mathbf{X}}^t) \cdot \hat{\mathbf{n}}. \tag{7.46}$$

This is an example of a non-holonomic constraint, covered in section §3.3. To see that this constraint is the correct one, consider linear velocities along $\mathbf{n}$. The update would be

$$\begin{aligned}
\dot{\mathbf{x}}_i &\rightarrow \dot{\mathbf{x}}_i - \frac{1}{m_i} \mu (1+e) (\dot{\mathbf{x}}_i - \dot{\mathbf{x}}_j), \\
\dot{\mathbf{x}}_j &\rightarrow \dot{\mathbf{x}}_j + \frac{1}{m_j} \mu (1+e) (\dot{\mathbf{x}}_i - \dot{\mathbf{x}}_j), \\
\Rightarrow (\dot{\mathbf{x}}_i - \dot{\mathbf{x}}_j) &\rightarrow (\dot{\mathbf{x}}_i - \dot{\mathbf{x}}_j) - \mu^{-1} \mu (1+e) (\dot{\mathbf{x}}_i - \dot{\mathbf{x}}_j), \\
&= -e (\dot{\mathbf{x}}_i - \dot{\mathbf{x}}_j).
\end{aligned} \tag{7.47}$$

The constraint derivative is

$$\dot{\mathcal{D}}\mathbf{c} = (\hat{\mathbf{n}}^T \quad (\mathbf{r}_i \times \hat{\mathbf{n}})^T \quad -\hat{\mathbf{n}}^T \quad -(\mathbf{r}_j \times \hat{\mathbf{n}})^T), \tag{7.48}$$

which is identical to $\mathcal{D}\mathbf{c}$ we derived from the contact normal constraint. So, the effective mass is also the same

$$\mu^{-1} = \hat{\mathbf{n}} \cdot \mathcal{M}^{-1} \hat{\mathbf{n}}. \tag{7.49}$$

With this, the update is

Contact normal velocity constraint with restitution
Lagrange multiplier update with clamping:

$$\begin{aligned}
\lambda_i^* &= \lambda_i^k - \xi \mu \left(\dot{\delta \mathbf{p}}(\dot{\mathbf{X}}^{t+1} + \Delta \dot{\mathbf{X}}^k / \Delta t) \cdot \hat{\mathbf{n}} + e \dot{\delta \mathbf{p}}(\dot{\mathbf{X}}^t) \cdot \hat{\mathbf{n}} \right), \\
\lambda_i^{k+1} &= \begin{cases} \lambda_i^* & \text{if } \lambda_i^* \geq 0 \\ 0 & \text{otherwise} \end{cases}
\end{aligned} \tag{7.50}$$

Delta:

$$\Delta = (\lambda_i^{k+1} - \lambda_i^k) \hat{\mathbf{n}}. \quad (7.51)$$

Configuration update:

$$\begin{aligned} \Delta \mathbf{x}'_i &= \Delta \mathbf{x}_i + \frac{1}{m_i} \Delta, \\ \Delta \boldsymbol{\omega}'_i &= \Delta \boldsymbol{\omega}_i + I_i^{-1} \mathbf{r}_i \times \Delta, \\ \Delta \mathbf{x}'_j &= \Delta \mathbf{x}_j - \frac{1}{m_j} \Delta, \\ \Delta \boldsymbol{\omega}'_j &= \Delta \boldsymbol{\omega}_j - I_j^{-1} \mathbf{r}_j \times \Delta. \end{aligned} \quad (7.52)$$

This is almost the same as (7.43). The difference is the displacement term

$$\begin{aligned} \text{Position :} \quad \Delta &= -\xi \mu \delta \mathbf{p}(\mathbf{X}^{t+1} + \Delta \mathbf{X}^k) \cdot \hat{\mathbf{n}} \hat{\mathbf{n}}, \\ \text{Velocity :} \quad \Delta &= -\xi \mu (1 + e) \dot{\delta \mathbf{p}}(\mathbf{X}^{t+1} + \Delta \mathbf{X}^k / \Delta t) \cdot \hat{\mathbf{n}} \hat{\mathbf{n}} \Delta t \end{aligned} \quad (7.53)$$

and

$$\delta \mathbf{p}(\mathbf{X}^{t+1} + \Delta \mathbf{X}^k) = \delta \mathbf{p}(\mathbf{X}^t) + \dot{\delta \mathbf{p}}(\mathbf{X}^{t+1} + \Delta \mathbf{X}^k / \Delta t) \Delta t. \quad (7.54)$$

So, to first order, the velocity constraint can be formulated as a position constraint with just a different target relative position.

7.7 Contact Friction Constraint

Friction acts to oppose relative tangential velocity at the contact point. The dry friction model divides the force into two modes: static friction for when two surfaces are not moving relative to each other, and dynamic friction when they are in relative motion in the contact plane. The model is that if the magnitude of the force is less than the static friction limit $\nu_s \|\mathbf{N}\|$ for some contact normal force $\mathbf{N}$ and static friction coefficient ν_s then the force prevents the points moving relative to each other in the contact plane. If, however, the magnitude of the force required to keep the points from moving relative to each other in the contact plane exceeds that limit then the bodies will start moving and a dynamic friction force of $\nu_d \|\mathbf{N}\| \hat{\mathbf{s}}$, where $\hat{\mathbf{s}}$ is the direction of the relative surface velocity and ν_d is a dynamic friction coefficient. The model is valid for $\nu_d \leq \nu_s$.

In [31] this scheme is derived as resulting from a principle of maximal dissipation.

Let the relative velocity at the contact points be

$$\dot{\delta \mathbf{p}} = \dot{\mathbf{p}}_i - \dot{\mathbf{p}}_j = (\dot{\mathbf{x}}_i + \boldsymbol{\omega}_i \times \mathbf{r}_i) - (\dot{\mathbf{x}}_j + \boldsymbol{\omega}_j \times \mathbf{r}_j). \quad (7.55)$$

We decompose this into normal and tangential components:

$$\dot{\delta \mathbf{p}}_{\perp} = \dot{\delta \mathbf{p}} - (\dot{\delta \mathbf{p}} \cdot \hat{\mathbf{n}}) \hat{\mathbf{n}}. \quad (7.56)$$

Where

$$\hat{\mathbf{t}} = \frac{\dot{\delta \mathbf{p}}_{\perp}}{\|\dot{\delta \mathbf{p}}_{\perp}\|}. \quad (7.57)$$

We impose the constraint

$$c(\dot{\mathbf{X}}) = \delta \dot{\mathbf{p}} \cdot \hat{\mathbf{t}}. \quad (7.58)$$

The derivative with respect to velocities is

$$\dot{D}c = (\hat{\mathbf{t}}^T \quad (\mathbf{r}_i \times \hat{\mathbf{t}})^T \quad -\hat{\mathbf{t}}^T \quad -(\mathbf{r}_j \times \hat{\mathbf{t}})^T). \quad (7.59)$$

Giving an effective mass of

$$\mu_t = (\hat{\mathbf{t}} \cdot \mathcal{M}^{-1} \hat{\mathbf{t}})^{-1}, \quad (7.60)$$

where $\mathcal{M}^{-1}$ is the inverse effective mass matrix used in the normal contact constraint.

The constraint update is

Contact friction constraint update

$$\begin{aligned} \boldsymbol{\lambda}^* &= \boldsymbol{\lambda}^k - \xi \mu_t \delta \dot{\mathbf{p}}(\mathbf{X}^{t+1} + \Delta \mathbf{X}^k / \Delta t) \cdot \hat{\mathbf{t}} \hat{\mathbf{t}}, \\ \boldsymbol{\lambda}^{k+1} &= \begin{cases} \boldsymbol{\lambda}^* & \text{if } \|\boldsymbol{\lambda}^*\| \leq \nu_s \|\mathbf{N}\| \\ \nu_d \|\mathbf{N}\| \hat{\boldsymbol{\lambda}}^* & \text{otherwise} \end{cases} \end{aligned} \quad (7.61)$$

with the incremental position and orientation updates as in the update for the contact normal constraint (7.43).

Note, this is a slight simplification. If there is relative tangential velocity it is good to construct the friction axis in that direction, but there can be other displacements from other constraints that would move the contact points in a direction off of this axis, but also perpendicular to the contact normal. There are a couple of different approaches to take:

1. Apply the friction constraint along two orthogonal friction axes.

So construct the orthonormal frame $\{\mathbf{n}, \hat{\mathbf{t}}_1, \hat{\mathbf{t}}_2 = \hat{\mathbf{t}} \times \mathbf{n}\}$. Taking this approach requires a slight modification to the algorithm. We compute $\boldsymbol{\lambda}_1^*$ and $\boldsymbol{\lambda}_2^*$ for each of the friction axes, then compare the magnitude of the sum to the static friction limit $\|\boldsymbol{\lambda}_1^* + \boldsymbol{\lambda}_2^*\| \leq \nu_s \|\mathbf{N}\|$, The reason for doing this is so that the magnitude of the friction is clamped to a circular limit in the friction plane, rather than a box if we were to clamp the individual directions.

2. Recompute the friction axis during the solve

This means storing the inverse effective mass matrix $\mathcal{M}^{-1}$ and re-projecting it onto a new $\hat{\mathbf{t}}$ each iteration.

Both of these get around the thing we brushed over that if the relative tangential velocity is zero when formulating the constraint we cannot construct the friction direction $\hat{\mathbf{v}}t$ or compute the effective mass. In the case of option 1, if there is no initial tangential velocity we just use two arbitrary axes perpendicular to the normal direction.

One reason to choose option 1 would be if you were warm starting the friction solve. In this case, it is good to have consistent axes from frame to frame. Or you could just re-project the previous result onto the new axes.

Note that this friction model can also be included in a primal formulation as a Rayleigh dissipation function of the tangential relative velocity $\delta\dot{\mathbf{p}}_{\perp}$,

$$\mathcal{F} = \begin{cases} \frac{1}{2}k_t|\delta\dot{\mathbf{p}}_{\perp}|^2, & k_t|\delta\dot{\mathbf{p}}_{\perp}| < \nu\|\mathbf{N}\|, \\ \nu\|\mathbf{N}\||\delta\dot{\mathbf{p}}_{\perp}| - \gamma, & \text{otherwise,} \end{cases} \quad \gamma = \frac{\nu^2\|\mathbf{N}\|^2}{2k_t}, \quad (7.62)$$

where k_t is the stiffness of the sticky-regime and γ makes $\mathcal{F}_f$ continuous in the stick/slip transition. The normal force $\mathbf{N}$ is held fixed during the friction update. The friction force is

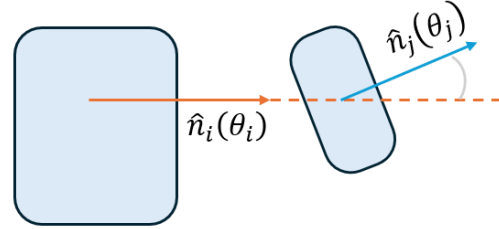
$$\mathbf{f} = -\frac{\partial\mathcal{F}}{\partial\delta\dot{\mathbf{p}}_{\perp}} = -\min\left(k_t, \frac{\nu\|\mathbf{N}\|}{|\delta\dot{\mathbf{p}}_{\perp}|}\right)\delta\dot{\mathbf{p}}_{\perp}, \quad (7.63)$$

which is the equivalent of (7.61) in primal form. See [18].

Primal / force-based approaches are developed in §8.1.

7.8 Angular Axis Constraint

For this constraint example we will look at a purely angular constraint. The goal is to limit the rotation of one body to be around an axis defined in the space of another body. Imagine a wheel constrained to rotate around an axis in the space of a car body, for instance.



There are a few different ways to define this constraint, but we choose

$$c(\boldsymbol{\theta}) = \sin^{-1} \|\hat{\mathbf{n}}_i \times \hat{\mathbf{n}}_j\|, \quad (7.64)$$

where $\hat{\mathbf{n}}$ are the axes on the respective bodies that we want to keep aligned.

The reason for choosing the sine of the angle being zero as the constraint, rather than looking at the cosine with $c(\boldsymbol{\theta}) = \hat{\mathbf{n}}_i \cdot \hat{\mathbf{n}}_j - 1$ is that around the point where the angle between the two is small sine is changing rapidly, whereas cosine is fairly flat, so the derivative around this point is more reliable with the sine formulation.

Taking the derivative with respect to $\boldsymbol{\theta}_i$ and $\boldsymbol{\theta}_j$ gives

$$\mathcal{D}\mathbf{c} = (-\hat{\mathbf{w}}^T \quad \hat{\mathbf{w}}^T) \quad (7.65)$$

where

$$\hat{\mathbf{w}} = \frac{\hat{\mathbf{n}}_i \times \hat{\mathbf{n}}_j}{\|\hat{\mathbf{n}}_i \times \hat{\mathbf{n}}_j\|} \quad (7.66)$$

is the unit rotation axis required to align the axes.

Using this, the effective inertia is

$$\mu = \left[\hat{\mathbf{w}} \cdot (I_i^{-1} + I_j^{-1}) \hat{\mathbf{w}} \right]^{-1}, \quad (7.67)$$

i.e. the projection of the effective inertia in the direction $\hat{\mathbf{w}}$.

With this, the update is

Axis alignment constraint

Lagrange multiplier update:

$$\lambda^{k+1} = \lambda_i^k - \mu \left(\sin^{-1} \|\hat{\mathbf{n}}_i(\boldsymbol{\theta}_i^{t+1} + \Delta\boldsymbol{\theta}_i) \times \hat{\mathbf{n}}_j(\boldsymbol{\theta}_j^{t+1} + \Delta\boldsymbol{\theta}_j)\| \right) \quad (7.68)$$

Delta:

$$\Delta = -(\lambda^{k+1} - \lambda^k)\hat{\mathbf{w}}. \quad (7.69)$$

Configuration update:

$$\begin{aligned} \Delta\boldsymbol{\omega}'_i &= \Delta\boldsymbol{\omega}_i + I_i^{-1}\Delta, \\ \Delta\boldsymbol{\omega}'_j &= \Delta\boldsymbol{\omega}_j - I_j^{-1}\Delta. \end{aligned} \quad (7.70)$$

As with previous constraints, if the initial error is zero then it is not possible to form $\hat{\mathbf{w}}$, so the projected error can be used instead. If both are zero, then an arbitrary axis can be used. In this case, $\hat{\mathbf{w}}$ must be computed per iteration using the projected error.

8 Compliant Constraints

8.1 Generalized Implicit Non-Linear Forces

So far, we have assumed that we can, by some other method / integration scheme, find the solutions to the equations of motion when there are no constraints. But, if we have non-linear forces and torques and we want to solve for them implicitly (i.e. taking the force term at $F(\mathbf{X}^{t+1})$ when solving for $\mathbf{X}^{t+1}$) it is not necessarily easy. But in general updating with implicit forces and torques is a good thing for stability, so we want to find a solution. We proceed using a similar first order expansion that we made when solving the constrained equation, except that this time we assume that we have a solution $\mathbf{X}^{t+1}, \dot{\mathbf{X}}^{t+1}$ that satisfies the equations of motion in the absence of forces and torques and then add a small displacement $\Delta\mathbf{X}$ to the solution and solve the full equations of motion up to first order in $\Delta\mathbf{X}$.

The equations of motion are

$$\frac{d}{dt} (M(\mathbf{X})\mathcal{V}) = \mathbf{F}(\mathbf{X}), \quad (8.1)$$

Taking $\mathbf{X}^{t+1} \rightarrow \mathbf{X}^{t+1} + \Delta\mathbf{X}$ we get

$$M(\mathbf{X}^{t+1} + \Delta\mathbf{X})(\dot{\mathbf{X}}^{t+1}\Delta t + \Delta\dot{\mathbf{X}}) - M(\mathbf{X}^{t+1})\dot{\mathbf{X}}^{t+1}\Delta t = \mathbf{F}(\mathbf{X}^{t+1})\Delta t^2 + (\mathcal{D}\mathbf{F})^T \Delta\mathbf{X}\Delta t^2, \quad (8.2)$$

which gives a linear equation for $\Delta\mathbf{X}$

Linearized system for an increment $\Delta\mathbf{X}$ required to add to the solution $\mathbf{X}^{t+1}$ of the free equations of motion that solves to first order the equations of motion with generalized forces:

$$(M + K)\Delta\mathbf{X} = \mathbf{F}(\mathbf{X}^{t+1})\Delta t^2 \quad (8.3)$$

with

$$K = -(\mathcal{D}\mathbf{F})^T \Delta t^2 \quad (8.4)$$

Since K is not necessarily block diagonal, we need to use an iterative approach to solve this system for $\Delta\mathbf{X}$. We again use the Gauss-Seidel algorithm. Following a similar method to section §4.3 results in an iterative update

Gauss-Seidel iteration for solving the force equation (8.3)

$$\Delta\mathbf{X}_i^{k+1} = \Delta\mathbf{X}_i^k + \xi [(M + K)_{ii}]^{-1} \left[\mathbf{F}_i(\mathbf{X}^{t+1} + \Delta\mathbf{X}^k)\Delta t^2 - M_i\Delta\mathbf{X}_i^k \right] \quad (8.5)$$

This is of the same form as the iterative update for the constraint forces, and so can be added as an additional term in the constraint solve update in the same iteration loop. The result is that, as we update the displacement $\Delta\mathbf{X}$ each iteration, we correct $\mathbf{X}$ with a change that corresponds to the force to be applied.

8.2 Connection to Vertex Block Descent

Notice in this approach that, instead of solving iteratively per constraint, we are solving per body. $\mathbf{F}_i$ is the sum over all of the forces on body i . This is the same methodology as applied in Vertex Block Descent (VBD) methods [11].

In the vertex block descent paper they define a local variational energy equivalent to

$$G(\mathbf{X}) = \frac{m}{2} \|\Delta \mathbf{x}\|^2 + E(\mathbf{X}) \quad (8.6)$$

which they minimize, leading to a linear system of

$$H \Delta \mathbf{x} = \mathbf{f}, \quad (8.7)$$

with

$$\begin{aligned} \mathbf{f} &= -m \Delta \mathbf{x} - \frac{\partial E}{\partial \mathbf{x}}, \\ H &= m \mathbb{1} + \frac{\partial^2 E}{\partial \mathbf{x}^2} \end{aligned} \quad (8.8)$$

We saw in 5.1 that this kind of minimization is equivalent to solving the constrained equations of motion as a first order perturbation on the unconstrained equations. Now we are finding a first order perturbation that solves the full equations of motion as a perturbation on the free equations of motion. So, our generalized force method is the extension of VBD to rigid bodies, identifying

$$\begin{aligned} \frac{\partial E}{\partial \mathbf{x}} &\sim \mathbf{F}, \\ \frac{\partial^2 E}{\partial \mathbf{x}^2} &\sim \mathcal{D}\mathbf{F}, \\ M(\mathbf{X}) &\sim m \end{aligned} \quad (8.9)$$

The method presented for solving for general implicit non-linear forces is a rigid body version of the method of Vertex Block Descent.

8.3 Connection to Lagrange Multiplier Method and PBD

We can reformulate the system to be solved by introducing an intermediate variable $\boldsymbol{\lambda}$ as

$$\Delta \mathbf{X} = M^{-1}(M + K)^T \boldsymbol{\lambda}. \quad (8.10)$$

Using this, the equation (8.3) becomes

$$((M + K)M^{-1}(M + K)^T) \boldsymbol{\lambda} = \mathbf{F}(\mathbf{X}^{t+1})\Delta t^2. \quad (8.11)$$

From this we get a Gauss-Seidel iterative update of

$$\boldsymbol{\lambda}_i^{k+1} = \boldsymbol{\lambda}_i^k + \xi \left((M + K)M^{-1}(M + K)^T \right)_{ii}^{-1} \left(\mathbf{F}(\mathbf{X}^{t+1})\Delta t^2 - (M + K)M^{-1}(M + K)^T \boldsymbol{\lambda}^k \right)_i \quad (8.12)$$

with

$$\Delta \mathbf{X}_i^{k+1} = \Delta \mathbf{X}_i^k + M_{ii}^{-1}(M + K)_i^T (\boldsymbol{\lambda}_i^{k+1} - \boldsymbol{\lambda}_i^k). \quad (8.13)$$

Rearranging the λ^{k+1} update we get

$$\lambda_i^{k+1} = \lambda_i^k + \xi \left((M + K)M^{-1}(M + K)^T \right)_{ii}^{-1} \left(\mathbf{F}(\mathbf{X}^{t+1} + \Delta \mathbf{X}^k) \Delta t^2 - M \Delta \mathbf{X}^k \right)_i \quad (8.14)$$

Compare this to (6.1). We see that they correspond if we interpret

$$\begin{aligned} \mathbf{c}(\mathbf{X} + \Delta \mathbf{X}) &= M \Delta \mathbf{X} - \mathbf{F}(\mathbf{X}^{t+1} + \Delta \mathbf{X}) \Delta t^2, \\ \mathcal{D}\mathbf{c} &= M - \mathcal{D}\mathbf{F} \Delta t^2 \\ &= M + K. \end{aligned} \quad (8.15)$$

We can transform the per body solve from the generalized implicit force system (8.3) into a per constraint solve by identifying the first order equations for the forces as constraints in a Lagrange multiplier system

$$\mathbf{c}(\mathbf{X}) = M \Delta \mathbf{X} - \mathbf{F}(\mathbf{X}^{t+1} + \Delta \mathbf{X}) \Delta t^2 = 0. \quad (8.16)$$

So, enforcing a constraint that the integrated force evaluated at the current configuration must produce a displacement of $M \Delta \mathbf{X}$.

With this, we are transforming the problem back to iterating over “constraints” with multipliers λ rather than iterating over bodies, where each constraint enforces an individual force equation rather than summing over all of the forces. While this does affect the algorithm and the convergence of the solution, we are in the end solving the same system and so both should converge to the same answer.

8.4 Compliant Constraint Forces and XPBD

The method of Lagrange multipliers is good for modelling stiff constraints where we want the constraint equation $\mathbf{c}(\mathbf{X}) = 0$ to be satisfied exactly. Often though we want to model systems with some compliance in the constraints, as this can help with stability and also be used to model things with spring-like behaviour. For these types of systems, we add a potential term rather than using a constraint.

$$V = \frac{1}{2} \mathbf{c}(\mathbf{X}) \cdot A \mathbf{c}(\mathbf{X}), \quad (8.17)$$

where $\mathbf{c}(\mathbf{X})$ are the constraint functions and A is a diagonal matrix of stiffness parameters. In addition we want to model damping. This is not naturally handled by the standard Lagrangian method, as that assumes conservative forces. The usual modification is to add a Rayleigh dissipation function [15]

$$\mathcal{F} = \frac{1}{2} \dot{\mathbf{X}} \cdot D \dot{\mathbf{X}} \quad (8.18)$$

and then the equations of motion are modified to

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{\mathbf{X}}} \right) - \frac{\partial L}{\partial \mathbf{X}} + \frac{\partial \mathcal{F}}{\partial \dot{\mathbf{X}}} = 0. \quad (8.19)$$

In our case we want to apply damping to the constraint function, so we want a dissipation term

$$\frac{1}{2} \dot{\mathbf{c}}(\mathbf{X}) \cdot B \dot{\mathbf{c}}(\mathbf{X}) \quad (8.20)$$

for some diagonal matrix of damping constants B . Using $\dot{\mathbf{c}}(\mathbf{X}) = \mathcal{D}\mathbf{c}\dot{\mathbf{X}}$ we find

$$D = (\mathcal{D}\mathbf{c})^T B \mathcal{D}\mathbf{c}. \quad (8.21)$$

With these additional terms the equations of motion become

$$\frac{d}{dt} (M(\mathbf{X})\mathcal{V}) = -(\mathcal{D}\mathbf{c})^T A\mathbf{c}(\mathbf{X}) - (\mathcal{D}\mathbf{c})^T B \mathcal{D}\mathbf{c}\dot{\mathbf{X}}. \quad (8.22)$$

so the force is

$$\mathbf{F} = -(\mathcal{D}\mathbf{c})^T \left(A\mathbf{c}(\mathbf{X}) + B \mathcal{D}\mathbf{c}\dot{\mathbf{X}} \right). \quad (8.23)$$

The equation (8.5) tells us how to iteratively update the system given this force. First we need to find K we do this by looking at $\mathbf{F}(\mathbf{X}^{t+1} + \Delta\mathbf{X})$

$$\begin{aligned} \mathbf{F}(\mathbf{X}^{t+1} + \Delta\mathbf{X}) &= -(\mathcal{D}\mathbf{c})^T \Big|_{\mathbf{X}^{t+1} + \Delta\mathbf{X}} \left(A\mathbf{c}(\mathbf{X}^{t+1} + \Delta\mathbf{X}) + B \mathcal{D}\mathbf{c}|_{\mathbf{X}^{t+1} + \Delta\mathbf{X}} (\dot{\mathbf{X}} + \Delta\mathbf{X}/\Delta t) \right), \\ &= F(\mathbf{X}^{t+1}) - (\mathcal{D}\mathbf{c})^T (A + B/\Delta t) \mathcal{D}\mathbf{c}\Delta\mathbf{X} \\ &\quad - (\mathcal{D}^2\mathbf{c})^T \left(A\mathbf{c}(\mathbf{X}^{t+1}) + B \mathcal{D}\mathbf{c}\dot{\mathbf{X}} \right) - (\mathcal{D}\mathbf{c})^T B (\mathcal{D}^2\mathbf{c}\Delta\mathbf{X}) \dot{\mathbf{X}}, \end{aligned} \quad (8.24)$$

where $\mathcal{D}^2\mathbf{c}$ is the constraint Hessian using the left trivialized derivative (3.37).

Therefore, comparing to (8.3)

$$\begin{aligned} K &= (\mathcal{D}\mathbf{c})^T (A\Delta t^2 + B\Delta t) \mathcal{D}\mathbf{c} + \\ &\quad (\mathcal{D}^2\mathbf{c})^T \left(A\Delta t^2 \mathbf{c}(\mathbf{X}^{t+1}) + B\Delta t \mathcal{D}\mathbf{c}\dot{\mathbf{X}}\Delta t \right) - (\mathcal{D}\mathbf{c})^T B\Delta t (\mathcal{D}^2\mathbf{c}\Delta\mathbf{X}) \dot{\mathbf{X}}\Delta t. \end{aligned} \quad (8.25)$$

The constraint Hessian term is neglected in XPBD, equivalent to Gauss–Newton approximation, so we will also assume it to be small and ignore it for the purposes of comparison to XPBD. This leaves

$$K = (\mathcal{D}\mathbf{c})^T (A\Delta t^2 + B\Delta t) \mathcal{D}\mathbf{c}. \quad (8.26)$$

Using this

$$\left(M + (\mathcal{D}\mathbf{c})^T (A\Delta t^2 + B\Delta t) \mathcal{D}\mathbf{c} \right) \Delta\mathbf{X} = -(\mathcal{D}\mathbf{c})^T \left(A\mathbf{c}(\mathbf{X}) + B \mathcal{D}\mathbf{c}\dot{\mathbf{X}} \right) \Delta t^2. \quad (8.27)$$

Again, we add a variable $\boldsymbol{\lambda}$ with $\Delta\mathbf{X} = M^{-1} (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda} \Delta t^2$, so that $\boldsymbol{\lambda}$ has the units of a force as in section §4.3. This gives

$$\left(M + (\mathcal{D}\mathbf{c})^T (A\Delta t^2 + B\Delta t) \mathcal{D}\mathbf{c} \right) M^{-1} (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda} \Delta t^2 = -(\mathcal{D}\mathbf{c})^T \left(A\mathbf{c}(\mathbf{X}) + B \mathcal{D}\mathbf{c}\dot{\mathbf{X}} \right) \Delta t^2. \quad (8.28)$$

Simplifying gives

$$\left(\mathbb{1} + (A\Delta t^2 + B\Delta t) \mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T \right) \boldsymbol{\lambda} \Delta t^2 = -A\Delta t^2 \mathbf{c}(\mathbf{X}) - B\Delta t \mathcal{D}\mathbf{c} (\dot{\mathbf{X}}\Delta t). \quad (8.29)$$

The Gauss–Seidel iterative update for $\boldsymbol{\lambda}$ is then

$$\begin{aligned} \boldsymbol{\lambda}_i^{k+1} \Delta t^2 &= \boldsymbol{\lambda}_i^k \Delta t^2 - \xi \left(\mathbb{1} + (A\Delta t^2 + B\Delta t) \mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T \right)_{ii}^{-1} \left(A\Delta t^2 \mathbf{c}(\mathbf{X}^{t+1} + \Delta\mathbf{X}^k) \right. \\ &\quad \left. + B\Delta t \mathcal{D}\mathbf{c} (\dot{\mathbf{X}}^{t+1} \Delta t + \Delta\mathbf{X}^k) + \boldsymbol{\lambda}_i^k \Delta t^2 \right)_i. \end{aligned} \quad (8.30)$$

To make the connection with XPBD we take

$$A_{ii}\Delta t^2 = \frac{1}{\tilde{\alpha}_i}, \quad (8.31)$$

$$B_{ii}\Delta t^2 = \tilde{\beta}_i, \quad (8.32)$$

$$\gamma_i = \frac{\tilde{\alpha}_i\tilde{\beta}_i}{\Delta t}. \quad (8.33)$$

Using these, the update becomes

$$\boldsymbol{\lambda}_i^{k+1}\Delta t^2 = \boldsymbol{\lambda}_i^k\Delta t^2 - \xi \left(\tilde{\alpha}_i + (1 + \gamma_i) \mathcal{D}\mathbf{c}M^{-1}(\mathcal{D}\mathbf{c})^T \right)_{ii}^{-1} \left(\mathbf{c}(\mathbf{X}^{t+1} + \Delta\mathbf{X}^k) \right. \quad (8.34)$$

$$\left. + \gamma_i \mathcal{D}\mathbf{c}(\dot{\mathbf{X}}^{t+1}\Delta t + \Delta\mathbf{X}^k) + \tilde{\alpha}_i \boldsymbol{\lambda}_i^k \Delta t^2 \right)_i \quad (8.35)$$

which is exactly the update presented in the XPBD paper [20], once we identify the XPBD multiplier with $\boldsymbol{\lambda}\Delta t^2$ (for a distance constraint XPBD's λ has the units of mass $\times$ length, whereas our $\boldsymbol{\lambda}$ is a force).

Summary:

In XPBD, instead of enforcing constraints through a Lagrange multiplier approach, constraint penalty terms are added to the potential energy

$$V = \frac{1}{2}\mathbf{c}(\mathbf{X}) \cdot A\mathbf{c}(\mathbf{X}), \quad \mathcal{F} = \frac{1}{2}\dot{\mathbf{c}}(\mathbf{X}) \cdot B\dot{\mathbf{c}}(\mathbf{X}), \quad (8.36)$$

together with a Rayleigh dissipation function $\mathcal{F}$ for the damping. These penalty terms add a spring and damping term for each constraint function allowing for soft constraint resolution. Previously, in PBD, softness was added to a Lagrange multiplier system using solver relaxation, which is iteration dependent. This was the big benefit of XPBD, and allowed for more consistent and stable systems.

The forces $\frac{\partial V}{\partial \mathbf{X}}$ are naturally solved using the per body method of (8.5). However, in their paper they treat the forces as constraints, as in (8.16) making the connection back to PBD and allowing for per constraint rather than per body solver iterations.

8.4.1 XPBD as a Regularization on PBD

Note that the presence of the additional $\tilde{\alpha}_i$ term in the denominator acts similarly to the regularization parameter we explored in section §4.8. For large stiffness values $\tilde{\alpha} \rightarrow 0$ and we get back to something like the Lagrange multiplier result. For simplicity, let the damping be zero and just focus on the spring like term. Let $k_i\Delta t^2 \rightarrow \infty$ so that $\tilde{\alpha}_i \rightarrow \epsilon$ for $\epsilon \rightarrow 0$. Then the XPBD update becomes

$$\boldsymbol{\lambda}_i^{k+1}\Delta t^2 = \boldsymbol{\lambda}_i^k\Delta t^2 - \xi \left(\epsilon + \mathcal{D}\mathbf{c}M^{-1}(\mathcal{D}\mathbf{c})^T \right)_{ii}^{-1} \left(\mathbf{c}(\mathbf{X}^{t+1} + \Delta\mathbf{X}) + \epsilon \boldsymbol{\lambda}_i^k \Delta t^2 \right)_i. \quad (8.37)$$

So for infinitely stiff spring coefficients k_i the update is exactly the same as the PBD update, but for non-infinite springs we get a relaxation term in the denominator, like we did when explicitly adding regularization in section §4.8, which helps with convergence of ill-posed problems (e.g. over-constrained systems).

For large but not infinite stiffness, XPBD behaves like regularized PBD update, making it more stable.

This is explored more in [18].

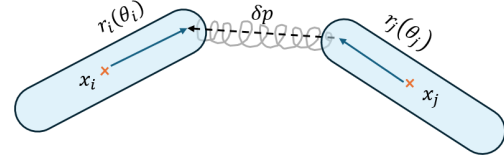
8.5 Compliant Constraint Examples

8.5.1 Spring Damper

Consider a system of two rigid bodies connected by a spring at an offset $\mathbf{r}_i$ and $\mathbf{r}_j$ from the respective centres of mass of the two bodies. The linear force is given by

$$\mathbf{f} = -k\delta\mathbf{p}(\mathbf{X}) - \nu\delta\dot{\mathbf{p}}(\mathbf{X}), \quad (8.38)$$

where k is the spring coefficient, ν is the damping coefficient, and $\delta\mathbf{p}(\mathbf{X})$ is the vector from the connection point on body i to body j .



$$\mathbf{p} = \mathbf{x} + \mathbf{r}(\boldsymbol{\theta}). \quad (8.39)$$

The torque is $\boldsymbol{\tau}_w = \mathbf{r} \times \mathbf{f}$, so the overall generalized force is

$$\mathbf{F}(\mathbf{X}) = \begin{pmatrix} \mathbf{f} \\ \mathbf{r}_i \times \mathbf{f} \\ -\mathbf{f} \\ -\mathbf{r}_j \times \mathbf{f} \end{pmatrix} = \begin{pmatrix} \mathbb{1} \\ [\mathbf{r}_i]_{\times} \\ -\mathbb{1} \\ -[\mathbf{r}_j]_{\times} \end{pmatrix} \mathbf{f}. \quad (8.40)$$

This force can be derived from a constraint

$$\mathbf{c}(\mathbf{X}) = \delta\mathbf{p}(\mathbf{X}) \quad (8.41)$$

and using the XPBD method of adding constraints as a spring-like potential and dissipation function

$$V = \frac{1}{2}k\|\delta\mathbf{p}\|^2, \quad \mathcal{F} = \frac{1}{2}\nu\|\delta\dot{\mathbf{p}}\|^2 \quad (8.42)$$

with

$$\mathcal{D}\mathbf{c} = \begin{pmatrix} \mathbb{1} & -[\mathbf{r}_i]_{\times} & -\mathbb{1} & [\mathbf{r}_j]_{\times} \end{pmatrix} \quad (8.43)$$

and

$$\begin{aligned} \mathbf{F}(\mathbf{X}) &= -(\mathcal{D}\mathbf{c})^T (k\mathbf{c}(\mathbf{X}) + \nu\dot{\mathbf{c}}(\mathbf{X})), \\ &= \begin{pmatrix} \mathbb{1} \\ [\mathbf{r}_i]_{\times} \\ -\mathbb{1} \\ -[\mathbf{r}_j]_{\times} \end{pmatrix} (-k\delta\mathbf{p} - \nu\delta\dot{\mathbf{p}}). \end{aligned} \quad (8.44)$$

We use the constraint derivative to compute the inverse effective mass

$$\begin{aligned}\mathcal{M}^{-1} &= \begin{pmatrix} \mathbb{1} & -[\mathbf{r}_i]_{\times} & -\mathbb{1} & [\mathbf{r}_j]_{\times} \end{pmatrix} \begin{pmatrix} \frac{1}{m_i} \mathbb{1} & & & \\ & I_i & & \\ & & \frac{1}{m_j} \mathbb{1} & \\ & & & I_j \end{pmatrix}^{-1} \begin{pmatrix} \mathbb{1} \\ [\mathbf{r}_i]_{\times} \\ -\mathbb{1} \\ -[\mathbf{r}_j]_{\times} \end{pmatrix}, \\ &= \left(\frac{1}{m_i} + \frac{1}{m_j} \right) \mathbb{1} - [\mathbf{r}_i]_{\times} I_i^{-1} [\mathbf{r}_i]_{\times} - [\mathbf{r}_j]_{\times} I_j^{-1} [\mathbf{r}_j]_{\times}.\end{aligned}\quad (8.45)$$

Comparing to (8.30) we see find the per iteration update:

Spring damper constraint update.

Lagrange multiplier update:

$$\boldsymbol{\lambda}^{k+1} \Delta t^2 = \boldsymbol{\lambda}^k \Delta t^2 - \xi \left(\mathbb{1} + \alpha \mathcal{M}^{-1} \right)^{-1} \left(k \Delta t^2 \delta \mathbf{p}(\mathbf{X}^{t+1} + \Delta \mathbf{X}) + \nu \Delta t \delta \dot{\mathbf{p}}(\mathbf{X}^{t+1} + \Delta \mathbf{X}) \Delta t + \boldsymbol{\lambda}^k \Delta t^2 \right), \quad (8.46)$$

where $\alpha = k \Delta t^2 + \nu \Delta t$

Delta:

$$\Delta = (\boldsymbol{\lambda}^{k+1} - \boldsymbol{\lambda}^k) \Delta t^2 \quad (8.47)$$

Configuration update:

$$\begin{aligned}\Delta \mathbf{x}'_i &= \Delta \mathbf{x}_i + \frac{1}{m_i} \Delta, \\ \Delta \boldsymbol{\omega}'_i &= \Delta \boldsymbol{\omega}_i + I_i^{-1} \mathbf{r}_i \times \Delta, \\ \Delta \mathbf{x}'_j &= \Delta \mathbf{x}_j - \frac{1}{m_j} \Delta, \\ \Delta \boldsymbol{\omega}'_j &= \Delta \boldsymbol{\omega}_j - I_j^{-1} \mathbf{r}_j \times \Delta,\end{aligned}\quad (8.48)$$

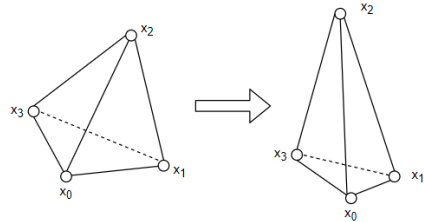
8.5.2 Volumetric Constraint

Up until this point we have been developing constraints between two rigid bodies, but it's worth showing how the method can be used for constraints between multiple bodies. As an example, consider a constraint between four particles making up the nodes of a tetrahedron. We introduce a constraint on the volume V of the tetrahedron

$$c(\mathbf{X}) = 6(V(\mathbf{X}) - V_0). \quad (8.49)$$

If $\mathbf{x}_a$, $a = 0, \dots, 3$, are the positions of the particles then the volume of the tetrahedron is

$$V(\mathbf{X}) = \frac{1}{6} \mathbf{d}_1 \cdot (\mathbf{d}_2 \times \mathbf{d}_3), \quad \mathbf{d}_a = \mathbf{x}_a - \mathbf{x}_0. \quad (8.50)$$



As we are dealing with point particles, there are no angular degrees of freedom and the constraint derivative is

$$\begin{aligned}
\mathcal{D}\mathbf{c} &= \begin{pmatrix} \frac{\partial c}{\partial \mathbf{x}_0} & \frac{\partial c}{\partial \mathbf{x}_1} & \frac{\partial c}{\partial \mathbf{x}_2} & \frac{\partial c}{\partial \mathbf{x}_3} \end{pmatrix}, \\
&= \left(-[(\mathbf{d}_2 \times \mathbf{d}_3) + (\mathbf{d}_3 \times \mathbf{d}_1) + (\mathbf{d}_1 \times \mathbf{d}_2)]^T \quad (\mathbf{d}_2 \times \mathbf{d}_3)^T \quad (\mathbf{d}_3 \times \mathbf{d}_1)^T \quad (\mathbf{d}_1 \times \mathbf{d}_2)^T \right), \\
&= 2 \begin{pmatrix} \mathbf{A}_0^T & \mathbf{A}_1^T & \mathbf{A}_2^T & \mathbf{A}_3^T \end{pmatrix}, \tag{8.51}
\end{aligned}$$

where $\mathbf{A}_a$ is the directed area (area times normal) of the triangle opposite the point a , e.g. $\mathbf{A}_1 = \frac{1}{2} \mathbf{d}_2 \times \mathbf{d}_3$, and $\mathbf{A}_0 = -(\mathbf{A}_1 + \mathbf{A}_2 + \mathbf{A}_3)$.

Using this, the effective mass is

$$\mu = \frac{1}{4} \left(\sum_{a=0}^3 \frac{1}{m_a} \|\mathbf{A}_a\|^2 \right)^{-1}. \tag{8.52}$$

Using this in an XPBD update we get

$$\begin{aligned}
\lambda^{k+1} \Delta t^2 &= \lambda^k \Delta t^2 - \xi \left(\tilde{\alpha} + (1 + \gamma) \mu^{-1} \right)^{-1} \\
&\quad \left(6(V(\mathbf{x}^{t+1} + \Delta \mathbf{x}^k) - V_0) + 2\gamma \sum_{a=0}^3 \mathbf{A}_a \cdot (\dot{\mathbf{x}}_a^{t+1} \Delta t + \Delta \mathbf{x}_a^k) + \tilde{\alpha} \lambda^k \Delta t^2 \right) \tag{8.53}
\end{aligned}$$

with

$$\Delta \mathbf{x}'_a = \Delta \mathbf{x}_a + \frac{1}{m_a} 2 \mathbf{A}_a (\lambda^{k+1} - \lambda^k) \Delta t^2. \tag{8.54}$$

So, each body is pushed in the direction of the normal of the triangle opposite to it.

9 Augmented Lagrangian Method

The augmented Lagrangian method [17, 27] combines the spirit of the compliant constraint method and the method of Lagrange multipliers.

We saw in (5.5) that solving the constrained Lagrangian to first order in $\Delta \mathbf{X}$ is equivalent to solving

$$\min_{\Delta \mathbf{X}} \frac{1}{2} \Delta \mathbf{X} \cdot M \Delta \mathbf{X} + \boldsymbol{\lambda} \cdot (\mathbf{c}(\mathbf{X}^{t+1}) + \mathcal{D}\mathbf{c}\Delta \mathbf{X}) \Delta t^2 \quad (9.1)$$

The augmented Lagrangian method adds an additional penalty term to the Lagrangian $\frac{1}{2}\mathbf{c}(\mathbf{X}) \cdot \rho \mathbf{c}(\mathbf{X})$ for a diagonal matrix of penalty coefficients ρ . For our linearized system this becomes

$$\begin{aligned} \min_{\Delta \mathbf{X}} \frac{1}{2} \Delta \mathbf{X} \cdot M \Delta \mathbf{X} + \boldsymbol{\lambda} \cdot (\mathbf{c}(\mathbf{X}^{t+1}) + \mathcal{D}\mathbf{c}\Delta \mathbf{X}) \Delta t^2 \\ + \frac{1}{2} \left(\mathbf{c}(\mathbf{X}^{t+1}) + \mathcal{D}\mathbf{c}\Delta \mathbf{X} + \frac{1}{2} \Delta \mathbf{X} \cdot \mathcal{D}^2 \mathbf{c} \Delta \mathbf{X} \right) \cdot \rho \left(\mathbf{c}(\mathbf{X}^{t+1}) + \mathcal{D}\mathbf{c}\Delta \mathbf{X} + \frac{1}{2} \Delta \mathbf{X} \cdot \mathcal{D}^2 \mathbf{c} \Delta \mathbf{X} \right) \Delta t^2. \end{aligned} \quad (9.2)$$

$$(9.3)$$

Where $\mathcal{D}^2 \mathbf{c}$ is the constraint Hessian using the left trivialized derivative (3.37). Note that I am including terms of order $\Delta \mathbf{X}^2$ and $\Delta \mathbf{X} \cdot \boldsymbol{\lambda}$ in the function as we want the result of the minimization to be linear in $\Delta \mathbf{X}$ and $\boldsymbol{\lambda}$,

The solution to this is

The Augmented Lagrangian system

$$H \Delta \mathbf{X} = -(\mathcal{D}\mathbf{c})^T (\boldsymbol{\lambda} + \rho \mathbf{c}(\mathbf{X}^{t+1})) \Delta t^2 \quad (9.4)$$

where

$$H = M + (\mathcal{D}\mathbf{c})^T \rho \Delta t^2 \mathcal{D}\mathbf{c} + \mathbf{c}(\mathbf{X}^{t+1}) \cdot \rho \Delta t^2 \mathcal{D}^2 \mathbf{c}. \quad (9.5)$$

The augmented Lagrangian method solves this iteratively by first fixing $\boldsymbol{\lambda}^k$ and solving for $\Delta \mathbf{X}$ and then updating the Lagrange multiplier

$$\boldsymbol{\lambda}^{k+1} = \boldsymbol{\lambda}^k + \rho \mathbf{c}(\mathbf{X}^{t+1} + \Delta \mathbf{X}) \quad (9.6)$$

This is then repeated for $k = 1$ to N iterations.

Notice that if the solution converges then $\boldsymbol{\lambda}^{k+1} \sim \boldsymbol{\lambda}^k$ and so $\mathbf{c}(\mathbf{X}^\infty) \sim 0$ and the constraint equations are satisfied.

Using a Gauss-Seidel to solve for the displacements we have

$$\Delta \mathbf{X}_i^{k+1} = \Delta \mathbf{X}_i^k - \xi H_{ii}^{-1} \left((\mathcal{D}\mathbf{c})^T (\boldsymbol{\lambda}^k + \rho \mathbf{c}(\mathbf{X}^{t+1} + \Delta \mathbf{X})) \Delta t^2 + H \Delta \mathbf{X}^k \right)_i. \quad (9.7)$$

This can be rewritten in the same way as section §4.6 as a non-linear Gauss-Seidel update:

Non-linear iteration update for the augmented Lagrangian method

$$\Delta \mathbf{X}_i^{k+1} = \Delta \mathbf{X}_i^k - \xi H_{ii}^{-1} \left((\mathcal{D}\mathbf{c})^T (\boldsymbol{\lambda}^k + \rho \mathbf{c}(\mathbf{X}^{t+1} + \Delta \mathbf{X})) \Delta t^2 + M \Delta \mathbf{X}^k \right)_i. \quad (9.8)$$

I.e. we compute the constraint exactly at the current state $\mathbf{X}^{t+1} + \Delta\mathbf{X}$. Compare this to (8.5) and we see that this is just the force update we computed earlier with

$$\mathbf{F}(\mathbf{X}) = -(\mathcal{D}\mathbf{c})^T (\boldsymbol{\lambda}^k + \rho \mathbf{c}(\mathbf{X})). \quad (9.9)$$

After calculating the displacement update at $\boldsymbol{\lambda}^k$, we then update the Lagrange multipliers:

$$\boldsymbol{\lambda}_j^{k+1} = \boldsymbol{\lambda}_j^k + (\rho \mathbf{c}(\mathbf{X}^{t+1} + \Delta\mathbf{X}))_j. \quad (9.10)$$

9.1 Augmented Vertex Block Descent

The method we presented in the previous section is essentially the update from the Augmented Vertex Block Descent (AVBD) method [14], but derived using the same exponential map representation of rotation used in the derivations of PBD and XPBD. However, in the algorithm presented in the AVBD paper, some additional features are presented to improve performance and stability. One such feature they present is the use of an approximation for the constraint Hessian term $\mathbf{c}(\mathbf{X}^{t+1}) \cdot \rho \Delta t^2 \mathcal{D}^2 \mathbf{c}$. They use a diagonal approximation, which is more efficient but also, importantly, ensures that H remains symmetric positive definite, and the system remains convergent.

9.1.1 AVBD, PBD and Geometric Stiffness

By doing some rearranging, we can make an interesting comparison to PBD. Let D be the Hessian term

$$D = \mathbf{c}(\mathbf{X}^{t+1}) \cdot \rho \Delta t^2 \mathcal{D}^2 \mathbf{c}. \quad (9.11)$$

Then the equation to be solved is

$$H' \Delta \mathbf{X} = -(\mathcal{D}\mathbf{c})^T (\boldsymbol{\lambda} + \rho \mathbf{c}(\mathbf{X}^{t+1})) \Delta t^2, \quad (9.12)$$

where

$$\begin{aligned} H' &= (M' + (\mathcal{D}\mathbf{c})^T \rho \Delta t^2 \mathcal{D}\mathbf{c}), \\ M' &= M + D. \end{aligned} \quad (9.13)$$

The inverse of H' can be found using the Woodbury matrix identity

$$(A + UCV)^{-1} = A^{-1} - A^{-1}U(C^{-1} + VA^{-1}U)^{-1}VA^{-1}. \quad (9.14)$$

Using this

$$H'^{-1} = M'^{-1} - M'^{-1}(\mathcal{D}\mathbf{c})^T ((\rho \Delta t^2)^{-1} + \mathcal{D}\mathbf{c} M'^{-1}(\mathcal{D}\mathbf{c})^T)^{-1} (\mathcal{D}\mathbf{c}) M'^{-1} \quad (9.15)$$

and

$$M' \Delta \mathbf{X} = -(\mathcal{D}\mathbf{c})^T ((\rho \Delta t^2)^{-1} + \mathcal{D}\mathbf{c} M'^{-1}(\mathcal{D}\mathbf{c})^T)^{-1} (\rho \Delta t^2)^{-1} (\boldsymbol{\lambda} + \rho \mathbf{c}(\mathbf{X}^{t+1})) \Delta t^2. \quad (9.16)$$

Multiplying through by $\mathcal{D}\mathbf{c} M'^{-1}$ gives

$$\mathcal{D}\mathbf{c} \Delta \mathbf{X} = -\mathcal{D}\mathbf{c} M'^{-1}(\mathcal{D}\mathbf{c})^T ((\rho \Delta t^2)^{-1} + \mathcal{D}\mathbf{c} M'^{-1}(\mathcal{D}\mathbf{c})^T)^{-1} (\rho \Delta t^2)^{-1} (\boldsymbol{\lambda} + \rho \mathbf{c}(\mathbf{X}^{t+1})) \Delta t^2. \quad (9.17)$$

and

$$(\mathbb{1} + (\rho \Delta t^2)^{-1} (\mathcal{D}\mathbf{c} M'^{-1}(\mathcal{D}\mathbf{c})^T)^{-1}) \mathcal{D}\mathbf{c} \Delta \mathbf{X} = -\mathbf{c}(\mathbf{X}^{t+1}) - (\rho \Delta t^2)^{-1} \boldsymbol{\lambda} \Delta t^2. \quad (9.18)$$

At convergence the linearized constraint is satisfied, $\mathbf{c}(\mathbf{X}^{t+1}) + \mathcal{D}\mathbf{c} \Delta\mathbf{X} = 0$, so substituting $\mathcal{D}\mathbf{c} \Delta\mathbf{X} = -\mathbf{c}(\mathbf{X}^{t+1})$ gives

$$\mathcal{D}\mathbf{c} M'^{-1}(\mathcal{D}\mathbf{c})^T \lambda \Delta t^2 = \mathbf{c}(\mathbf{X}^{t+1}). \quad (9.19)$$

So, the equation for the Lagrange multipliers is the same as in the PBD case, but with

$$M \rightarrow M + D \quad (9.20)$$

(Note, we get $+\mathbf{c}$ due to the usual sign convention we're following for this section for λ where $\mathbf{c} > 0$ gives $\lambda > 0$).

With AVBD, the D term is approximated by its diagonal. Since M is also block diagonal then it is simple to find the inverse M'^{-1} and the matrix $\mathcal{D}\mathbf{c} M'^{-1}(\mathcal{D}\mathbf{c})^T$ is symmetric and the system is guaranteed to converge.

AVBD does two things:

1. It converts the problem to a new matrix inverse problem with different convergence properties, where the Gauss-Seidel update is per body instead of per constraint
2. It adds in a diagonal Hessian term that can improve accuracy and convergence.

However, the addition of the Hessian term can be applied to any of the methods by making an adjustment to the mass matrix

$$M \rightarrow M + \mathbf{c}(\mathbf{X}^{t+1}) \cdot \rho \Delta t^2 \mathcal{D}^2 \mathbf{c}. \quad (9.21)$$

where AVBD takes the diagonal of the Hessian term to make the calculation simpler and to guarantee convergence.

This Hessian term here is also called geometric stiffness and is discussed in more detail in [1, 35].

9.2 Convergence Comparison vs. PBD For Large Mass Ratios

It's interesting to compare the matrix we're inverting with the augmented Lagrangian method and the force based method of section §8.1 to the matrix inverse from PBD.

$$\begin{aligned} \text{PBD: } A &= \mathcal{D}\mathbf{c} M^{-1}(\mathcal{D}\mathbf{c})^T, \\ \text{AVBD: } A &= M + (\mathcal{D}\mathbf{c})^T \rho \Delta t^2 \mathcal{D}\mathbf{c} + \mathbf{c}(\mathbf{X}^{t+1}) \cdot \rho \Delta t^2 \mathcal{D}^2 \mathbf{c}, \\ \text{Force: } A &= M - (\mathcal{D}\mathbf{F})^T \Delta t^2 \end{aligned} \quad (9.22)$$

In PBD, the mass matrix is sandwiched between the constraint derivative terms, so mass terms can appear in the off-diagonals with terms like $(\mathcal{D}\mathbf{c})_i M_j^{-1} (\mathcal{D}\mathbf{c})_k^T$ for two constraints i and k that share a body j . The diagonal contains the full constraint effective mass inverse $A_{ii} \sim \mu_{ij}^{-1} = 1/m_i + 1/m_j$, whereas the off diagonals contain only the shared body's inverse mass $\sim 1/m_j$.

This means that, when looking at the per row diagonal dominance

$$r_i = \frac{\sum_{j \neq i} |A_{ij}|}{|A_{ii}|} \quad (9.23)$$

with PBD we get terms that depend on the mass ratio of the bodies, $r_i \sim \frac{1/m_j}{1/m_i + 1/m_j} = \frac{m_i}{m_i + m_j}$. If the shared body j is *light* compared with its neighbours, $m_j \ll m_i$, then $r_i \sim 1$ and the system will struggle to converge: a light body sandwiched between heavy ones is the hard case for PBD. This is the configuration analysed in section §10.

For the augmented Lagrangian method and for the force based method, the mass term appears on its own and is block diagonal, so a body's row has diagonal $m_j + \sum_c \rho_c \Delta t^2$ over the constraints c attached to it and off diagonals $\rho_c \Delta t^2$. For a body with two constraints of equal penalty this gives $r_j \sim \frac{2\rho\Delta t^2}{m_j + 2\rho\Delta t^2}$. A heavy body now *improves* convergence, the opposite of PBD. However, a light body coupled by stiff penalties, $\rho\Delta t^2 \gg m_j$, again gives $r_j \rightarrow 1$. The two methods therefore have different hard cases rather than one being insensitive to mass ratios; see section §10 for the chain example.

The benefit of the augmented Lagrangian method is that it effectively combines the Lagrange multiplier method of PBD with the constraint penalty method of XPBD, and in doing so can support soft constraints as well as hard constraints. In XPBD, hard constraints require the stiffness term in the constraint penalty function to go to infinity, but with the Augmented Lagrangian method, as it also includes a Lagrange multiplier term, that is not the case.

In addition, the convergence of the augmented Lagrangian method depends on mass ratios differently from PBD: heavy bodies help rather than hurt, while light bodies coupled by stiff penalties remain difficult.

9.3 Point to Point Constraint

For comparison, we can implement the point to point stiff constraint example in section §7.1 using the Augmented Lagrangian method.

The constraint is

$$c(\mathbf{X}) = \|\delta\mathbf{p}\| \quad (9.24)$$

for

$$\delta\mathbf{p} = \mathbf{p}_i - \mathbf{p}_j \quad (9.25)$$

where

$$\mathbf{p} = \mathbf{x} + \mathbf{r}(\boldsymbol{\theta}). \quad (9.26)$$

From this we get a constraint derivative

$$\mathcal{D}\mathbf{c} = \begin{pmatrix} \hat{\delta\mathbf{p}}^T & (\mathbf{r}_i \times \hat{\delta\mathbf{p}})^T & -\hat{\delta\mathbf{p}}^T & -(\mathbf{r}_j \times \hat{\delta\mathbf{p}})^T \end{pmatrix}. \quad (9.27)$$

The Hessian has two contributions,

$$\mathcal{D}^2\mathbf{c} = \frac{1}{|\delta\mathbf{p}|} (A + G), \quad (9.28)$$

where the first comes from the curvature of $\|\cdot\|$ acting on the linearised map,

$$A = \begin{pmatrix} (\mathbb{1} - P) & -(\mathbb{1} - P) [\mathbf{r}_i]_{\times} & -(\mathbb{1} - P) & (\mathbb{1} - P) [\mathbf{r}_j]_{\times} \\ [\mathbf{r}_i]_{\times} (\mathbb{1} - P) & -[\mathbf{r}_i]_{\times} (\mathbb{1} - P) [\mathbf{r}_i]_{\times} & -[\mathbf{r}_i]_{\times} (\mathbb{1} - P) & [\mathbf{r}_i]_{\times} (\mathbb{1} - P) [\mathbf{r}_j]_{\times} \\ -(\mathbb{1} - P) & (\mathbb{1} - P) [\mathbf{r}_i]_{\times} & (\mathbb{1} - P) & -(\mathbb{1} - P) [\mathbf{r}_j]_{\times} \\ -[\mathbf{r}_j]_{\times} (\mathbb{1} - P) & [\mathbf{r}_j]_{\times} (\mathbb{1} - P) [\mathbf{r}_i]_{\times} & [\mathbf{r}_j]_{\times} (\mathbb{1} - P) & -[\mathbf{r}_j]_{\times} (\mathbb{1} - P) [\mathbf{r}_j]_{\times} \end{pmatrix}, \quad (9.29)$$

with P the projection matrix

$$P = \hat{\delta \mathbf{p}} \hat{\delta \mathbf{p}}^T, \quad (9.30)$$

and the second comes from the curvature of the rotation map itself,

$$G = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & \frac{1}{2} (\delta \mathbf{p} \mathbf{r}_i^T + \mathbf{r}_i \delta \mathbf{p}^T) - (\delta \mathbf{p} \cdot \mathbf{r}_i) \mathbb{1} & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -\frac{1}{2} (\delta \mathbf{p} \mathbf{r}_j^T + \mathbf{r}_j \delta \mathbf{p}^T) + (\delta \mathbf{p} \cdot \mathbf{r}_j) \mathbb{1} \end{pmatrix}. \quad (9.31)$$

Note that G is written in terms of $\delta \mathbf{p}$ rather than $\hat{\delta \mathbf{p}}$, so that the single prefactor $1/|\delta \mathbf{p}|$ in (9.28) applies to both terms.

The A term is the one that appears if the constraint gradient (9.27) is differentiated as though $\delta \mathbf{p}$ were a linear function of the configuration. It is not: the rotational degrees of freedom enter through the exponential map, and a second derivative sees the second order term of that map, which the first derivative does not. Expanding,

$$e^{[\boldsymbol{\omega}] \times} \mathbf{r} = \mathbf{r} + \boldsymbol{\omega} \times \mathbf{r} + \frac{1}{2} \boldsymbol{\omega} \times (\boldsymbol{\omega} \times \mathbf{r}) + \mathcal{O}(|\boldsymbol{\omega}|^3), \quad (9.32)$$

so $\delta \mathbf{p}$ picks up a second order term

$$\delta \mathbf{p} \rightarrow \delta \mathbf{p} + \frac{1}{2} (\boldsymbol{\omega}_i \times (\boldsymbol{\omega}_i \times \mathbf{r}_i) - \boldsymbol{\omega}_j \times (\boldsymbol{\omega}_j \times \mathbf{r}_j)) + \dots \quad (9.33)$$

and therefore $c = |\delta \mathbf{p}|$ picks up the scalar

$$\frac{1}{2} \hat{\delta \mathbf{p}} \cdot (\boldsymbol{\omega} \times (\boldsymbol{\omega} \times \mathbf{r})) = \frac{1}{2} [(\hat{\delta \mathbf{p}} \cdot \boldsymbol{\omega})(\boldsymbol{\omega} \cdot \mathbf{r}) - (\hat{\delta \mathbf{p}} \cdot \mathbf{r})|\boldsymbol{\omega}|^2]. \quad (9.34)$$

The second order Taylor term of c is $\frac{1}{2} \mathcal{D}^2 c(v, v)$, so the two factors of $\frac{1}{2}$ cancel and the contribution to the Hessian is the quadratic form

$$(\hat{\delta \mathbf{p}} \cdot \boldsymbol{\omega})(\boldsymbol{\omega} \cdot \mathbf{r}) - (\hat{\delta \mathbf{p}} \cdot \mathbf{r})|\boldsymbol{\omega}|^2, \quad (9.35)$$

whose matrix is the angular block of $G/|\delta \mathbf{p}|$ in (9.31). The sign is opposite for body j , since $\mathbf{p}_j$ enters $\delta \mathbf{p}$ with a minus sign.

Because $[\mathbf{a}]_{\times} [\mathbf{b}]_{\times} = \mathbf{b} \mathbf{a}^T - (\mathbf{a} \cdot \mathbf{b}) \mathbb{1}$, the angular blocks of G can be written compactly as the symmetric part of a product of two skew matrices,

$$G_i = \frac{1}{2} ([\mathbf{r}_i]_{\times} [\delta \mathbf{p}]_{\times} + [\delta \mathbf{p}]_{\times} [\mathbf{r}_i]_{\times}), \quad G_j = -\frac{1}{2} ([\mathbf{r}_j]_{\times} [\delta \mathbf{p}]_{\times} + [\delta \mathbf{p}]_{\times} [\mathbf{r}_j]_{\times}), \quad (9.36)$$

or equivalently, as a quadratic form, $G_i(\boldsymbol{\omega}, \boldsymbol{\omega}) = -(\boldsymbol{\omega} \times \mathbf{r}_i) \cdot (\boldsymbol{\omega} \times \delta \mathbf{p})$. This is the form in which geometric stiffness is usually written for articulated rigid bodies [1].

We can also compute $(\mathcal{D} \mathbf{c})^T \mathcal{D} \mathbf{c}$

$$(\mathcal{D} \mathbf{c})^T \mathcal{D} \mathbf{c} = \begin{pmatrix} P & -P[\mathbf{r}_i]_{\times} & -P & P[\mathbf{r}_j]_{\times} \\ [\mathbf{r}_i]_{\times} P & -[\mathbf{r}_i]_{\times} P[\mathbf{r}_i]_{\times} & -[\mathbf{r}_i]_{\times} P & [\mathbf{r}_i]_{\times} P[\mathbf{r}_j]_{\times} \\ -P & P[\mathbf{r}_i]_{\times} & P & -P[\mathbf{r}_j]_{\times} \\ -[\mathbf{r}_j]_{\times} P & [\mathbf{r}_j]_{\times} P[\mathbf{r}_i]_{\times} & [\mathbf{r}_j]_{\times} P & -[\mathbf{r}_j]_{\times} P[\mathbf{r}_j]_{\times} \end{pmatrix}. \quad (9.37)$$

Letting

$$Q = \begin{pmatrix} \mathbb{1} & -[\mathbf{r}_i]_{\times} & -\mathbb{1} & [\mathbf{r}_j]_{\times} \end{pmatrix}, \quad (9.38)$$

these two are just

$$(\mathcal{D}\mathbf{c})^T \mathcal{D}\mathbf{c} = Q^T P Q, \quad A = Q^T (\mathbb{1} - P) Q, \quad (9.39)$$

so that, since $c = |\delta\mathbf{p}|$,

$$(\mathcal{D}\mathbf{c})^T \mathcal{D}\mathbf{c} + c(\mathbf{X}) \mathcal{D}^2 \mathbf{c} = Q^T P Q + Q^T (\mathbb{1} - P) Q + G = Q^T Q + G, \quad (9.40)$$

i.e. the projection operators cancel and only G survives. Plugging this into

$$H = M + \rho\Delta t^2 (\mathcal{D}\mathbf{c})^T \mathcal{D}\mathbf{c} + \rho\Delta t^2 c(\mathbf{X}) \mathcal{D}^2 \mathbf{c}(\mathbf{X}) \quad (9.41)$$

gives

$$H = M + \rho\Delta t^2 \begin{pmatrix} \mathbb{1} & -[\mathbf{r}_i]_{\times} & -\mathbb{1} & [\mathbf{r}_j]_{\times} \\ [\mathbf{r}_i]_{\times} & -[\mathbf{r}_i]_{\times} [\mathbf{r}_i]_{\times} & -[\mathbf{r}_i]_{\times} & [\mathbf{r}_i]_{\times} [\mathbf{r}_j]_{\times} \\ -\mathbb{1} & [\mathbf{r}_i]_{\times} & \mathbb{1} & -[\mathbf{r}_j]_{\times} \\ -[\mathbf{r}_j]_{\times} & [\mathbf{r}_j]_{\times} [\mathbf{r}_i]_{\times} & [\mathbf{r}_j]_{\times} & -[\mathbf{r}_j]_{\times} [\mathbf{r}_j]_{\times} \end{pmatrix} + \rho\Delta t^2 G. \quad (9.42)$$

Since M and G are both block diagonal it makes sense to combine them. Defining

$$M' = M + \rho\Delta t^2 G, \quad (9.43)$$

the H matrix for this particular system can be written in the simple form

$$H = M' + Q^T (\rho\Delta t^2) Q. \quad (9.44)$$

The effect of the geometric stiffness is therefore entirely absorbed into a modified inertia for each body,

$$\begin{aligned} I'_i &= I_i + \rho\Delta t^2 \left(\frac{1}{2} (\delta\mathbf{p} \mathbf{r}_i^T + \mathbf{r}_i \delta\mathbf{p}^T) - (\delta\mathbf{p} \cdot \mathbf{r}_i) \mathbb{1} \right), \\ I'_j &= I_j - \rho\Delta t^2 \left(\frac{1}{2} (\delta\mathbf{p} \mathbf{r}_j^T + \mathbf{r}_j \delta\mathbf{p}^T) - (\delta\mathbf{p} \cdot \mathbf{r}_j) \mathbb{1} \right), \end{aligned} \quad (9.45)$$

with the linear blocks of M' unchanged.

Unlike M , however, M' is not guaranteed to be positive definite. The eigenvalues of G_i are

$$\frac{1}{2} (|\delta\mathbf{p}| |\mathbf{r}_i| - \delta\mathbf{p} \cdot \mathbf{r}_i), \quad -\frac{1}{2} (|\delta\mathbf{p}| |\mathbf{r}_i| + \delta\mathbf{p} \cdot \mathbf{r}_i), \quad -\delta\mathbf{p} \cdot \mathbf{r}_i, \quad (9.46)$$

one non-negative and one non-positive, so G_i is indefinite whenever $\delta\mathbf{p} \neq 0$.

A sufficient condition for I'_i to remain positive definite is

$$\rho\Delta t^2 |\delta\mathbf{p}| |\mathbf{r}_i| < \lambda_{\min}(I_i), \quad (9.47)$$

i.e. the term is harmless for stiff, nearly satisfied constraints and dangerous for large penalty coefficients combined with large constraint error. Note that $G \propto \delta\mathbf{p}$, so the geometric stiffness vanishes as the constraint converges and $M' \rightarrow M$. This is the reason AVBD replaces this term with a diagonal approximation, as discussed in section §9.1: the approximation is chosen to keep H symmetric positive definite unconditionally.

In the standard Gauss-Seidel algorithm, we solve the equation

$$H \Delta \mathbf{X} = -(\mathcal{D}\mathbf{c})^T (\boldsymbol{\lambda} + \rho \mathbf{c}(\mathbf{X}^{t+1})) \Delta t^2 \quad (9.48)$$

row by row or, in the rigid body case, body by body (i.e. 6 rows at a time). Before we do that, it is interesting to look at what the exact solution is for the full inverse of the 12×12 matrix H in this case of a single constraint between two bodies.

9.3.1 Exact Solution

The inverse of (9.44) can be found using the Woodbury matrix identity

$$(A + UCV)^{-1} = A^{-1} - A^{-1}U(C^{-1} + VA^{-1}U)^{-1}VA^{-1}. \quad (9.49)$$

Using this we get

$$H^{-1} = M'^{-1} - M'^{-1}Q^T \left(\frac{1}{\rho\Delta t^2} \mathbb{1} + QM'^{-1}Q^T \right)^{-1} QM'^{-1}. \quad (9.50)$$

Notice that $\left(\frac{1}{\rho\Delta t^2} \mathbb{1} + QM'^{-1}Q^T \right)^{-1}$ is a 3×3 matrix inversion

$$\left(\frac{1}{\rho\Delta t^2} \mathbb{1} + QM'^{-1}Q^T \right)^{-1} = \rho\Delta t^2 (\mathbb{1} + \rho\Delta t^2 \mu^{-1})^{-1}, \quad (9.51)$$

where $\mu^{-1} = QM'^{-1}Q^T$ is the inverse effective mass matrix, which takes the same form as in the stiff case but with the modified inertias (9.45),

$$\mu^{-1} = \left(\frac{1}{m_i} + \frac{1}{m_j} \right) \mathbb{1} - [\mathbf{r}_i]_{\times} I_i'^{-1} [\mathbf{r}_i]_{\times} - [\mathbf{r}_j]_{\times} I_j'^{-1} [\mathbf{r}_j]_{\times}. \quad (9.52)$$

Since G is a rank two update to a scalar shift of the inertia, I'^{-1} can itself be evaluated with a further Woodbury step involving only a 2×2 inverse, so (9.52) costs no more than the uncorrected version.

Using this, the formula for $\Delta \mathbf{X}$ is (assuming $\boldsymbol{\lambda}^0 = 0$)

$$M' \Delta \mathbf{X}^{k+1} = -Q^T (\mathbb{1} + \rho\Delta t^2 \mu^{-1})^{-1} \rho\Delta t^2 (\mathbf{p}(\mathbf{X}_i^{t+1}) - \mathbf{p}(\mathbf{X}_j^{t+1})). \quad (9.53)$$

So, if

$$\mathbf{f} = -(\mathbb{1} + \rho\Delta t^2 \mu^{-1})^{-1} \rho\Delta t^2 (\mathbf{p}(\mathbf{X}_i^{t+1}) - \mathbf{p}(\mathbf{X}_j^{t+1})) \quad (9.54)$$

then

$$\begin{aligned} \Delta \mathbf{x}_i &= \frac{1}{m_i} \mathbf{f}, \\ \Delta \boldsymbol{\omega}_i &= I_i'^{-1} (\mathbf{r}_i \times \mathbf{f}), \\ \Delta \mathbf{x}_j &= -\frac{1}{m_j} \mathbf{f}, \\ \Delta \boldsymbol{\omega}_j &= -I_j'^{-1} (\mathbf{r}_j \times \mathbf{f}). \end{aligned} \quad (9.55)$$

This is the result for a soft constraint. Notice that it is of the same form as the result for the stiff point to point constraint of section §7.1, except that now μ^{-1} is the full effective mass matrix rather than a scalar, and the inertia of each body is modified by the geometric stiffness term.

9.3.2 Iterative Solution

Solving per body instead of for the full matrix, we can again use the Woodbury matrix identity, except this time for

$$H_i = M_i + \rho\Delta t^2 Q_i^T Q_i \quad (9.56)$$

with

$$M_i = \begin{pmatrix} m_i \mathbb{1} & 0 \\ 0 & I_i \end{pmatrix} \quad Q_i = (\mathbb{1} \quad -[\mathbf{r}_i]_{\times}) \quad (9.57)$$

The inverse is

$$H_i^{-1} = M_i^{-1} - M_i^{-1} Q_i^T \left(\frac{1}{\rho \Delta t^2} \mathbb{1} + Q_i M_i^{-1} Q_i^T \right)^{-1} Q_i M_i^{-1}. \quad (9.58)$$

Note that $\left(\frac{1}{\rho \Delta t^2} \mathbb{1} + Q_i M_i^{-1} Q_i^T \right)^{-1}$ is

$$\left(\frac{1}{\rho \Delta t^2} \mathbb{1} + Q_i M_i^{-1} Q_i^T \right)^{-1} = \rho \Delta t^2 (\mathbb{1} + \rho \Delta t^2 \mu_i^{-1})^{-1}, \quad (9.59)$$

where μ_i^{-1} is the inverse effective mass matrix of body i alone

$$\mu_i^{-1} = \frac{1}{m_i} \mathbb{1} - [\mathbf{r}_i]_{\times} I_i^{-1} [\mathbf{r}_i]_{\times}. \quad (9.60)$$

Using this, the formula for $\Delta \mathbf{X}_i$ is

$$M_i \Delta \mathbf{X}_i^{k+1} = Q_i^T (1 + \rho \Delta t^2 \mu_i^{-1})^{-1} \left(\rho \Delta t^2 Q_i \Delta \mathbf{X}_i^k - (\lambda^k \hat{\delta} \mathbf{p}^k + \rho \delta \mathbf{p}^k) \Delta t^2 \right). \quad (9.61)$$

So, if

$$\mathbf{f} = -(1 + \rho \Delta t^2 \mu_i^{-1})^{-1} \left(\rho \Delta t^2 Q_i \Delta \mathbf{X}_i^k - (\lambda^k \hat{\delta} \mathbf{p}^k + \rho \delta \mathbf{p}^k) \Delta t^2 \right) \quad (9.62)$$

then

$$\begin{aligned} \Delta \mathbf{x}_i &= \frac{1}{m_i} \mathbf{f}, \\ \Delta \boldsymbol{\omega}_i &= I_i^{-1} \mathbf{r}_i \times \mathbf{f}, \end{aligned} \quad (9.63)$$

The formula for $\Delta \mathbf{X}_j$ will be the same but with $\mathbf{f}$ replaced by $-\mathbf{f}$. There is a difference from the exact solution though. On the first iteration, if $\boldsymbol{\lambda}^0 = 0$, when solving for $\mathbf{X}_i^{k+1}$, $\mathbf{f}$ is

$$\mathbf{f}_i^1 = -(\mathbb{1} + \rho \Delta t^2 \mu_i^{-1})^{-1} \rho \Delta t^2 \left(\mathbf{p}(\mathbf{X}_i^{t+1}) - \mathbf{p}(\mathbf{X}_j^{t+1}) \right), \quad (9.64)$$

i.e. the constraint function is evaluated in the unperturbed state. However, as we are using Gauss-Seidel, when we go so solve for $\Delta \mathbf{X}_j^{k+1}$ we use the latest value of $\Delta \mathbf{X}_i$, so

$$\mathbf{f}_j^1 = \left(\mathbb{1} + \rho \Delta t^2 \mu_j^{-1} \right)^{-1} \rho \Delta t^2 \left(\mathbf{p}(\mathbf{X}_i^{t+1} + \Delta \mathbf{X}_i^{k+1}) - \mathbf{p}(\mathbf{X}_j^{t+1}) \right), \quad (9.65)$$

i.e. the constraint is evaluated in the new configuration for body i of $\mathbf{X}_i^{t+1} + \Delta \mathbf{X}_i^{k+1}$, and the mass term is μ_j and not μ_i . We do not apply an equal and opposite impulse due to the constraint in a single iteration, and so momentum is not conserved per iteration. However, over the course of multiple iterations the system should converge to the correct solution.

To illustrate this consider, for simplicity, the case of particles instead of bodies, where we just have linear degrees of freedom

$$\begin{aligned}
\mathbf{f}_i^1 &= -\frac{m_i \rho \Delta t^2}{m_i + \rho \Delta t^2} \delta \mathbf{x}^{t+1}, \\
\Delta \mathbf{x}_i^1 &= \frac{1}{m_i} \mathbf{f}_i^1 = -\frac{\rho \Delta t^2 / m_i}{1 + \rho \Delta t^2 / m_i} \delta \mathbf{x}^{t+1}, \\
\mathbf{f}_j^1 &= \frac{m_j \rho \Delta t^2}{m_j + \rho \Delta t^2} \left(\delta \mathbf{x}^{t+1} - \frac{\rho \Delta t^2 / m_i}{1 + \rho \Delta t^2 / m_i} \delta \mathbf{x}^{t+1} \right), \\
&= \frac{m_j \rho \Delta t^2}{(m_j + \rho \Delta t^2)(1 + \rho \Delta t^2 / m_i)} \delta \mathbf{x}^{t+1}, \\
\Delta \mathbf{x}_j^1 &= \frac{\rho \Delta t^2 / m_j}{(1 + \rho \Delta t^2 (1/m_i + 1/m_j) + (\rho \Delta t^2)^2 / (m_i m_j))} \delta \mathbf{x}^{t+1}, \tag{9.66}
\end{aligned}$$

Even after one iteration, $\Delta \mathbf{x}^1$ is approaching the correct solution of

$$\Delta \mathbf{x}_j^1 = \frac{\rho \Delta t^2 / m_j}{1 + \rho \Delta t^2 (1/m_i + 1/m_j)} \delta \mathbf{x}^{t+1} \tag{9.67}$$

with an error in the denominator of order $(\rho \Delta t^2)^2$.

For the next iteration λ gets updated

$$\lambda^1 = \rho \|\delta \mathbf{p}(\mathbf{x}^{t+1} + \Delta \mathbf{x}^k)\| \tag{9.68}$$

then subsequent updates bring $\Delta \mathbf{x}_i$ and $\Delta \mathbf{x}_j$ closer to the exact solution.

The iterative update converges quickly to the exact solution, but per iteration, since forces are applied asymmetrically, momentum is not necessarily conserved.

10 Convergence of Constraint Chains

In this section, we can bring together the results from the previous sections and analyze the convergence of a simple chain of constrained rigid bodies using the different methodologies. We will analyze how convergence is affected by the number of iterations and the mass ratios of the bodies in the chain.

For more detail on the derivation of the results of this section see, for example, [37].

10.1 Setup

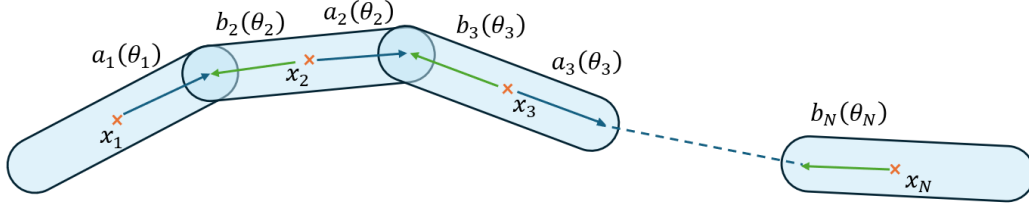


Figure 1: Rigid chain connected by point to point constraints.

The setup is a set of bodies $i = 1, \dots, N$ with point to point constraints between adjacent bodies

$$\mathbf{c}(\mathbf{X}) = \begin{pmatrix} \|\delta \mathbf{p}_{12}\| \\ \|\delta \mathbf{p}_{23}\| \\ \vdots \\ \|\delta \mathbf{p}_{N-1\ N}\| \end{pmatrix} \quad (10.1)$$

where

$$\delta \mathbf{p}_{ij} = \mathbf{x}_i + \mathbf{a}_i(\boldsymbol{\theta}_i) - \mathbf{x}_j - \mathbf{b}_j(\boldsymbol{\theta}_j) \quad (10.2)$$

The constraint derivative matrix is

$$\mathcal{D}\mathbf{c} = \begin{pmatrix} \hat{\delta \mathbf{p}}_{12}^T & (\mathbf{a}_1 \times \hat{\delta \mathbf{p}}_{12})^T & -\hat{\delta \mathbf{p}}_{12}^T & -(\mathbf{b}_2 \times \hat{\delta \mathbf{p}}_{12})^T & & & \\ & \hat{\delta \mathbf{p}}_{23}^T & (\mathbf{a}_2 \times \hat{\delta \mathbf{p}}_{23})^T & -\hat{\delta \mathbf{p}}_{23}^T & -(\mathbf{b}_3 \times \hat{\delta \mathbf{p}}_{23})^T & & \\ & & \ddots & & & \ddots & \\ & & & -\hat{\delta \mathbf{p}}_{N-1\ N}^T & -(\mathbf{b}_N \times \hat{\delta \mathbf{p}}_{N-1\ N})^T & & \end{pmatrix} \quad (10.3)$$

And the matrix $\mathcal{D}\mathbf{c} M^{-1}(\mathcal{D}\mathbf{c})^T$ is block tri-diagonal

$$\mathcal{D}\mathbf{c} M^{-1}(\mathcal{D}\mathbf{c})^T = \begin{pmatrix} \mu_{12}^{-1} & -\alpha_{13} & & & \\ -\alpha_{13} & \mu_{23}^{-1} & -\alpha_{24} & & \\ & -\alpha_{24} & \mu_{34}^{-1} & & \\ & & & \ddots & \\ & & & & \mu_{N-1\ N}^{-1} \end{pmatrix} \quad (10.4)$$

where the diagonal elements are the inverse effective mass between adjacent bodies

$$\mu_{i\ i+1}^{-1} = \hat{\delta \mathbf{p}}_{i\ i+1} \cdot \left(\left(\frac{1}{m_i} + \frac{1}{m_{i+1}} \right) \mathbb{1} - [\mathbf{a}_i]_{\times} I_i^{-1} [\mathbf{a}_i]_{\times} - [\mathbf{b}_{i+1}]_{\times} I_{i+1}^{-1} [\mathbf{b}_{i+1}]_{\times} \right) \hat{\delta \mathbf{p}}_{i\ i+1} \quad (10.5)$$

and the off diagonal elements represent coupling between adjacent constraints

$$\alpha_{i+2} = \hat{\mathbf{p}}_{i+1} \cdot \left(\frac{1}{m_{i+1}} \mathbb{1} - [\mathbf{b}_{i+1}]_{\times} I_{i+1}^{-1} [\mathbf{a}_{i+1}]_{\times} \right) \hat{\mathbf{p}}_{i+2}, \quad (10.6)$$

which enters the matrix with a minus sign because the block of $\mathcal{D}\mathbf{c}_i$ for body $i+1$ carries $-\hat{\mathbf{p}}_{i+1}$ while that of $\mathcal{D}\mathbf{c}_{i+1}$ carries $+\hat{\mathbf{p}}_{i+2}$. In general, the coupling will be strongest when $\hat{\mathbf{p}}_{i+1} = \hat{\mathbf{p}}_{i+2}$, e.g. the bodies are all lined up in a straight line. In this case, the correction to solve one constraint will directly feed into the error for the adjacent constraint and the matrix reduces to

$$\begin{pmatrix} \left(\frac{1}{m_1} + \frac{1}{m_2}\right) & -\frac{1}{m_2} & & \\ -\frac{1}{m_2} & \left(\frac{1}{m_2} + \frac{1}{m_3}\right) & & \\ & & \ddots & \\ & & & \left(\frac{1}{m_{N-1}} + \frac{1}{m_N}\right) \end{pmatrix}. \quad (10.7)$$

Notice that only the first and last rows are strictly diagonally dominant. For the other rows

$$\frac{\sum_{i \neq j} |A_{ij}|}{|A_{ii}|} = \frac{\frac{1}{m_i} + \frac{1}{m_{i+1}}}{\frac{1}{m_i} + \frac{1}{m_{i+1}}} = 1. \quad (10.8)$$

The system is still guaranteed to converge, since the matrix is SPD, but the rate of convergence may be slow.

10.2 Effect of Iteration Count

To examine the rate of convergence, we find the spectral radius of the Gauss-Seidel iteration matrix $G = -(D + L)^{-1}L^T$. First, assume that all masses are equal, say $m_i = 1$, then the matrix is

$$\begin{pmatrix} 2 & -1 & & \\ -1 & 2 & -1 & \\ & -1 & \ddots & \\ & & & 2 \end{pmatrix}. \quad (10.9)$$

In this case, the iteration matrix is

$$G = \begin{pmatrix} 0 & \frac{1}{2} & & \\ 0 & \frac{1}{4} & \frac{1}{2} & \\ 0 & \frac{1}{8} & \frac{1}{4} & \frac{1}{2} \\ \vdots & \vdots & \vdots & \ddots \end{pmatrix} \quad (10.10)$$

and the spectral radius is

$$\rho(G) = \cos^2\left(\frac{\pi}{N}\right). \quad (10.11)$$

So for large N the spectral radius goes to

$$\rho(G) \rightarrow 1 - \frac{\pi^2}{N^2} \text{ as } N \rightarrow \infty. \quad (10.12)$$

So, as you would expect, as the number of links in the chain goes to infinity, the spectral radius goes to 1 and so the rate of convergence goes to zero.

In general, the error from one iteration to the next will decrease like

$$\begin{aligned}\|\epsilon^k\| &\sim |\rho(G)|^k \|\epsilon_0\|, \\ &\sim \left| \cos\left(\frac{\pi}{N}\right) \right|^{2k} \|\epsilon_0\|, \\ &\sim \exp\left(-\frac{k\pi^2}{N^2}\right) \|\epsilon_0\| \text{ for large } N\end{aligned}\tag{10.13}$$

So, to reduce the error to $1 - \alpha$ times the original error takes

$$k \sim \frac{N^2}{\pi^2} \ln\left(\frac{1}{1 - \alpha}\right)\tag{10.14}$$

iterations. See figure 2 for an example plot showing the iterations required to remove different fractions of the original error in a chain of length 10.

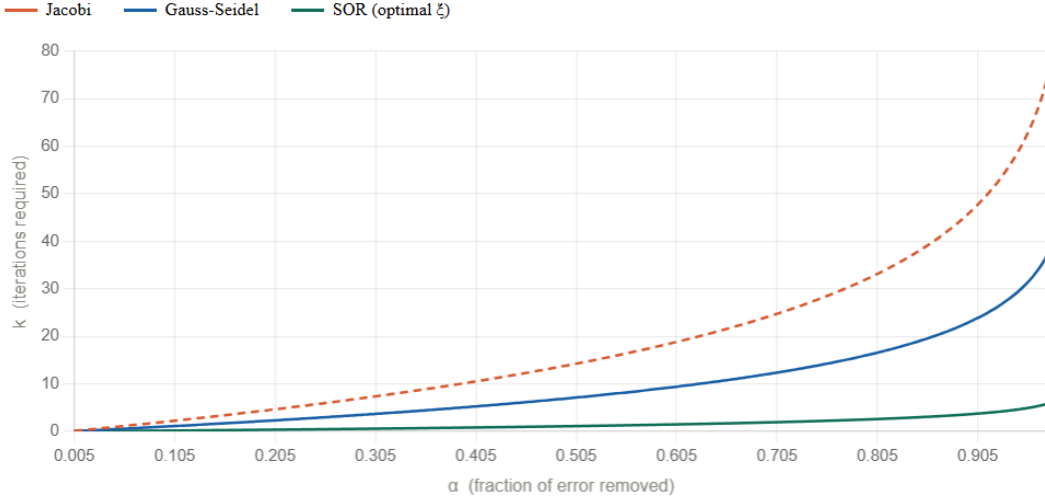


Figure 2: Comparing Gauss-Seidel vs. Jacobi vs. Gauss-Seidel with optimal SOR methods on the number of iterations required to reduce a fraction of the error in a chain of 10 bodies. The optimal relaxation in the SOR method is $\xi^* \sim 1.5$

10.2.1 Relaxation

The addition of a relaxation factor ξ modifies the iteration matrix by

$$G_\xi = (D + \xi L)^{-1}((1 - \xi)D - \xi L^T).\tag{10.15}$$

To find the eigenvalues λ of G_ξ we use two standard results. First, for the equal-mass chain the eigenvalues of the Jacobi iteration matrix G_J are

$$\mu_k = \cos\left(\frac{k\pi}{N}\right), \quad k = 1, \dots, N - 1.\tag{10.16}$$

Second, since A is tridiagonal (consistently ordered), the eigenvalues λ of G_ξ are related to the Jacobi eigenvalues μ_k by Young's relation [37]

$$(\lambda + \xi - 1)^2 = \xi^2 \mu_k^2 \lambda.\tag{10.17}$$

Rearranging, this is a quadratic equation in λ :

$$\lambda^2 - (\xi^2 \mu_k^2 - 2(\xi - 1))\lambda + (\xi - 1)^2 = 0 \quad (10.18)$$

with discriminant

$$\Delta = \xi^4 \mu_k^4 - 4\xi^2 \mu_k^2 (\xi - 1). \quad (10.19)$$

For $\xi = 1$ (standard Gauss-Seidel), $\Delta = \mu_k^4 > 0$ and both roots are real. As $\xi \rightarrow 2$, $\Delta \rightarrow 16\mu_k^2(\mu_k^2 - 1)$, which is negative since $|\mu_k| < 1$, giving two complex conjugate roots with magnitude $|\lambda| = \xi - 1 \rightarrow 1$. So $|\lambda|$ is minimized by the value of ξ for which the discriminant is zero, since that is the boundary between real and complex roots. Setting $\Delta = 0$ for the dominant Jacobi eigenvalue $\mu_1 = \cos(\frac{\pi}{N})$ gives

$$\xi^{*2} \mu_1^2 = 4(\xi^* - 1). \quad (10.20)$$

Using $\sqrt{1 - \mu_1^2} = \sin(\frac{\pi}{N})$, this gives

$$\xi^* = \frac{2}{1 + \sin(\frac{\pi}{N})}. \quad (10.21)$$

When $\Delta = 0$ the quadratic has a repeated root

$$\lambda^* = \xi^* - 1 = \frac{1 - \sin(\frac{\pi}{N})}{1 + \sin(\frac{\pi}{N})}, \quad (10.22)$$

In the limit of large N this gives

$$\xi^* \sim 2\left(1 - \frac{\pi}{N}\right), \quad \rho(G_{\xi^*}) \sim 1 - \frac{2\pi}{N}. \quad (10.23)$$

This is a substantial improvement over the standard Gauss-Seidel result $\rho(G) \sim 1 - \frac{\pi^2}{N^2}$, as the iteration count to remove a fraction α of the error is now

$$k \sim \frac{N}{2\pi} \ln\left(\frac{1}{1 - \alpha}\right), \quad (10.24)$$

scaling with N instead of N^2 .

The practical difficulty with SOR in general scenarios is that we do not always have a fixed constraint topology, so we cannot pre-compute an optimal ξ .

10.2.2 Jacobi Iterations

It is instructive to briefly look at the Jacobi method for solving the linear system. This is the same as the Gauss-Seidel method in that we solve row by row, but the difference being that we do not propagate the result from the solution of the previous row to the solution of the next row. In this method, the iteration matrix is

$$G_J = -D^{-1}(L + L^T). \quad (10.25)$$

For the case of a straight chain with equal masses, this is

$$G_J = \begin{pmatrix} 0 & \frac{1}{2} & & & \\ \frac{1}{2} & 0 & \frac{1}{2} & & \\ & \frac{1}{2} & 0 & \frac{1}{2} & \\ & & \ddots & \ddots & \ddots \end{pmatrix} \quad (10.26)$$

and the spectral radius is

$$\rho(G_J) = \cos\left(\frac{\pi}{N}\right). \quad (10.27)$$

As A is a tridiagonal matrix, Young's theorem holds and

$$\rho(G) = \rho(G_J)^2. \quad (10.28)$$

and, for large N , to reduce the error to $1 - \alpha$ times the original error takes

$$k \sim \frac{2N^2}{\pi^2} \ln\left(\frac{1}{1 - \alpha}\right), \quad (10.29)$$

which is exactly twice the number of iterations required by the Gauss-Seidel algorithm. This is why Gauss-Seidel is generally preferred.

10.2.3 AVBD Iterations

In the AVBD method, instead of inverting $\mathcal{D}\mathbf{c} M^{-1}(\mathcal{D}\mathbf{c})^T$, we are inverting H , where

$$H = M + (\mathcal{D}\mathbf{c})^T \rho \Delta t^2 \mathcal{D}\mathbf{c} + \mathbf{c}(\mathbf{X}^{t+1}) \cdot \rho \Delta t^2 \frac{\partial^2 \mathbf{c}}{\partial \mathbf{X}^2}. \quad (10.30)$$

For the straight chain of equal masses this is

$$H = \begin{pmatrix} (m + \rho \Delta t^2)\mathbb{1} & -\rho \Delta t^2 \mathbb{1} & & & \\ -\rho \Delta t^2 \mathbb{1} & (m + 2\rho \Delta t^2)\mathbb{1} & -\rho \Delta t^2 \mathbb{1} & & \\ & -\rho \Delta t^2 \mathbb{1} & (m + 2\rho \Delta t^2)\mathbb{1} & -\rho \Delta t^2 \mathbb{1} & \\ & & & \ddots & \\ & & & & (m + \rho \Delta t^2)\mathbb{1} \end{pmatrix} \quad (10.31)$$

This is a block tridiagonal matrix with additional terms on the boundaries (so the first and last diagonal elements are different from the interior). The boundary differences do not affect the spectral radius for large N so we can use standard result for the spectral radius of the iteration matrix of

$$\rho(G) \sim \left(\frac{2\rho \Delta t^2}{m + 2\rho \Delta t^2}\right)^2 \cos^2\left(\frac{\pi}{1 + N}\right). \quad (10.32)$$

This is the same as for the PBD but with the additional scaling factor

$$\beta = \left(\frac{2\rho \Delta t^2}{m + 2\rho \Delta t^2}\right). \quad (10.33)$$

And now reducing the error to $1 - \alpha$ times the original error requires roughly

$$k \sim \left(\frac{1}{1 - 2\frac{N^2}{\pi^2} \ln \beta}\right) \frac{N^2}{\pi^2} \ln\left(\frac{1}{1 - \alpha}\right) \quad (10.34)$$

In the case of a large stiffness value $\rho \rightarrow \infty$ so $\beta \rightarrow 1$ and this reduces to the PBD result. For smaller stiffness values $\beta < 1$ and the number of required iterations is reduced. This can also be seen by looking at the diagonal dominance of the matrix H

$$r_i = \frac{\sum_{i \neq j} |A_{ij}|}{|A_{ii}|} = \frac{2\rho \Delta t^2}{m + 2\rho \Delta t^2} \quad (10.35)$$

When $m \gg \rho \Delta t^2$, r_i becomes small and $k \rightarrow 0$. This makes sense, as the system consists of weak springs trying to pull together infinite masses, so the solution is that no movement and no iterations are required.

Two remarks are needed to interpret this correctly. First, for any fixed $\beta < 1$ the term $2 \frac{N^2}{\pi^2} \ln \beta$ in the denominator grows with N , so that as $N \rightarrow \infty$

$$k \rightarrow \frac{\ln(1/(1-\alpha))}{-2 \ln \beta}, \quad (10.36)$$

independent of N . The N^2 scaling of PBD is recovered only in the hard constraint limit $\beta \rightarrow 1$. Second, and more importantly, this is the convergence of the *inner* Gauss-Seidel sweep at fixed λ . The augmented Lagrangian method also iterates the multipliers, $\lambda \leftarrow \lambda + \rho \mathbf{c}$, and that outer iteration has its own rate. For the linearized problem with an exact inner solve, one multiplier update contracts the constraint error by the matrix $(\mathbb{1} + \rho \Delta t^2 A)^{-1}$, where $A = \mathcal{D} \mathbf{c} M^{-1} (\mathcal{D} \mathbf{c})^T$ is the PBD matrix. For the equal-mass chain the smallest eigenvalue of A is $(2 - 2 \cos(\pi/N))/m \sim \pi^2/(mN^2)$, so the slowest mode of the outer iteration contracts by

$$\left(1 + \frac{\pi^2 \rho \Delta t^2}{mN^2}\right)^{-1} \sim 1 - \frac{\pi^2 \rho \Delta t^2}{mN^2}, \quad (10.37)$$

i.e. of order $mN^2/(\pi^2 \rho \Delta t^2)$ multiplier updates are needed to remove the long wavelength stretch of the chain. Softening ρ therefore speeds up the inner sweep by the same factor by which it slows down the outer multiplier update: the N^2 is not removed, it is moved from one loop to the other. In the hard limit $\rho \rightarrow \infty$ the outer iteration converges in one step and the inner sweep needs N^2/π^2 iterations, as for PBD. AVBD interleaves one sweep per multiplier update and ramps ρ up over the iterations precisely to manage this trade-off; a convergence analysis of the interleaved iteration is beyond the scope of these notes.

What this does show is that in the hard constraint limit AVBD, like PBD, requires the number of iterations to scale like N^2 in order to solve the chain to the same error, and that away from that limit the cost of the long wavelength modes moves into the multiplier iteration rather than disappearing.

10.3 Effect of Mass Ratios

Consider the matrix (10.7). So far, we have assumed all masses are equal. If we now make the end masses m_1 and m_N different, with $m_i = m$, for $i = 2, \dots, N-1$ and $m_1 = m_N = M$, with $M > m$. This changes the diagonal dominance of the first and last rows

$$r_{\text{ends}} = \frac{\frac{1}{m}}{\frac{1}{m} + \frac{1}{M}} \sim 1, \text{ for } M \gg m. \quad (10.38)$$

This means that all rows now have r either 1 or close to 1. This has a big impact on convergence. If just one end had a light mass then there is at least one row with a smaller r , which eventually allows the system to converge as the error correction from that row can propagate up the chain. With both ends having a heavy mass, the error correction on the ends will only solve a fraction of the error, this then has to propagate back up the chain to the other end, where again a fraction of the error will be solved. This then repeats.

To analyze the convergence of this we start by looking at the Jacobi iteration matrix $G_J = -D^{-1}(L + L^T)$. If we find the spectral radius of this we can then infer the spectral radius of

the Gauss-Seidel iteration matrix using the Young's theorem relation $\rho(G) = \rho(G_J)^2$, which holds for tridiagonal matrices, like we have for the chain example.

Explicitly, setting $m_i = 1$ for simplicity, the matrix $\mathcal{D}\mathbf{c} M^{-1}(\mathcal{D}\mathbf{c})^T$ is

$$A = \mathcal{D}\mathbf{c} M^{-1}(\mathcal{D}\mathbf{c})^T = \begin{pmatrix} 1+\gamma & -1 & & \\ -1 & 2 & -1 & \\ & -1 & \ddots & \\ & & & 1+\gamma \end{pmatrix}, \quad (10.39)$$

where $\gamma = \frac{m}{M}$ is the mass ratio.

If the μ is an eigenvalue of G_J , then

$$A\mathbf{x} = (1 - \mu)D\mathbf{x}. \quad (10.40)$$

This is a generalized eigenvector problem. For $\gamma \rightarrow 0$ the matrix A has an eigenvector $\mathbf{v} = (1 \ 1 \ \dots 1)^T$ with eigenvalue zero. We can use this using a Rayleigh quotient to find an approximate eigenvalue λ closest to zero, which will give the dominant eigenvalue of G_J $\mu = 1 - \lambda$ which is the eigenvalue closest to 1.

$$\begin{aligned} \lambda &\sim \frac{\mathbf{v}^T A \mathbf{v}}{\mathbf{v}^T D \mathbf{v}}, \\ &\sim \frac{2\gamma}{2\gamma + 2 + 2(N-3)}, \\ &\sim \frac{\gamma}{N} \text{ for large } N \end{aligned} \quad (10.41)$$

so

$$\mu \sim 1 - \frac{\gamma}{N} \quad (10.42)$$

and the spectral radius for the Gauss-Seidel iteration matrix is

$$\begin{aligned} \rho(G) &\sim (1 - \frac{\gamma}{N})^2, \\ &\sim 1 - \frac{2\gamma}{N}. \end{aligned} \quad (10.43)$$

Using this, the number of iterations required to reduce the error by a factor of $1 - \alpha$ is for small γ

$$k \sim \frac{N}{2\gamma} \ln \left(\frac{1}{1 - \alpha} \right). \quad (10.44)$$

So, say you have a chain of length $N = 10$, for a mass ratio of 1 we have the previous result $k \sim 10 \ln(1/(1 - \alpha))$, and with a mass ratio of 100 this will be $k \sim 500 \ln(1/(1 - \alpha))$, so a mass ratio of 100 means having to increase the number of iterations by $\sim 50\times$ to solve for the same correction to the initial error.

10.3.1 Relaxation

Following the same procedure from the previous section §10.2.1 we can find an optimal relaxation value in this case of large masses at either end

$$\begin{aligned} \xi^* &= \frac{2}{1 + \sqrt{1 - \rho(G)}}, \\ &\sim \frac{2}{1 + \sqrt{2\gamma/N}}. \end{aligned} \quad (10.45)$$

With this, the optimal spectral radius is

$$\rho(G_\xi^*) \sim 1 - 2\sqrt{\frac{2\gamma}{N}} \quad (10.46)$$

and the number of iterations goes like

$$k \sim \frac{1}{2}\sqrt{\frac{N}{2\gamma}} \ln\left(\frac{1}{1-\alpha}\right). \quad (10.47)$$

This is an improvement over the non-relaxed result, going from a N/γ scaling to a $\sqrt{N/\gamma}$ scaling. For example, with a chain length of 10 and a mass ratio of 100 the iterations now go like $k \sim 11 \ln(1/(1-\alpha))$ compared to $k \sim 500 \ln(1/(1-\alpha))$ for no relaxation.

10.3.2 AVBD Iterations

For AVBD, setting the two end masses to M makes the matrix H become

$$H = \begin{pmatrix} (M + \rho\Delta t^2)\mathbb{1} & -\rho\Delta t^2\mathbb{1} & & & \\ -\rho\Delta t^2\mathbb{1} & (m + 2\rho\Delta t^2)\mathbb{1} & -\rho\Delta t^2\mathbb{1} & & \\ & -\rho\Delta t^2\mathbb{1} & (m + 2\rho\Delta t^2)\mathbb{1} & -\rho\Delta t^2\mathbb{1} & \\ & & & \ddots & \\ & & & & (M + \rho\Delta t^2)\mathbb{1} \end{pmatrix} \quad (10.48)$$

The effect of this is to improve the diagonal dominance for the first and last constraint rows

$$\begin{aligned} r_{\text{ends}} &= \frac{\rho\Delta t^2}{M + \rho\Delta t^2}, \\ &\sim \frac{\rho\Delta t^2}{M} \quad \text{for large } M \end{aligned} \quad (10.49)$$

Proceeding using the Rayleigh quotient method used in the PBD case, we find

$$\rho(G) \sim \left(\frac{\rho\Delta t^2 N}{M + \rho\Delta t^2 N} \right)^2 \quad (10.50)$$

and

$$k \sim \frac{\rho\Delta t^2 N}{2M} \ln\left(\frac{1}{1-\alpha}\right), \quad (10.51)$$

where the assumption that N is large but $\rho\Delta t^2/M$ is not as large. In this range the number of inner sweep iterations at fixed λ scales with N instead of N^2 : heavy end bodies make the inner solve easier, the opposite of PBD. Two caveats apply. This holds only while the end mode is the slowest one, i.e. while $\rho\Delta t^2 \lesssim m$ so that the interior mode $\beta^2 \cos^2(\pi/N)$ of the previous section does not dominate; and the outer multiplier iteration is unaffected by M , since the end constraint's effective mass is $1/m + 1/M \sim 1/m$, and it retains the $O(N^2 m/(\rho\Delta t^2))$ long wavelength mode found above.

10.4 Results

These results are for the simplified chain, where all of the bodies are in a straight line and the angular terms vanish.

For hard constraints both AVBD and PBD require the iteration count to scale with the square of the number of bodies in the chain in order to correct the same error. AVBD can soften the constraint resolution through the ρ parameter, which makes the inner Gauss-Seidel sweep converge in an N -independent number of iterations at the cost of moving the long wavelength stretch into the outer multiplier iteration, which then needs $O(N^2 m / (\rho \Delta t^2))$ updates. Using SOR can greatly improve the convergence of Gauss-Seidel, but finding the optimal relaxation parameter requires prior knowledge and analysis of the constraint matrix.

When large masses are added to either end of the chain the convergence of PBD is greatly affected. For large mass ratio $1/\gamma$, Gauss-Seidel iterations scale by N/γ , meaning the mass ratio is directly proportional to the required number of iterations. AVBD's inner sweep does better in this scenario in the regime $\rho \Delta t^2 / M \sim 1$, scaling with N ; its multiplier iteration is unaffected by the heavy ends.

11 Summary

We first went through a derivation of position based dynamics and showed the following

- Position Based Dynamics can be derived as a first order approximate solution to the constrained rigid body equations of motion
- This is equivalent to solving the constraint equations at first order subject to a least squares minimization over the metric defined by the mass matrix M

The convergence section proved the following

- For a symmetric mass matrix, the Gauss-Seidel solution for the PBD rigid body system is guaranteed to converge.
- The rate of convergence depends on the spectral radius of the Gauss-Seidel update matrix, and can be estimated by the diagonal dominance of the matrix $\mathcal{D}\mathbf{c} M^{-1}(\mathcal{D}\mathbf{c})^T$

Next, we went through finding a first order solution to the general equations of motion for non-linear forces and showed:

- Solving the linearized equations of motion for forces instead of constraints is equivalent to the Vertex Block Descent method where the iteration loop is over bodies instead of constraints.
- XPBD swaps constraints with Lagrange multipliers for compliant constraints implemented through potential penalty functions
- This naturally fits into the VBD framework, but XPBD transforms the problem into a constraint based update through a change of variables
- The XPBD update is equivalent to the PBD least squares minimization but with the addition on a regularization term proportional to the inverse of the spring stiffness.

Next, we went through the Augmented Lagrangian method and showed:

- The Augmented Lagrangian method mixes both the constraint penalty method and the Lagrange multiplier method, supporting both stiff and compliant constraints
- AVBD is a flavour of this that approximates the Hessian term by taking only the diagonal. This ensures convergence.
- The Hessian term is known as geometric stiffness, and can be included in all position based methods as a modification to the mass matrix.

Finally, the convergence section went through the convergence of a simple chain of bodies and showed

- For chains with uniform mass and hard constraints both PBD and AVBD require the number of iterations to scale with the number of bodies squared for convergence; softening the AVBD penalty moves this cost from the Gauss-Seidel sweep to the multiplier iteration rather than removing it.
- Successive Over Relaxation can greatly improve convergence. An optimal SOR parameter can reduce the convergence scaling to N instead of N^2 .
- The body based iteration in VBD, AVBD and force based update methods changes how mass ratios affect convergence: heavy bodies improve the convergence of the sweep rather than degrading it as in PBD, while light bodies coupled by stiff penalties remain the hard case.

References

- [1] Sheldon Andrews, Marek Teichmann, and Paul G. Kry. Geometric stiffness for real-time constrained multibody dynamics. *Computer Graphics Forum*, 36(2):235–246, 2017.
- [2] David Baraff. An introduction to physically based modeling: Rigid body simulation. *SIGGRAPH Course Notes*, 1997.
- [3] David Baraff. Rigid body simulation ii: Nonpenetration constraints. *SIGGRAPH Course Notes*, 1999.
- [4] Joao Barata and Mahir Hussein. The moore-penrose pseudoinverse. a tutorial review of the theory. *Brazilian Journal of Physics*, 42, 10 2011.
- [5] Timothy D. Barfoot. *State Estimation for Robotics*. Cambridge University Press, Cambridge, UK, 2017.
- [6] Joachim Baumgarte. Stabilization of constraints and integrals of motion in dynamical systems. *Computer Methods in Applied Mechanics and Engineering*, 1(1):1–16, 1972.
- [7] Jan Bender, Matthias Müller, and Miles Macklin. A survey on position based dynamics, 2017. In *Eurographics 2017 – Tutorials*. The Eurographics Association, 2017.
- [8] Dennis S. Bernstein, Ankit Goel, and Omran Kouba. Deriving euler’s equation for rigid-body rotation via lagrangian dynamics with generalized coordinates. *Mathematics*, 11(12):2727, June 2023.
- [9] Anthony M. Bloch. *Nonholonomic Mechanics and Control*, volume 24 of *Interdisciplinary Applied Mathematics*. Springer, New York, 2003.
- [10] Erin Catto. Iterative dynamics with temporal coherence. In *Game Developers Conference (GDC)*, 2005.
- [11] Anka He Chen, Ziheng Liu, Yin Yang, and Cem Yuksel. Vertex block descent. *ACM Transactions on Graphics*, 43(4):1–16, 2024.
- [12] Chris Doran and Anthony Lasenby. *Geometric Algebra for Physicists*. Cambridge University Press, Cambridge, UK, 2003.
- [13] Kenny Erleben. *Game Physics Engine Development*. CRC Press, Boca Raton, FL, 2 edition, 2010.
- [14] Chris Giles, Elie Diaz, and Cem Yuksel. Augmented vertex block descent. *ACM Transactions on Graphics*, 44(4):1–12, July 2025.
- [15] Herbert Goldstein, Charles P. Poole, and John L. Safko. *Classical Mechanics*. Addison-Wesley, San Francisco, 3 edition, 2002.
- [16] David Hestenes. *New Foundations for Classical Mechanics*, volume 15 of *Fundamental Theories of Physics*. Reidel / Kluwer Academic Publishers, Dordrecht, Netherlands, 1986. Second revised edition published 1999, ISBN 978-0792355144.
- [17] Magnus R. Hestenes. Multiplier and gradient methods. *Journal of Optimization Theory and Applications*, 4(5):303–320, 1969.

- [18] Miles Macklin, Kenny Erleben, Matthias Müller, Nuttapong Chentanez, Stefan Jeschke, and Tae-Yong Kim. Primal/dual descent methods for dynamics. *Computer Graphics Forum*, 39(8):89–100, 2020.
- [19] Miles Macklin, Kenny Erleben, Matthias Müller, Nuttapong Chentanez, Stefan Jeschke, and Viktor Makoviychuk. Non-smooth Newton methods for deformable multi-body dynamics. *ACM Transactions on Graphics*, 38(5):140:1–140:20, 2019.
- [20] Miles Macklin, Matthias Müller, and Nuttapong Chentanez. Xpbd: Position-based simulation of compliant constrained dynamics. *ACM SIGGRAPH Motion in Games*, 2016.
- [21] Miles Macklin, Kier Storey, Michelle Lu, Pierre Terdiman, Nuttapong Chentanez, Stefan Jeschke, and Matthias Müller. Small steps in physics simulation. In *Proceedings of the 18th Annual ACM SIGGRAPH/Eurographics Symposium on Computer Animation*, SCA '19, Los Angeles, CA, USA, 2019. ACM.
- [22] J. E. Marsden and M. West. Discrete mechanics and variational integrators. *Acta Numerica*, 10:357–514, 2001.
- [23] Ian Millington. *Game Physics Engine Development*. Morgan Kaufmann, 2007.
- [24] Matthias Müller, Miles Macklin, Nuttapong Chentanez, Stefan Jeschke, and Tae-Yong Kim. Detailed rigid body simulation with extended position based dynamics. *Computer Graphics Forum*, 39(8):101–112, 2020. Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Computer Animation (SCA 2020).
- [25] Matthias Müller, Bruno Heidelberger, Marcus Hennix, and John Ratcliff. Position Based Dynamics. In Cesar Mendoza and Isabel Navazo, editors, *Vriphys: 3rd Workshop in Virtual Reality, Interactions, and Physical Simulation*. The Eurographics Association, 2006.
- [26] Roger Penrose. A generalized inverse for matrices. *Proceedings of the Cambridge Philosophical Society*, 51:406–413, 01 1955.
- [27] M. J. D. Powell. A method for nonlinear constraints in minimization problems. *Optimization*, pages 283–298, 1969.
- [28] J.-P. Ryckaert, G. Ciccotti, and H. J. C. Berendsen. Numerical integration of the cartesian equations of motion of a system with constraints: molecular dynamics of n -alkanes. *J. Comput. Phys.*, 23:327–341, 1977.
- [29] Morten Silcowitz, Sarah Niebe, and Kenny Erleben. Projected gauss–seidel subspace minimization method for interactive rigid body dynamics. In *Proceedings of the International Conference on Computer Graphics Theory and Applications (GRAPP)*. SciTePress, 2010.
- [30] Joan Solà, Jérémie Deray, and Dinesh Atchuthan. A micro Lie theory for state estimation in robotics, 2018.
- [31] David E. Stewart. Rigid-body dynamics with friction and impact. *SIAM Review*, 42(1):3–39, 2000.
- [32] A Tikhonov and V Arsenin. *Solution of Ill-Posed Problem*. 01 1977.

- [33] David Tong. Rigid body dynamics. <https://www.damtp.cam.ac.uk/user/tong/dynamics/three.pdf>, 2019. Lecture notes, Department of Applied Mathematics and Theoretical Physics, University of Cambridge.
- [34] Richard Tonge, Feodor Benevolenski, and Andrey Voroshilov. Mass splitting for jitter-free parallel rigid body simulation. *ACM Transactions on Graphics*, 31(4):105:1–105:8, 2012.
- [35] Maxime Tournier, Matthieu Nesme, Benjamin Gilles, and François Faure. Stable constrained dynamics. *ACM Transactions on Graphics*, 34(4):132:1–132:10, 2015. Proceedings of ACM SIGGRAPH 2015.
- [36] Lloyd N. Trefethen and David Bau III. *Numerical Linear Algebra*. SIAM, Philadelphia, PA, USA, 1997.
- [37] Richard S. Varga. *Matrix Iterative Analysis*. Springer, New York, 2 edition, 2000.
- [38] Anthony Zee. *Group Theory in a Nutshell for Physicists*. In a Nutshell. Princeton University Press, 2016.